

Do Transformer ao Agente

Um livro cronológico e acessível
sobre modelos de linguagem, RAG, agentes e
ferramentas

*Modelo de linguagem + contexto + recuperação + ferramentas
+ ambiente + memória + iteração + verificação*

August 13, 2026

Como este livro foi escrito

Este livro foi produzido com **vibe coding**: uma escrita colaborativa e iterativa entre um humano e um modelo de linguagem, com curadoria e revisão humanas ao longo de todo o processo. Cada capítulo seguiu o ciclo que o próprio livro ensina --- objetivo, estado, informação, hipótese, ação, observação, verificação e iteração.

A ideia deste livro nasceu de uma inquietação: acompanhar a evolução acelerada dos modelos de linguagem pós-Transformer, em especial o momento atual dos sistemas *agentic*. Nada ilustrava essa trajetória de ponta a ponta --- do Transformer ao agente --- de forma cronológica e prática, e este livro tenta preencher essa lacuna.

Ele foi sugerido e conduzido por **Fábio Vila**, que pode ser encontrado no Telegram (<https://t.me/viladx2>) e no GitHub (<https://github.com/fabiovila>).

Aqui, dizer isso sem rodeios faz parte do método: o valor do livro não está na autoria exclusiva, mas no processo que o produziu. Ser honesto sobre como ele foi feito é coerente com o que ele defende.

Fábio Vila

Telegram: <https://t.me/viladx2>

GitHub: <https://github.com/fabiovila>

Do Transformer ao Agente

Um livro temático e cronológico, acessível, sobre modelos de linguagem, RAG, agentes e ferramentas

Sobre o livro

Este livro conta, em ordem cronológica e temática, a história dos sistemas modernos baseados em modelos de linguagem:

- os fundamentos da modelagem de linguagem que precederam o Transformer;
- a arquitetura Transformer e a ascensão dos modelos pré-treinados (BERT, GPT);
- a escala, a emergência e a descoberta dos poucos exemplos (few-shot);
- o nascimento e a evolução da geração aumentada por recuperação (RAG);
- o alinhamento por instrução humana e o surgimento do ChatGPT;
- a aprendizagem de uso de ferramentas (tool use) e o *function calling*;
- a era dos agentes, dos frameworks e dos sistemas multi-agente;
- os protocolos de interoperabilidade (MCP, A2A);
- e a síntese do "loop cognitivo" que liga tudo isso.

Não é uma história apenas para especialistas. Cada capítulo introduz os conceitos necessários antes de avançar, sempre partindo do princípio didático:

Não basta saber a resposta. É preciso saber **qual processo produz uma resposta confiável**.

O leitor acompanhará não só *o que* foi criado, mas *por que* cada inovação foi uma resposta a uma limitação concreta da abordagem anterior — e *como* essas peças se combinam hoje em sistemas de produção.

Estrutura do livro

Parte	Era	Período	Capítulos
00	Prefácio	—	como ler este livro
01	Era fundacional	~1950-2020	1-4
02	Era do RAG clássico	2020-2021	5-6
03	Era do alinhamento	2021-2023	7-8
04	Era das ferramentas	2021-2023	9-11
05	Era do RAG como sistema	2023-2025	12
06	Era dos agentes	2022-2025	13-15
07	Era dos protocolos	2024-2026	16-17

Parte	Era	Período	Capítulos
08	Síntese	—	18-19

As partes seguem a cronologia em que cada estrutura *amadureceu*: o modelo (2020) → o RAG clássico (2020-2021) → o alinhamento (2021-2023) → as ferramentas (2021-2023) → o RAG como sistema (2023-2025) → os agentes (2022-2025) → os protocolos (2024-2026). Os períodos se sobrepõem porque as estruturas coexistiram e evoluíram juntas — por isso algumas faixas se cruzam. Uma técnica também pode preceder sua era (o RLHF é de 2017, antes do RAG de 2020).

Cada capítulo segue um formato didático fixo:

1. **Abertura em prosa** — situa o capítulo na história: o problema que a era enfrentava, o *porquê* e o que se ambicionava. Se você só puder ler uma seção, leia esta.
2. **Seções temáticas** — o corpo; cada seção desenvolve uma estrutura ou ideia, define o jargão na primeira ocorrência e usa diagramas em ASCII.
3. **A lição estrutural** — seção final do corpo que sintetiza a herança conceitual da era; presente em todos os capítulos, sempre com o mesmo nome.
4. **Para o engenheiro** — quadro final colorido com os takeaways práticos: decisões, armadilhas e o que levar para o projeto real.
5. **Fontes** — a linha **Fontes:** com as referências que sustentam o capítulo, no formato [Autor, ano] – descrição curta; a bibliografia completa está em fontes.md.

Estado do livro

- ☒ Pesquisa inicial e linha do tempo (fontes verificadas em fontes.md)
- ☒ Estrutura de capítulos e arquivos de gestão
- ☒ Prefácio (cap. 0)
- ☒ Capítulos 1-6 (Era fundacional + RAG clássico)
- ☒ Capítulos 7-8 (Era do alinhamento)
- ☒ Capítulos 9-11 (Era das ferramentas)
- ☒ Capítulo 12 (RAG avançado/modular)
- ☒ Capítulos 13-15 (Era dos agentes)
- ☒ Capítulos 16-17 (Era dos protocolos)
- ☒ Capítulos 18-19 (Síntese)
- ☒ Glossário — ampliado com "Por que importa" e conceitos novos (MCP, A2A, LoRA, tool use, avaliação)
- ☒ Revisão estilística geral — fechamento único ("Para o engenheiro"), formato único de fontes, aspas curvas
- ☐ Revisão cruzada de datas e fontes
- ☐ Consolidação e formatação final

Arquivos do projeto

Arquivo	Propósito
README.md	este arquivo (visão geral e estado)
SUMARIO.md	sumário detalhado do livro
cronologia.md	linha do tempo dos marcos com datas e fontes
fontes.md	bibliografia consolidada com links
NOTAS.md	instruções de manutenção para futuras sessões
capitulos/...	conteúdo do livro por era

Princípios editoriais

1. **Cronológico:** a ordem é a da história, e cada capítulo aponta o que veio antes e depois.
2. **Didático:** conceitos são introduzidos do zero; jargão técnico é definido na primeira ocorrência.
3. **Verificável:** toda afirmação factual aponta para uma fonte em fontes.md.
4. **Estrutural:** não se trata de listar modelos, mas de ensinar as **estruturas** que se repetem (escala, recuperação, loop de agente, verificação).
5. **Sistemático:** o livro assume que inteligência efetiva emerge de modelo + contexto + retrieval + ferramentas + ambiente + memória + iteração + verificação.

Contents

Do Transformer ao Agente	ii
Um livro temático e cronológico, acessível, sobre modelos de linguagem, RAG, agentes e ferramentas	ii
Sobre o livro	ii
Estrutura do livro	ii
Estado do livro	iii
Arquivos do projeto	iii
Princípios editoriais	iv
 Prefácio	 2
 Capítulo 0 — Como ler este livro	 2
Contexto histórico	2
Ideia central	2
Conteúdo didático	2
A tese em uma figura	2
A pergunta que orienta tudo	3
Como usar cada seção dos capítulos	3
Convenções	3
 Parte I — Era fundacional (≈1950-2020)	 5
 Capítulo 1 — Antes do Transformer: a longa busca por uma máquina que fale	 5
Uma linha do tempo de tentativas	5
O que significa, afinal, “modelar linguagem”?	5
A era das regras: quando a linguagem era gramática	6
A era das contagens: estatística contra a maldição	6
A era neural: palavras viram vetores	6
Redes recorrentes e o drama da memória	7
Seq2seq e a semente da atenção	7
O que ainda faltava em 2016	7
A lição estrutural	8
 Capítulo 2 — Attention Is All You Need: o nascimento do Transformer (2017)	 9
Por que descartar a recorrência?	9
A atenção como operação central	9
Muitas cabeças, muitas perguntas	10
A ordem precisa voltar	11
Uma arquitetura com duas metades	11
Três linhagens para o futuro	12
O que o Transformer mudou de fato	12

A lição estrutural	12
Capítulo 3 — Pré-treinamento: ELMo, BERT e GPT (2018-2019)	14
Pré-treinar para depois adaptar	14
ELMo: a palavra ganha dependência do contexto	15
BERT: entender olhando para os dois lados	15
GPT: gerar como objetivo universal	15
GPT-2: quando prever texto virou multitarefa	15
A lição estrutural	16
Por que isso importa para RAG e agentes	16
Capítulo 4 — Escala e emergência: GPT-3 e as leis de escala (2020-2022)	18
A matemática da escala	18
GPT-3 e a tarefa como contexto	18
Capacidades emergentes	19
A infraestrutura que sustentou a escala	19
O que a escala deixou de herança	19
A lição estrutural	20
Parte II — Era do RAG clássico (2020-2021)	22
Capítulo 5 — As origens do Retrieval-Augmented (2020): a memória fora dos pesos	22
Memória paramétrica e não-paramétrica	22
A base lexical: o padrão BM25	22
DPR: o retriever denso	23
REALM e kNN-LM: integrações alternativas	23
Além da busca vetorial simples	23
A lição estrutural	24
Capítulo 6 — RAG clássico: Lewis et al. (2020) e as primeiras arquiteturas	25
A arquitetura básica e o treinamento de ponta a ponta	25
A fronteira entre recuperação e raciocínio	26
Variações estruturais: FiD e RETRO	26
FiD: a fusão que acontece no decoder	26
RETRO: memória externa como eixo de escala	27
O vocabulário que permanece	27
A lição estrutural	28
Parte III — Era do alinhamento (2021-2023)	30
Capítulo 7 — De GPT-3 a ChatGPT: instrução, RLHF e o momento em que o público chegou	30
A origem do RLHF	31
Da preferência humana ao InstructGPT	31
A matemática do alinhamento	32
O que o alinhamento muda — e o que não muda	32
Chain-of-Thought: quando o prompt também muda o comportamento	33
LaMDA e a ideia de diálogo fundamentado	33
ChatGPT: quando o modelo virou produto	34
As limitações definem a próxima geração	34
A lição estrutural	35

Capítulo 8 — Modelos abertos e a corrida (2023)	37
Um ecossistema que deixou de ter um único centro	37
LLaMA: quando os pesos chegam à comunidade — e a arquitetura favorece o uso	38
LLaMA 2/3: da pesquisa à adoção	38
Mistral e BLOOM: alternativas ao caminho dominante	38
Síntese das mudanças arquiteturais e de inferência	39
LoRA: adaptar sem recomençar	39
O contraponto: GPT-4	40
A segunda metade da história: quando o modelo se torna plataforma	40
A relação com RAG, ferramentas e agentes	41
O que significa “aberto”?	41
O que a abertura muda para o engenheiro	42
A lição estrutural	42
 Parte IV — Era das ferramentas (2021-2023)	 45
Capítulo 9 — Do WebGPT ao Toolformer: como o modelo ganhou mãos	45
WebGPT: o primeiro modelo que agiu em um ambiente	45
MRKL: roteamento neuro-simbólico	46
PAL: raciocínio via código	46
Toolformer: o modelo que aprende sozinho quando chamar	47
Dois caminhos: dizer como ou aprender quando	47
HuggingGPT: o LLM como controlador	48
A lição estrutural	48
 Capítulo 10 — ReAct e o loop razão-ação (2022)	50
O problema que a ação única não resolvia	50
O padrão	50
Raciocínio e ação se alimentam	51
Os números	51
Uma forma, não um framework	51
Aprender com o feedback do loop	52
Os limites do loop	52
A lição estrutural	52
 Capítulo 11 — Function calling, structured outputs e computer use (2023-2024)	54
ChatGPT plugins: o primeiro ecossistema em escala de produto	54
Function calling: a ferramenta vira primitiva de API	54
Expansão e divergência: três provedores, três formatos	55
Medir para industrializar	55
Structured Outputs: o fim do JSON inválido	55
Computer use: quando a ferramenta é a tela inteira	56
Convergência: do parâmetro de API ao protocolo	56
A lição estrutural	57
 Parte V — Era do RAG como sistema (2023-2025)	 59
Capítulo 12 — De RAG ingênuo a RAG modular e agêntico (2023-2025)	59
Por que “recuperar e colar” erra: o RAG ingênuo	59
Advanced RAG I: pré-processamento e chunking	60

Advanced RAG II: indexação e recuperação	60
Modular RAG: o pipeline vira sistema	61
Aprendendo a recuperar: Self-RAG, FLARE, RAFT, IRCOT	62
Estruturando a memória: RAPTOR e GraphRAG	62
RAG versus contexto longo: o trade-off que não some	63
Agentic RAG: recuperação como ferramenta no loop	64
A lição estrutural	64
Parte VI — Era dos agentes (2022-2025)	67
Capítulo 13 — Frameworks de agentes: de AutoGPT a LangGraph	67
A explosão de 2023: AutoGPT e BabyAGI	67
O que é um agente, afinal	67
De cadeias a grafos: LangChain e LangGraph	68
Equipes, conversas e artefatos	68
Planejamento além do linear	69
A lição estrutural	69
Capítulo 14 — Sistemas multi-agente e simulação social	71
A conversa como método: CAMEL	71
A vila que acordou: Generative Agents	71
A empresa que produz software: ChatDev	72
Recrutar, decidir, executar e avaliar: AgentVerse	72
Chatter: o custo invisível	72
De conversas a artefatos: MetaGPT	73
A aritmética da coordenação	73
A lição estrutural	73
Capítulo 15 — Avaliação de agentes: o que medir e por que é difícil	75
O laboratório dos oito ambientes: AgentBench	75
A internet e o desktop de verdade: WebArena e OSWorld	75
O usuário que responde: tau-bench e a consistência	76
Métricas de processo, não só de resposta	76
LLM-as-judge: barato, viesado, inevitável	76
A lição estrutural	77
Parte VII — Era dos protocolos (2024-2026)	79
Capítulo 16 — MCP: o USB-C dos dados e ferramentas	79
O problema das N×M integrações	79
As primitivas: tools, resources e prompts	79
Adoção relâmpago e governança	80
MCP e agentes: a camada vertical do mundo	80
Os limites que ninguém deveria ignorar	80
A lição estrutural	81
Capítulo 17 — A2A e a interoperabilidade entre agentes	82
Quando o mundo é outro agente	82
Horizontal e vertical: A2A versus MCP	83
Adoção e governança: de 50 parceiros à Linux Foundation	83
A evolução 0.3: gRPC e segurança enterprise	83

O cenário 2025–2026: dois padrões, uma mesma forma	83
A lição estrutural	84

Parte VIII — Síntese 86

Capítulo 18 — O loop cognitivo: RAG + agentes + ferramentas como um só sistema 86

Um só padrão, quatro ângulos	86
A tabela de equivalências: ferramentas como extensões cognitivas	87
RAG não é só retrieval: é construção de evidência	87
Geração não é validação	87
O projeto didático: pergunta → evidência → ação → verificação	88
A lição estrutural	88

Capítulo 19 — Avaliação, limites e o horizonte 90

O elo fraco: a consistência entre tentativas	90
Segurança e reversibilidade: a escala das ações	90
Custo e o critério de parada	91
O horizonte 2026+	91
As perguntas que o campo ainda não respondeu	91
A lição estrutural	92
A pergunta final	92

Anexo A — Cronologia 95

Cronologia 95

Sementes (1950–2016)	95
O Transformer e o pré-treinamento (2017–2019)	95
Escala, emergência e as leis de escala (2020–2022)	95
Nascimento do RAG (2020–2022)	96
Alinhamento e o momento ChatGPT (2021–2022)	96
Tool use: a pesquisa (2021–2023)	96
Tool use: a indústria (2023–2024)	96
RAG avançado (2023–2025)	97
Agentes (2023–2025)	97
Protocolos (2024–2026)	97
Observação de método	98

Anexo B — Glossário 100

Glossário 100

Fundamentos de modelos de linguagem	100
RAG e recuperação	101
Alinhamento e ferramentas	102
Agentes e avaliação	103
Protocolos e sistema	103

Anexo C — Fontes 105

Fontes 105

Modelagem de linguagem e redes neurais (fundamentos)	105
--	-----

O Transformer e o pré-treinamento	105
Escala e emergência	105
RAG — papers fundadores	106
RAG — surveys e avanços (2023–2025)	106
Alinhamento e produtos	106
Tool use — pesquisa	107
Tool use — indústria e APIs	107
Agentes	108
Protocolos e interoperabilidade	108
Visões de conjunto (história e survey de LLMs)	109
Critério de inclusão e verificação	109

Anexo D — NOTAS: manutenção do livro **111**

NOTAS — Manutenção do livro	111
Como este livro é gerido	111
Formato dos capítulos	111
Regras de escrita	111
Workflow de escrita	111
Verificação de fatos	112
Contexto e ferramentas	112
Próximos passos sugeridos	112

Prefácio

Capítulo 0 — Como ler este livro

Objetivo

Todo livro de IA tem data de validade — este tenta escapar dessa regra. Em vez de catalogar o GPT-n da vez, ele narra as **estruturas que se repetem** ao longo da história (escala, recuperação, loop de agente, verificação) e como cada era as combinou de um jeito novo. Antes de mergulhar, o leitor precisa de duas coisas: a tese central — *um modelo de linguagem é o núcleo de um sistema maior, não um oráculo* — e um mapa: as eras cronológicas e as seções que organizam cada capítulo. Este capítulo entrega as duas, e avisa que o resto do livro vai cobrar uma postura desconfortável: **verificar** o que se aprende, em vez de apenas acreditar.

Contexto histórico

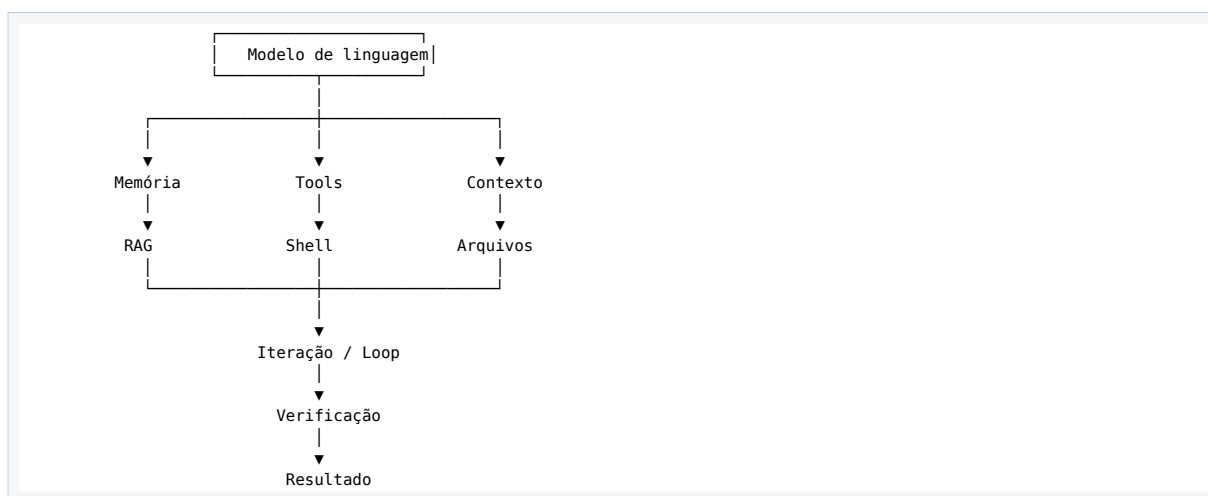
Livros técnicos sobre “inteligência artificial” costumam envelhecer rápido. Este livro tenta envelhecer devagar: em vez de catalogar modelos (GPT-n, LLaMA-n), ele narra **as estruturas que se repetem** ao longo da história — escala, recuperação, loop de agente, verificação — e mostra como cada era as combinou de forma nova.

Ideia central

Um modelo de linguagem não é apenas um sistema que recebe texto e produz texto. Ele é o **núcleo de inferência** de um sistema maior, e a inteligência efetiva desse sistema emerge da combinação de modelo, contexto, recuperação, ferramentas, ambiente, memória, iteração e verificação.

Conteúdo didático

A tese em uma figura



Ao longo do livro, cada parte desta figura é **inventada ou amadurecida em um período histórico**:

- o **modelo de linguagem** ganha forma com o Transformer (2017) e a escala (2020);
- o **contexto** vira engenharia deliberada com o pré-treinamento e o *in-context learning*;
- a **memória externa** vira RAG (2020);

- as **ferramentas** viram função de primeira classe com o *function calling* (2023);
- o **loop** vira agente com ReAct (2022) e os frameworks de agentes (2023);
- a **verificação** vira disciplina com a avaliação de agentes (2023–2025).

As eras se sobrepõem. As partes seguem a cronologia em que cada estrutura *amadureceu* — modelo (2020), RAG clássico (2020–2021), alinhamento (2021–2023), ferramentas (2021–2023), RAG como sistema (2023–2025), agentes (2022–2025), protocolos (2024–2026). Os períodos se sobrepõem porque as estruturas coexistiram e evoluíram juntas: o alinhamento e as ferramentas são contemporâneos (2021–2023), e a era dos agentes (2022–2025) cruza a do RAG como sistema (2023–2025). Não se leia essas faixas como fronteiras — a ordem do livro é a ordem em que cada estrutura amadurece, não a única linha do tempo possível. As estruturas não morrem quando a próxima era começa: elas se combinam, e o capítulo 18 junta todas. E o inverso também vale: uma técnica pode ser bem mais antiga do que a era em que aparece. O RLHF (2017) precedeu o próprio RAG (2020) e só amadureceu para linguagem em 2022; a recuperação de informação existe desde os anos 1970 e virou RAG em 2020. A era não marca o nascimento da técnica, mas o momento em que ela amadureceu aplicada a modelos de linguagem.

A pergunta que orienta tudo

Em cada era, há uma pergunta de fundo. Não pergunte apenas:

“O modelo consegue fazer isso?”

Pergunte:

“Que combinação de modelo, contexto, ferramentas, ambiente, memória, feedback e verificação torna isso possível de maneira confiável?”

Como usar cada seção dos capítulos

1. **Abertura em prosa** — situa o capítulo na história: o problema que a era enfrentava, o *porquê* e o que se ambicionava. Se você só puder ler uma seção, leia esta.
2. **Seções temáticas** — o corpo do capítulo; cada seção desenvolve uma estrutura ou uma ideia, define o jargão técnico na primeira ocorrência e usa diagramas em ASCII quando ajudam.
3. **A lição estrutural** — a seção final do corpo sintetiza o que a era deixa de herança conceitual; presente em todos os capítulos, sempre com o mesmo nome.
4. **Para o engenheiro** — quadro final colorido com os takeaways práticos: decisões, armadilhas e o que levar para o projeto real.
5. **Fontes** — a linha **Fontes:** com os artigos que sustentam o capítulo, no formato [Autor et al., ano] – descrição curta; as referências completas estão em fontes.md.

Convenções

- **Notação:** [Autor et al., ano] remete à bibliografia em fontes.md.
- **Diagramas:** ASCII simples; desenhe mentalmente antes de avançar.
- **Recuos didáticos:** cada capítulo presume apenas os capítulos anteriores.

Para o engenheiro

- Antes de escolher um modelo, desenhe o **sistema**: no seu problema, o que é memória (banco, índice), o que é ferramenta (API, calculadora, shell) e o que é contexto (prompt, documentos).
- Separe sempre “o que o modelo sabe” (pesos) de “o que o sistema pode consultar ou executar”. É essa distinção que decide onde RAG e ferramentas entram.
- Use as eras como mapa mental de decisão: knowledge dinâmico → recuperação; cálculo/ação → ferramentas; tarefa multi-passo → loop de agente.
- Se só puder ler uma seção de um capítulo, leia a **abertura**; se for implementar, leia o **Para o engenheiro**.

Fontes: Este capítulo sintetiza a orientação do arquivo AGENTS.md deste repositório, que define a perspectiva operacional do livro. Referências históricas para as peças citadas (Transformer, RAG, function calling, ReAct, avaliação de agentes) estão em fontes.md e serão detalhadas nos capítulos correspondentes.

Parte I — Era fundacional (≈1950-2020)

Capítulo 1 — Antes do Transformer: a longa busca por uma máquina que fale

Antes de 2017, “fazer a máquina falar” já era uma obsessão de décadas — e uma coleção de fracassos elegantes. Este capítulo não começa no Transformer de propósito: para entender por que aquele ano foi o ano zero, é preciso caminhar pelo que veio antes, pelo que cada abordagem resolveu e, sobretudo, pelo que cada uma deixou em aberto. A pergunta que atravessa tudo é simples e cruel: por que uma máquina não entende “a casa é azul”? As respostas da época — primeiro regras, depois contagens, depois vetores — formam a pré-história dos grandes modelos de linguagem, e cada uma delas ilumina um pedaço do problema enquanto revela outro. A ambição nunca foi escrever um dicionário; era comprimir a linguagem em algo que um computador pudesse carregar, manipular e, de alguma forma, generalizar. O capítulo termina em 2016, com três portas entreabertas — portas que o Transformer vai arrombar de uma só vez.

Uma linha do tempo de tentativas

A história dos modelos de linguagem modernos se estende muito antes de qualquer arquitetura profunda. Para sentir o peso do que o Transformer representou, vale olhar para trás e perceber quantas peças precisaram ser inventadas, uma a uma, ao longo de quase setenta anos:

1950	Turing propõe o teste de conversação
1954	Georgetown demonstra tradução automática (regras)
1986	Redes neurais e backpropagation
1997	LSTM
2003	Primeiro modelo neural de linguagem (Bengio)
2013	word2vec (embeddings)
2014	seq2seq (encode-decode)
2015	Atenção (Bahdanau)
2016	Google NMT (produto: seq2seq + atenção em produção)
2017	Transformer ← ponto de partida do resto deste livro

Cada linha dessa tabela não é apenas um marco cronológico: é uma resposta parcial a uma pergunta que se recusava a ser respondida por inteiro. Linguagem é sequencial, estruturada e ambígua — e cada geração de pesquisadores resolveu um desses três aspectos enquanto deixava os outros dois em aberto, até que o Transformer apareceu com a resposta que combinava todas as peças: atenção total, em paralelo, sobre tudo o que veio antes.

O que significa, afinal, “modelar linguagem”?

Antes de mergulhar nas tentativas históricas, convém fixar o que está em jogo. Um modelo de linguagem atribui uma probabilidade a sequências de palavras. A forma mais comum é a modelagem autoregressiva: dado o que já foi escrito, prever a próxima palavra. Matematicamente, isso se escreve assim:

$$P(\text{"a casa é azul"}) = P(a) \cdot P(\text{casa} \mid a) \cdot P(\text{é} \mid a \text{ casa}) \cdot P(\text{azul} \mid a \text{ casa é})$$

Parece trivial, mas é poderoso. Um bom preditor de próxima palavra acaba aprendendo gramática, vocabulário, fatos sobre o mundo e até estilo — tudo como subproduto de tentar adivinhar o token seguinte. Quase toda inovação que este livro vai explorar, do GPT ao RAG aos agentes autônomos, preserva essa espinha dorsal: gerar a próxima palavra condicionada ao contexto. O que muda, de era em era, é *como* o contexto é representado e *quanto* dele a máquina consegue enxergar.

A era das regras: quando a linguagem era gramática

Entre 1950 e 1980, a abordagem dominante tratava a linguagem como um sistema de regras. Gramáticas formais, dicionários bilíngues, regras de tradução escritas à mão por linguistas pacientes. O experimento de Georgetown, em 1954, traduziu frases do russo para o inglês diante de jornalistas e prometeu que em cinco anos a tradução automática seria um problema resolvido. Não foi. O que ficou claro, ao longo das duas décadas seguintes, é que regras não escalam para a ambiguidade real da língua. “A casa é azul” exige saber o que é uma casa, o que é azul, e que o adjetivo concorda com o substantivo — e isso para uma frase de cinco palavras. Multiplique por um parágrafo, por um livro, por uma conversa inteira, e o castelo de regras desmorona sob o próprio peso.

A era das contagens: estatística contra a maldição

A partir dos anos 1980, a comunidade trocou regras por contagens. Em vez de dizer à máquina *como* a língua funciona, deixou-se que ela observasse *o que aparece*. Modelos de n-gramas estimam a probabilidade de uma palavra a partir das $n-1$ anteriores, simplesmente contando frequências num corpus:

```
P(azul | casa é) ≈ contagem("casa é azul") / contagem("casa é")
```

É uma ideia honesta e brutalmente simples. E funciona — até certo ponto. O problema tem nome: a maldição da dimensionalidade. A maioria das frases possíveis nunca apareceu no corpus de treino. Quando n cresce, as contagens ficam esparsas demais; truques de suavização remendam, mas não resolvem, porque o modelo não generaliza de verdade para combinações novas. Um bigrama em Python é literalmente um dicionário de contagens, e já revela a fratura:

```
from collections import defaultdict, Counter

corpus = ["a casa é azul", "o céu é azul", "a casa é grande"]
contagens = defaultdict(Counter)

for frase in corpus:
    tokens = ["<início>"] + frase.split()
    for w1, w2 in zip(tokens, tokens[1:]):
        contagens[w1][w2] += 1

def prob(w1, w2):
    total = sum(contagens[w1].values())
    return contagens[w1][w2] / total if total else 0

print(prob("é", "azul"))    # 2/3 → "azul" seguiu "é" em 2 das 3 frases
print(prob("casa", "roxa")) # 0/3 → nunca visto: probabilidade zero
```

A última linha é o veredito da era inteira: o modelo só “acerta” o que viu. Se a combinação não está no corpus, a probabilidade é zero e ponto final. O Brown Corpus, referência da época, tinha cerca de um milhão de palavras — um instantâneo de uma década do inglês. As contagens de um bigrama cabem na memória de qualquer máquina atual; a maldição não está no volume de dados, mas no crescimento combinatorial de n . A estatística resolveu a rigidez das regras, mas criou outra prisão: a dependência absoluta do que já foi observado.

A era neural: palavras viram vetores

A virada conceitual veio em 2003, quando Bengio e colegas propuseram algo radicalmente diferente: em vez de contar, *aprender uma função contínua* que estima a probabilidade condicional. A ideia-chave é distribuir o aprendizado por palavras similares — se o modelo nunca viu “casa é azul” mas já viu “céu é azul”, ele transfere conhecimento de uma situação para a outra. É aqui que nascem os embeddings: cada palavra vira um vetor numérico em um espaço de alta dimensão, e a proximidade geométrica entre vetores codifica proximidade de significado.

```
mapa de palavras (espaço de embeddings)
manhã ~ tarde ~ noite
gato ~ cachorro
rei - homem + mulher ≈ rainha
```

O word2vec, de Mikolov e colaboradores em 2013, transformou essa intuição em ferramenta prática. Treinado sobre o corpus de notícias do Google — cerca de 100 bilhões de palavras —, o modelo produziu vetores de 300 dimensões por palavra e mostrou que relações semânticas e sintáticas emergiam espontaneamente da geometria. De repente, “máquina que fala” parecia menos delírio e mais engenharia.

Mas embeddings sozinhos não bastam: eles capturam o significado de palavras isoladas, não a dinâmica de uma frase inteira se desdobrando no tempo.

Redes recorrentes e o drama da memória

A linguagem é sequencial. Para prever a próxima palavra, o contexto relevante pode estar longe — muito longe. As Redes Neurais Recorrentes (RNNs) enfrentam esse fato processando tokens um a um, mantendo um estado oculto que, em tese, “lembra” de tudo o que passou:

```
t0: estado h0 ← f(embedding("a"), h_ini)
t1: estado h1 ← f(embedding("casa"), h0)
t2: estado h2 ← f(embedding("é"), h1)
t3: previsão ← softmax(h2 · W) → próxima palavra
```

Na teoria, a memória é infinita. Na prática, durante o treinamento, o gradiente — a medida de “quanto devo ajustar cada peso” — precisa viajar para trás por muitos passos temporais, e a cada passo ele se multiplica por números menores que um. O resultado é o chamado *vanishing gradient*: o sinal de erro desaparece antes de alcançar os tokens distantes, e a rede simplesmente esquece o começo da frase. A LSTM, proposta por Hochreiter e Schmidhuber em 1997, introduziu “portões” que regulam o que lembrar e o que descartar — uma melhoria enorme, sem dúvida. Mas a LSTM ainda processa um token de cada vez, o que impede paralelização e mantém um limite prático de memória. Em textos longos, o começo da frase já é neblina quando o modelo chega ao final.

Seq2seq e a semente da atenção

O paradigma seq2seq, consolidado por Sutskever e por Cho em 2014, separou o problema em duas redes distintas:

```
entrada → ENCODER (lê tudo) → vetor de contexto → DECODER (escreve) → saída
```

A arquitetura é elegante, mas esconde um gargalo severo: todo o significado da entrada precisa caber em um único vetor no final do encoder. Uma frase de cinquenta palavras, com todas as suas nuances, reduzida a um ponto no espaço. A resposta veio em 2015, com Bahdanau e colaboradores: em vez de espremer tudo em um vetor, o decoder pode *olhar de volta* para partes relevantes da entrada, ponderando cada posição por relevância.

```
decoder ao escrever "azul" →
  consulta: "qual posição da entrada é mais relevante?"
  resposta: a palavra "céu" (alto peso), "a" (peso baixo)
```

Essa ideia — relevância como pesos sobre posições — é a semente de tudo o que vem a seguir. Em 2016, o Google NMT levou seq2seq com atenção para produção e derrotou a tradução estatística que dominava havia uma década. Parecia o auge. Mas não era.

O que ainda faltava em 2016

Apesar do entusiasmo, as melhores arquiteturas de 2016 continuavam sendo recorrentes no coração, e três limitações estruturais permaneciam abertas como feridas. Primeiro, a *paralelização*: RNNs processam token a token, sequencialmente, e não há como acelerar toda a sequência de uma vez — o que, na era dos GPUs e dos clusters massivos, era um desperdício doloroso. Segundo, a *memória de longo alcance*: mesmo a LSTM tem um limite prático em dependências muito distantes, e textos reais são cheios de referências cruzadas que atravessam parágrafos inteiros. Terceiro, a *unificação*: cada tarefa exigia arquitetura e treinamento próprios; não havia um caminho comum de “pré-treinar uma vez e reutilizar para tudo”.

O Transformer, que ocupará o capítulo seguinte, foi a resposta a essas três limitações de uma só vez. Mas vale notar o que ele *não* inventou: a ideia de embedding veio da era neural, a estrutura de atenção veio de Bahdanau, e o objetivo de prever a próxima palavra veio da era estatística. Revoluções raramente começam do zero — elas arrombam portas que outros já haviam destrancado.

A lição estrutural

A pré-história deixa a lição mais antiga deste livro, e ela se repetirá em cada era: **o que permanece não é a técnica, é a representação que generaliza**. As regras de Georgetown morreram por não escalar; as contagens dos n-gramas, por não generalizar para o que nunca foi visto; o word2vec, por fixar cada palavra a um único ponto; e as RNNs, por comprimirem o passado em um estado que a memória perdia. Cada resposta resolveu um pedaço do problema e entregou o resto para a próxima. O fio que liga 1950 a 2016 é o mesmo que atravessará o livro inteiro: **a capacidade não está na regra escrita, mas no processo que a aprende** — e o processo que aprende com uma situação transfere para as outras. Essa é a diferença entre memorizar e generalizar, e é ela que o capítulo seguinte transforma em máquina.

Para o engenheiro

Se você está do lado da engenharia, há três lições que essa pré-história deixa gravadas. Não tente varrer combinações de n-gramas: a maldição da dimensionalidade garante que você vai esbarrar em dados que nunca viu; prefira representações aprendidas, que generalizam por similaridade. Embeddings são o pré-requisito de todo sistema de busca semântica — é neles que “gato” e “felino” passam a ser vizinhos — e testar a qualidade do embedding antes de otimizar o índice economiza semanas de dor. E, para sequências longas, fique atento ao caminho entre tokens distantes: arquiteturas com passo curto, como atenção ou convolução, capturam dependências de longo alcance muito melhor do que recorrência. A atenção de Bahdanau, aliás, é o molde mental de retrieval: pesos por posição dizem onde olhar. A mesma ideia reaparece em cross-attention, em RAG, em praticamente tudo que este livro vai construir daqui para frente.

Fontes: [Turing, 1950] — teste de conversação; [Bengio et al., 2003] — primeiro modelo neural de linguagem; [Mikolov et al., 2013] — word2vec/embeddings; [Hochreiter & Schmidhuber, 1997] — LSTM; [Sutskever et al., 2014] — seq2seq; [Cho et al., 2014] — seq2seq/GRU; [Bahdanau et al., 2015] — atenção; [Wu et al., 2016] — Google NMT. Visão geral histórica: [Wikipedia, Transformer]; [Springer, 2025]; [MDPI, 2023].

Capítulo 2 — Attention Is All You Need: o nascimento do Transformer (2017)

Em 2017, havia uma convicção quase silenciosa na comunidade de processamento de linguagem natural: se a linguagem era uma sequência, então qualquer modelo sério precisaria de algum mecanismo de memória sequencial. As RNNs e LSTMs pareciam inevitáveis, e a atenção era vista como um acessório útil — algo que ajudava o decoder a olhar para trás, mas não como o coração da arquitetura. Foi nesse cenário que um grupo pequeno fez uma aposta radical: e se a recorrência não fosse necessária? E se cada palavra pudesse conversar diretamente com todas as outras, ao mesmo tempo, sem passar por nenhum intermediário? Essa pergunta, que à primeira vista parecia apenas uma provocação técnica, acabou se tornando uma das mudanças mais profundas da história recente da inteligência artificial.

O artigo *Attention Is All You Need*, publicado por Vaswani e colaboradores em 12 de junho de 2017, não propôs apenas mais uma melhoria incremental. Ele sugeriu descartar por completo as redes recorrentes e construir tudo — leitura, relacionamento entre palavras, geração — a partir de mecanismos de atenção. A hipótese era ousada demais para passar despercebida. Um dos autores, Jakob Uszkoreit, suspeitava que atenção sozinha poderia bastar para tradução automática, contrariando a sabedoria dominante da época, inclusive a de seu pai, o linguista computacional Hans Uszkoreit. O resultado, no entanto, foi mais forte do que a aposta inicial: um modelo mais rápido de treinar, mais paralelizável e com desempenho superior em tradução. O Transformer original tinha dimensões modestas para os padrões atuais, mas sua estrutura provou-se extraordinariamente escalável. Quase tudo o que viria depois — GPT, BERT, T5, agentes, RAG, tool use — descende direta ou indiretamente dessa arquitetura.

A ideia central pode ser resumida em uma frase: a relevância entre duas palavras pode ser calculada diretamente, sem estados ocultos recorrentes, e esse cálculo pode ser feito em paralelo para todas as posições da sequência. Parece simples, mas essa mudança altera tudo. Em vez de uma memória comprimida que precisa sobreviver a muitos passos intermediários, o Transformer oferece acesso direto: qualquer token pode olhar para qualquer outro token imediatamente, com um peso de relevância calculado na hora.

Por que descartar a recorrência?

Para entender a força da aposta, é preciso lembrar o que significava treinar uma arquitetura recorrente. Uma RNN processa tokens um após o outro. Isso quer dizer que a palavra na posição mil só é processada depois das novecentas e noventa e nove anteriores. No treinamento, essa dependência sequencial cobra um preço alto: GPUs são excelentes em fazer muitas contas ao mesmo tempo, mas não conseguem acelerar bem uma cadeia longa de passos estritamente dependentes. Além disso, a informação de posições distantes precisa atravessar muitas transformações de estado antes de chegar ao ponto atual. A cada transformação, algo se perde. O sinal enfraquece, a memória degrada, e o modelo começa a tratar o início do texto como uma lembrança vaga.

O Transformer rompe com essa lógica. Em vez de comprimir o passado em um estado que se transforma passo a passo, ele permite que cada posição consulte diretamente todas as outras. A distância deixa de ser um caminho longo e cheio de intermediários; torna-se apenas mais um par de posições cuja relevância pode ser calculada. Com isso, duas limitações históricas começam a cair ao mesmo tempo: o paralelismo deixa de ser bloqueado pela ordem de processamento, e dependências distantes passam a ser alcançáveis sem que a informação precise sobreviver a uma longa cadeia de estados.

A atenção como operação central

A operação que torna isso possível é a chamada *scaled dot-product attention*. Nela, cada posição da sequência gera três vetores a partir de sua representação:

```

Query (Q) → "o que eu estou procurando?"
Key (K) → "o que eu ofereço?"
Value (V) → "o conteúdo que entrego se for escolhido"

```

A intuição é elegante. O Query representa a pergunta que uma posição faz ao restante da sequência. As Keys representam aquilo que cada outra posição oferece como resposta possível. Quando Query e Key são compatíveis, a posição correspondente ganha peso. O Value, por sua vez, é o conteúdo efetivamente transmitido quando aquela posição é escolhida. Em outras palavras, Query e Key decidem a relevância; Value carrega a informação.

Para cada posição, calcula-se a compatibilidade entre seu Query e os Keys de todas as outras posições:

```

pesos = softmax( Q · KT / √dk )
saída = pesos · V

```

A divisão por $\sqrt{d_k}$ existe para estabilizar os valores durante o cálculo — e é daí que vem o termo *scaled*. Depois do softmax, os pesos se tornam uma distribuição de relevância. A saída de cada posição é, então, uma soma ponderada dos Values, em que cada Value contribui de acordo com a importância que recebeu.

Para tornar isso concreto, imagine um exemplo pequeno, com dimensão 2 apenas para caber no papel. Suponha os tokens “a”, “casa” e “é”. A posição “a” possui query $Q = [1, 0]$, e as chaves são $K_a = [1, 0]$, $K_{casa} = [0, 1]$ e $K_é = [1, 1]$. Os produtos escalares, que medem a compatibilidade inicial, são $[1, 0, 1]$. Dividindo por $\sqrt{2} \approx 1,41$ e aplicando softmax, temos algo como:

```

escores escalados ≈ [0,71, 0, 0,71]
softmax([0,71, 0, 0,71]) ≈ [0,40, 0,20, 0,40]

```

Nesse exemplo, “a” presta atenção igualmente a “a” e a “é”, e pouca atenção a “casa”. Os pesos sobre as posições são exatamente essa distribuição. Em implementações reais, as dimensões não são 2, mas centenas — no artigo original, $d_{model} = 512$ — e os vetores são aprendidos durante o treinamento. Ainda assim, a lógica essencial permanece a mesma.

Em código, a atenção é surpreendentemente compacta:

```

import torch
import torch.nn.functional as F

def atencao(Q, K, V):
    d_k = Q.shape[-1]
    escores = Q @ K.transpose(-2, -1) / d_k ** 0.5
    pesos = F.softmax(escores, dim=-1)
    return pesos @ V

```

Há também um exemplo intuitivo que ajuda a perceber por que essa operação é tão poderosa. Considere a frase:

“O animal não atravessou a rua porque estava cansado.”

Para interpretar corretamente a frase, o modelo precisa relacionar o estado de cansaço ao animal, não à rua. Em uma arquitetura recorrente, essa relação precisa sobreviver a vários passos intermediários. Na atenção, cada posição pode perguntar diretamente às demais: “quem é relevante para mim agora?” E a resposta pode vir imediatamente, sob a forma de pesos altos sobre a posição correta.

Muitas cabeças, muitas perguntas

Uma única atenção já é capaz de capturar relações relevantes, mas a linguagem não é feita de um único tipo de relação. Há dependências sintáticas, semânticas, referenciais, discursivas. Às vezes, a palavra importante é o sujeito; às vezes, é o objeto; às vezes, é um pronome lá atrás; às vezes, é um modificador próximo. Se houvesse apenas um mecanismo de atenção, ele precisaria dar conta de todos esses padrões ao mesmo tempo.

O Transformer resolve isso executando várias atenções em paralelo, cada uma com projeções diferentes de Query, Key e Value. Esse mecanismo é chamado de *multi-head attention*:

```

entrada —┐ cabeça 1 → atenção ┐
           └─┬─ cabeça 2 → atenção ┤ concatena → projeção → saída
              └─┬─ cabeça 3 → atenção ┤

```

A ideia é que diferentes cabeças possam se especializar em diferentes tipos de dependência. Uma cabeça pode aprender a ligar adjetivos a substantivos; outra pode rastrear coreferências; outra pode

capturar relações de longo alcance. Análises posteriores confirmaram empiricamente que cabeças distintas tendem a desenvolver comportamentos distintos. Assim, o modelo não apenas presta atenção — ele presta atenção de várias maneiras simultaneamente.

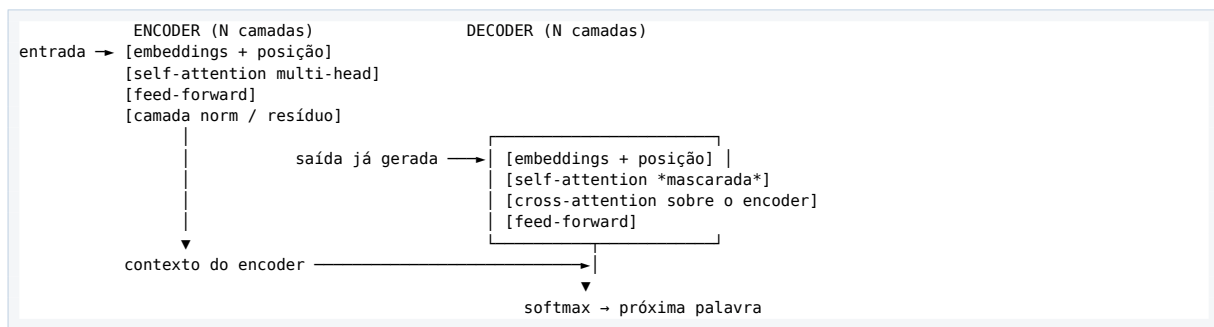
A ordem precisa voltar

Havia, porém, uma armadilha. A atenção, por si só, é insensível à ordem dos tokens. Ela opera como se estivesse lidando com um conjunto, não com uma sequência. Sem informação posicional, frases como “o gato mordeu o cão” e “o cão mordeu o gato” produziriam representações idênticas, porque contêm exatamente as mesmas palavras. Isso seria inaceitável para qualquer modelo de linguagem minimamente sério.

A solução foi injetar informação posicional diretamente nas representações. O artigo original fez isso por meio de embeddings posicionais senoidais, que adicionam a cada posição um sinal único e previsível. Modelos posteriores desenvolveram alternativas aprendidas e variantes mais sofisticadas, como RoPE e ALiBi. O ponto essencial é que a atenção fornece o acesso direto entre posições, mas a informação posicional devolve a ordem ao modelo. Sem ela, o Transformer enxergaria as palavras, mas não a frase.

Uma arquitetura com duas metades

O Transformer de 2017 não era apenas uma operação de atenção isolada. Ele foi apresentado como uma arquitetura completa de encoder-decoder, herdando do seq2seq a divisão entre leitura e geração, mas substituindo a recorrência por atenção em todos os lugares.



Três detalhes dessa arquitetura são especialmente importantes. O primeiro é a *self-attention*: cada posição atende a outras posições da mesma sequência. No encoder, isso significa que a entrada pode se relacionar internamente consigo mesma. No decoder, significa que a saída já gerada pode consultar o próprio passado.

O segundo detalhe é a *masked self-attention* no decoder. Como o modelo está prevendo a próxima palavra, ele não pode trapacear olhando para o futuro. A máscara impede que posições ainda não geradas influenciem a previsão atual. É esse cuidado que torna o Transformer usável para geração autoregressiva, token a token.

O terceiro detalhe é a *cross-attention*. Além de olhar para si mesmo, o decoder também atende à saída do encoder, conectando a entrada lida à saída que está sendo produzida. Trata-se da generalização direta da atenção de Bahdanau, agora integrada a uma arquitetura inteiramente baseada em atenção.

Para dimensionar mentalmente o tamanho do Transformer original, vale observar sua configuração:

Hiperparâmetro	Transformer base	Transformer big
d_model (dimensão das representações)	512	1024
Cabeças de atenção	8	16
Camadas (encoder + decoder)	6 + 6	6 + 6
Feed-forward (camada oculta)	2048	4096
Parâmetros	~65M	~213M

Comparados aos bilhões e trilhões de parâmetros dos modelos atuais, esses números parecem pequenos. Mas a estrutura essencial é a mesma. O que mudou depois não foi a invenção de uma arquitetura completamente nova, e sim a escala, os dados e o refinamento de treino.

Três linhagens para o futuro

Embora o Transformer original tenha nascido como um modelo encoder-decoder, o mais importante para o restante da história não foi exatamente o modelo de 2017, e sim as três linhagens que derivaram dele:

Encoder-only	BERT	→	entendimento (classificação, perguntas-respostas)
Decoder-only	GPT	→	geração autoregressiva ← base dos LLMs modernos
Encoder-decoder	T5	→	transformações texto-texto

A linhagem *encoder-only* usa apenas a metade encodadora do Transformer. Ela enxerga a sequência inteira de uma vez, de forma bidirecional, e por isso é especialmente adequada para tarefas de entendimento: classificar texto, extrair informação, responder perguntas a partir de um trecho fornecido. O BERT é o exemplo clássico dessa família.

A linhagem *decoder-only* usa apenas a metade decodadora. Ela gera token a token, sempre olhando para trás, respeitando a causalidade. É a família que domina os grandes modelos de linguagem modernos, porque se mostrou extremamente eficaz para geração, raciocínio em linguagem natural e uso interativo. O GPT é seu representante mais famoso.

A linhagem *encoder-decoder* preserva as duas metades e permanece útil para tarefas de transformação de texto em texto, como tradução, sumarização e reformulação. O T5 é um exemplo importante dessa abordagem.

O Transformer de 2017 era encoder-decoder. Nos anos seguintes, BERT e GPT exploraram caminhos diferentes: um apostou no entendimento bidirecional; outro, na geração causal. Esses dois ramos pareciam distintos, mas voltariam a se encontrar mais tarde, especialmente em sistemas de RAG, onde compreensão e geração precisam trabalhar juntas.

O que o Transformer mudou de fato

O impacto do Transformer pode ser entendido em três grandes frentes. A primeira é o paralelismo. Como a self-attention é computada sobre todas as posições de uma vez, o treinamento se tornou muito mais eficiente em GPUs. O modelo original treinou em cerca de três dias e meio usando oito GPUs e, mesmo assim, alcançou estado da arte em tradução automática. Para a época, isso era impressionante — não apenas pelo resultado, mas pelo fato de que a arquitetura permitia aproveitar muito melhor o hardware disponível.

A segunda frente é o acesso direto a dependências de longo alcance. Em uma RNN, a informação entre tokens distantes precisa atravessar muitos estados intermediários. No Transformer, qualquer posição pode atender diretamente a qualquer outra. A distância deixa de ser um funil de degradação e passa a ser apenas mais uma relação a ser ponderada.

A terceira frente é a escalabilidade. O Transformer se mostrou compatível com escalas de dados e parâmetros que as arquiteturas recorrentes não suportavam bem. Isso não era apenas uma vantagem técnica: era uma porta aberta para o futuro. Os capítulos seguintes explorarão como essa escalabilidade permitiu transformar um modelo de tradução em uma base para sistemas cada vez mais gerais.

O teste original foi feito no WMT 2014, um benchmark de tradução automática com aproximadamente 4,5 milhões de pares inglês-alemão e cerca de 36 milhões de pares inglês-francês. O Transformer big alcançou BLEU 28,4 em inglês para alemão e BLEU 41,8 em inglês para francês. BLEU é uma métrica que mede a sobreposição de n-gramas entre a tradução gerada e referências humanas, variando de 0 a 100. Na prática, o resultado mostrava que, com uma fração do custo de treinamento em comparação com modelos recorrentes de ponta, o Transformer conseguia empatar ou superar o estado da arte de então.

A lição estrutural

O Transformer não venceu por ser mais complexo — venceu por **remover a estrutura que limitava**: o estado sequencial comprimido. Toda vez que um sistema obriga a informação a atravessar um funil de transformações intermediárias, algo se perde no caminho; o ganho do Transformer veio de substituir o funil por acesso direto — qualquer posição pode olhar qualquer posição, na hora. Essa lição tem duas faces que o livro inteiro vai reutilizar. A primeira: **arquitetura é a escolha de quais distâncias importam** — quando a distância deixa de ser um caminho e vira uma relação ponderada, o problema que parecia estrutural vira questão de escala. A segunda: **Query, Key e Value são o molde de todo retrieval** — relevância decidida por compatibilidade, conteúdo entregue pelo Value —, e esse é, palavra por palavra, o desenho da busca semântica e do RAG. O Transformer de 2017 não previu o RAG; previu

a forma que o RAG usa.

Para o engenheiro

Para quem projeta sistemas, o Transformer deixa algumas lições duradouras. A primeira é que Query e Key definem relevância, enquanto Value carrega o conteúdo. Sempre que você pensar em um sistema de recuperação, vale perguntar: qual é a chave que determina se algo é relevante? E qual é o valor que deve ser retornado quando essa relevância é detectada? Essa separação mental é mais útil do que parece, porque ela aparece não apenas na atenção, mas também em busca semântica, cross-attention e RAG.

A segunda lição é que geradores são causais. Um modelo decoder-only só pode olhar para trás; ele não pode ver o futuro do texto que ainda vai produzir. Isso tem implicações práticas diretas na construção de prompts, agentes e tool use. O contexto fornecido ao modelo precisa conter tudo o que ele precisa saber antes da geração. Não se pode contar com uma suposta visão futura do processo.

A terceira lição é que ordem importa. Se o seu pipeline reordena dados, serializa documentos de forma inconsistente ou altera a posição de trechos relevantes, os embeddings posicionais mudarão a interpretação do modelo. Preservar a ordem ao preparar documentos para o contexto não é um detalhe cosmético; é parte da semântica que o Transformer consome.

Por fim, é importante lembrar que todo modelo moderno que você usa é, em algum nível, um Transformer. A chamada janela de contexto é, na prática, a quantidade de informação sobre a qual o modelo pode aplicar atenção em um determinado momento. É ela que limita quantos documentos, instruções, exemplos ou passos de raciocínio cabem em um prompt. Compreender isso não é apenas compreender uma arquitetura: é compreender a base material sobre a qual praticamente todo o ecossistema atual de LLMs foi construído.

Fontes: [Vaswani et al., 2017] — Attention Is All You Need (Transformer); [Bahdanau et al., 2015] — atenção em seq2seq; [Wikipedia, Transformer] — visão geral da arquitetura; [Zhao et al., 2023] — survey de LLMs.

Capítulo 3 — Pré-treinamento: ELMo, BERT e GPT (2018-2019)

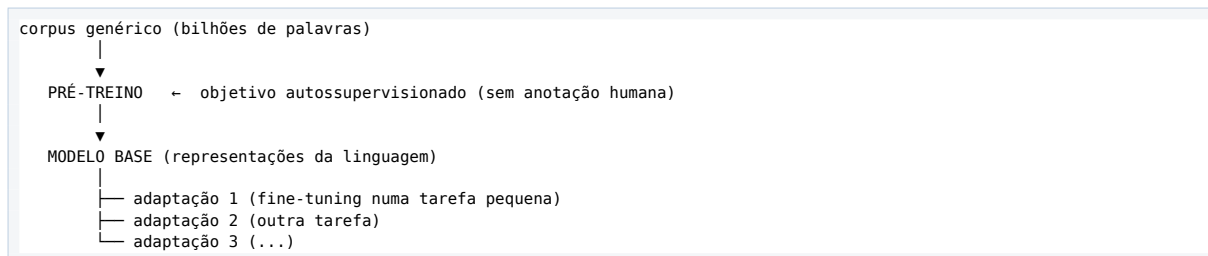
Resolvido o problema da arquitetura com o Transformer, restava outro obstáculo, menos vistoso porém mais caro: para cada tarefa — classificar, responder, traduzir, resumir — ainda era preciso construir um modelo e um conjunto de dados praticamente do zero. A ideia que emergiu entre 2018 e 2019 foi, ao mesmo tempo, preguiçosa e genial: treinar uma única vez sobre um corpus gigante e genérico, depois adaptar esse modelo a tarefas específicas. A pergunta era quase filosófica: uma rede treinada apenas para prever a próxima palavra seria capaz, de brinde, de responder perguntas, traduzir e resumir? ELMo, BERT e GPT responderam em sequência, cada um à sua maneira. O mais insolente deles, o GPT-2, revelou que prever texto pode ensinar multitarefa sem nenhuma supervisão explícita. A ambição mudou de escala: não se tratava mais de construir máquinas para tarefas isoladas, mas de criar uma base comum que contivesse todas elas. É aqui que o contexto começa a virar código — a semente do que, mais tarde, este livro chamará de engenharia de contexto.

O Transformer já havia provado que atenção pura funciona. A próxima pergunta era: como aproveitar esse poder para além da tradução? A resposta veio da combinação de duas ideias antigas com a nova arquitetura. A primeira era a noção de embeddings contextuais: uma palavra não tem um significado fixo; “a margem do rio” e “a margem de lucro” usam a mesma palavra com sentidos diferentes, e qualquer representação séria precisa capturar essa variação. A segunda era o pré-treinamento: em vez de aprender cada tarefa isoladamente, aprender primeiro com uma quantidade massiva de texto genérico, para só depois especializar-se. Entre 2018 e 2019, três modelos definiram o mapa dessa nova era: ELMo, que trouxe contextualização ainda com LSTM; BERT, que apostou no entendimento bidirecional; e GPT, que apostou na geração causal. Este capítulo segue principalmente o ramo GPT, porque é dele que descendem os LLMs modernos — mas BERT é indispensável para entender o RAG, que aparecerá mais adiante.

A ideia central pode ser enunciada de forma simples: treine primeiro em texto genérico em escala, depois adapte. E quanto mais genérica e grande for a base, mais tarefas ela desbloqueia sem treino específico. Essa mudança de perspectiva transformou o campo. O modelo deixou de ser um especialista estreito, moldado para uma única função, e passou a ser um generalista treinado na linguagem do mundo, capaz de receber instruções e se ajustar a novas demandas com muito pouco esforço adicional.

Pré-treinar para depois adaptar

Antes dessa virada, cada tarefa exigia modelo e dados próprios. Classificação tinha seu pipeline; perguntas-respostas tinha outro; tradução, outro ainda. O novo paradigma quebrou essa lógica ao propor um fluxo único:



O ponto crucial é que o pré-treino é autossupervisionado: o “rótulo” é derivado do próprio texto. Isso significa que não é preciso pagar o custo humano de anotar milhões de exemplos. O texto já contém, em si mesmo, sinais suficientes para ensinar algo sobre linguagem. Essa mudança permitiu usar quantidades massivas de dados sem depender de curadoria manual, e foi isso que tornou economicamente viável treinar modelos cada vez maiores.

ELMo: a palavra ganha dependência do contexto

ELMo, publicado por Peters e colaboradores em 2018, não era um Transformer; era uma bi-LSTM pré-treinada. Ainda assim, introduziu a ideia-chave que mudaria tudo: representações contextuais. Diferentemente do word2vec, que atribuía um único vetor fixo a cada palavra, o ELMo produzia um vetor que mudava conforme a frase. A palavra “banco” não era representada sempre da mesma forma; sua representação dependia do contexto em que aparecia. O modelo alimentava tarefas específicas com esses vetores contextualizados. Arquiteturalmente, ficou datado. Conceitualmente, marcou a transição decisiva: a palavra deixou de ser um ponto fixo no espaço e passou a ser uma função do contexto.

BERT: entender olhando para os dois lados

BERT, publicado por Devlin e colaboradores em 2018, é encoder-only: ele vê a sequência inteira de uma vez. Essa escolha arquitetural tem consequências profundas. Como o modelo pode olhar simultaneamente para o que vem antes e para o que vem depois de uma palavra, ele constrói representações especialmente ricas para tarefas de entendimento. Seus dois objetivos de pré-treino eram o Masked Language Modeling e o Next Sentence Prediction. O primeiro escondia palavras e pedia ao modelo para preenchê-las usando o contexto dos dois lados. O segundo tentava ensinar se duas frases eram consecutivas.

Um exemplo clássico de MLM ajuda a perceber a força do mecanismo:

```
Entrada: 0 elefante não atravessou a [MASK] porque estava cansado.
Saída:   rua (candidatos com score: rua 0,89 · estrada 0,78 · linha 0,31)
```

O detalhe importante é o que o modelo usa para decidir: o lado direito da frase — “porque estava cansado” — ajuda a escolher “rua”. É isso que torna o BERT bidirecional. Ele não está apenas prevendo o próximo token olhando para trás, como os geradores causais do ramo GPT; ele está interpretando a frase como um todo, com acesso ao contexto completo. Na época, o BERT superou o estado da arte em praticamente todas as tarefas de entendimento de linguagem natural. Seu impacto duradouro apareceu também fora da pesquisa: em outubro de 2019, o Google passou a usar BERT no processamento de consultas de busca, um sinal claro de que a técnica havia saído dos artigos e entrado em produção massiva.

GPT: gerar como objetivo universal

GPT, publicado por Radford e colaboradores em 2018, seguiu o caminho oposto. Ele é decoder-only: gera token a token, de forma causal, olhando apenas para trás. Seu pré-treino usa a modelagem autoregressiva clássica: prever a próxima palavra. Para adaptar o modelo a uma tarefa, faz-se fine-tuning supervisionado sobre a base pré-treinada. A aposta do GPT era a simplicidade. Em vez de objetivos e arquiteturas customizados para cada tarefa, um único objetivo — prever texto — e uma arquitetura uniforme. Essa escolha parecia mais modesta do que a do BERT, mas acabaria se revelando extraordinariamente poderosa.

GPT-2: quando prever texto virou multitarefa

GPT-2, publicado por Radford e colaboradores em 2019, escalou o decoder-only para 1,5 bilhão de parâmetros e fez uma descoberta que mudou a direção da pesquisa: sem nenhum treino específico para a tarefa, o modelo conseguia responder perguntas, traduzir, resumir e gerar texto coerente — bastava continuar o prompt. Os autores chamaram isso de “modelos de linguagem são aprendizes multitarefa não supervisionados”. A implicação era profunda: talvez não fosse necessário ensinar cada tarefa separadamente; talvez bastasse apresentar a tarefa no contexto certo.

O corpus usado foi o WebText: cerca de 40 GB de texto, vindos de 8 milhões de documentos coletados a partir de links do Reddit com mais de 3 upvotes. Esse filtro simples servia como um sinal de qualidade. O WebText virou o protótipo da receita que o GPT-3 expandiria depois: texto da internet filtrado por algum sinal humano indireto. Para dimensionar a escala da época, vale observar os quatro modelos centrais deste capítulo:

Modelo	Arquitetura	Camadas	d_model	Params	Corpus de pré-treino
BERT-base (2018)	encoder-only	12	768	110M	BooksCorpus + Wikipedia (~3,3B palavras)

Modelo	Arquitetura	Camadas	d_model	Params	Corpus de pré-treino
BERT-large (2018)	encoder-only	24	1024	340M	idem
GPT-1 (2018)	decoder-only	12	768	117M	BooksCorpus (~800M palavras)
GPT-2 (2019)	decoder-only	48	1600	1,5B	WebText (~40 GB / 8M docs)

O BooksCorpus é um dataset com cerca de 11.000 livros não publicados; a Wikipedia inglesa completa de 2018 entra no pré-treino do BERT. Esses dois pilares foram a base textual dessa era. Vistos de hoje, os números parecem pequenos, mas foi ali que se formou o germe do zero-shot: executar uma tarefa apenas descrevendo-a no contexto de entrada, sem exemplos e sem ajuste. O próprio GPT-2 era grande demais para a época. A primeira versão nem foi publicada integralmente, por receio de uso indevido — um dos momentos mais marcantes da história pública da OpenAI.

A lição estrutural

Parte do legado dessa fase está no vocabulário que ainda hoje organiza o campo. Alguns termos nasceram ali ou ganharam ali sua forma moderna:

Termo	Significado
zero-shot	resolver tarefa sem exemplos, só com instrução/descrição no prompt
few-shot	resolver tarefa com alguns exemplos dados no prompt (amadurecido no GPT-3, cap. 4)
fine-tuning	continuar o treinamento em dados específicos da tarefa
pré-treino	treinamento genérico autossupervisionado em grande corpus

Note o vínculo com a tese deste livro: o contexto passa a ser um canal de programação. A tarefa é “descrita”, não “treinada”. Isso parece uma mudança de vocabulário, mas é uma mudança de paradigma. Em vez de modificar os pesos do modelo para cada nova necessidade, começa-se a modificar aquilo que se coloca diante dele. É o início da engenharia de contexto que os capítulos seguintes explorarão com RAG, ferramentas e agentes.

Por que isso importa para RAG e agentes

Este capítulo não é apenas uma etapa histórica; é uma divisão de trabalho que permanece atual. O BERT fornece o retriever de muitos sistemas RAG. O DPR, por exemplo, usa uma linhagem encoder para representar consultas e documentos. Entender o que é uma representação contextual ajuda a entender por que busca semântica funciona — e por que às vezes falha. O GPT, por sua vez, fornece o generator: o cerne de todos os LLMs e agentes que aparecem neste livro.

A lição mais profunda, contudo, vem do GPT-2: prever texto ensina muito. É isso que torna RAG e agentes viáveis. Um bom modelo base já “sabe” usar informação quando ela lhe é apresentada no contexto. Ele não precisa ser treinado do zero para cada documento novo; ele precisa receber o documento certo, no lugar certo, no formato certo. Essa percepção é uma das chaves que conectam pré-treinamento, recuperação e agentes.

Para o engenheiro

Para quem projeta sistemas, a primeira lição é escolher a família pela tarefa. Encoders são adequados para classificação e entendimento; decoders, para chat e geração; encoder-decoders, para tradução e transformações texto-para-texto. Na prática atual, você raramente treina esses modelos do zero; você os escolhe. Essa decisão arquitetural inicial ainda importa, mesmo quando o modelo final já vem pronto como serviço.

A segunda lição é que pré-treino autossupervisionado significa dados sem anotação. Se falta dado rotulado, o caminho natural é partir de um modelo pré-treinado e fazer fine-tuning. Treinar do zero sem um motivo muito forte costuma ser caro, lento e desnecessário. A base já carrega linguagem, mundo e estilo; o fine-tuning apenas ajusta a direção.

A terceira lição é que, antes de fine-tunar, vale testar zero-shot ou few-shot no prompt. Essa abordagem é barata, reversível e hoje cobre muito do que, em 2018, exigia treinamento. Em produção, o zero-shot sobrevive até como fallback: antes de orquestrar retrieval ou ferramentas, muitas vezes vale tentar resolver o problema apenas com uma boa instrução.

Por fim, lembre que zero-shot foi onde tudo começou. Ele não é apenas um truque de prompt; é a primeira evidência empírica de que o contexto pode programar o modelo. Quando um sistema de RAG injeta documentos no prompt, ou quando um agente injeta resultados de ferramentas, está estendendo exatamente essa intuição: se o modelo aprende com o texto à sua frente, então controlar esse texto é controlar parte do comportamento do modelo.

Fontes: [Peters et al., 2018] — ELMo; [Devlin et al., 2018] — BERT; [Radford et al., 2018] — GPT-1; [Radford et al., 2019] — GPT-2; [Wikipedia, Transformer] — visão geral da arquitetura; [Zhao et al., 2023] — survey de LLMs.

Capítulo 4 — Escala e emergência: GPT-3 e as leis de escala (2020-2022)

Se a previsão da próxima palavra ensina a máquina a estruturar a linguagem, o que ocorre quando expandimos massivamente essa capacidade? Em 2020, a comunidade de inteligência artificial decidiu testar essa hipótese em larga escala. O GPT-3 não introduziu uma nova arquitetura; ele aplicou a estrutura do Transformer a um volume de dados e parâmetros em uma escala até então inexplorada. O resultado confirmou uma suspeita crescente: a tarefa podia ser comunicada ao modelo por meio do próprio contexto, dispensando novas fases de treinamento. Paralelamente, a formulação das leis de escala transformou o aumento de modelos de uma aposta empírica em uma ciência previsível. Contudo, o fenômeno mais notável desse período foi o aparecimento de capacidades que não haviam sido explicitamente programadas — como aritmética e raciocínio lógico —, um processo que a literatura passou a chamar de emergência. Este capítulo examina como a escala redefiniu o treinamento de modelos e quais limitações estruturais permaneceram, criando a necessidade das técnicas que dominariam os anos seguintes.

A matemática da escala

Até 2020, aumentar o tamanho de um modelo era uma decisão baseada em intuição e disponibilidade de hardware. O trabalho de Kaplan e colaboradores mudou esse panorama ao demonstrar que o desempenho de modelos de linguagem segue leis de potência. O erro de previsão diminui de forma previsível à medida que se aumentam três variáveis: o número de parâmetros, o volume de dados e a capacidade computacional.

Matematicamente, a perda (erro) em relação ao número de parâmetros N pode ser aproximada por:

$$L(N) \approx (N_c / N)^{\alpha_N} \quad \text{com } \alpha_N \approx 0,076$$

Em um gráfico em escala logarítmica, essa relação se manifesta como uma linha reta. Cada multiplicação por dez no tamanho do modelo reduz o erro por um fator constante. A mesma regularidade se aplica ao tamanho do conjunto de dados (D) e à computação total (C). Essa descoberta teve um impacto imediato na indústria: se o retorno do investimento em computação é previsível, o treinamento de modelos gigantes deixa de ser um experimento e passa a ser uma estratégia de engenharia.

Dois anos depois, o projeto Chinchilla (Hoffmann et al., 2022) refinou essa compreensão. A pesquisa mostrou que os modelos anteriores, incluindo o GPT-3, eram grandes demais para a quantidade de dados em que haviam sido treinados. Para um orçamento computacional fixo, existe uma proporção ótima entre parâmetros e tokens. A regra prática que emergiu desse estudo — escalar modelo e dados na mesma proporção — ditou o desenho das gerações seguintes, priorizando a curadoria de dados de alta qualidade em vez de apenas empilhar mais parâmetros.

GPT-3 e a tarefa como contexto

O GPT-3 (Brown et al., 2020) manteve a arquitetura *decoder-only* do GPT-2, mas expandiu suas dimensões para 175 bilhões de parâmetros, distribuídos em 96 camadas, com uma janela de contexto de 2048 tokens. A mudança fundamental, no entanto, não foi estrutural, mas operacional. O modelo demonstrou que a adaptação a novas tarefas não exigia mais o ajuste fino (*fine-tuning*) dos pesos da rede.

Antes (fine-tuning):	dados da tarefa → treinar mais → modelo especializado
GPT-3 (in-context):	prompt com instrução + exemplos → resposta

Esse paradigma, chamado de aprendizado em contexto (*in-context learning*), significa que o modelo congela seus pesos após o pré-treino. A “programação” da tarefa ocorre inteiramente na janela de contexto. Um *prompt* com alguns exemplos (*few-shot*) é suficiente para alinhar o comportamento do modelo:

```
"Traduza do inglês para o português.
dog → cachorro
cat → gato
house → ..." → o modelo responde "casa"
```

Para sustentar um modelo dessa magnitude, foi necessário compor o maior *corpus* textual da época, totalizando cerca de 499 bilhões de tokens. A composição dessa base de dados revela uma decisão de engenharia crucial: a filtragem.

Fonte	Participação	Tokens (~)
CommonCrawl (filtrado)	60%	410B
WebText2	22%	19B
Books1	8%	12B
Books2	8%	55B
Wikipedia	3%	3B

A tabela ilustra que o conhecimento do modelo é um reflexo direto de sua dieta textual. O CommonCrawl bruto contém dezenas de terabytes de ruído; o investimento em filtros de qualidade foi o que permitiu extrair sinal desse volume. O conhecimento, uma vez consolidado nos pesos, torna-se estático — uma limitação que exigiria soluções externas, como a busca semântica (RAG), nos capítulos seguintes.

Capacidades emergentes

À medida que os modelos cruzavam a casa das centenas de bilhões de parâmetros, pesquisadores começaram a documentar um padrão recorrente: certas tarefas apresentavam desempenho medíocre em modelos pequenos, mas sofriam saltos abruptos de precisão a partir de um limiar crítico de escala. Wei e colaboradores (2022) catalogaram esse fenômeno sob o termo “capacidades emergentes”.

Exemplos notáveis incluíam operações aritméticas de múltiplos dígitos, raciocínio lógico encadeado e tradução entre idiomas com poucos recursos. Uma leitura técnica desse fenômeno revela que a emergência é, em grande parte, uma propriedade das métricas de avaliação. Quando uma métrica exige uma sequência exata de passos (como a resposta final de uma equação), o modelo parece “aprender de repente” a resolver o problema, embora a probabilidade interna de acerto venha crescendo de forma contínua com a escala.

Independentemente da interpretação, a implicação prática era clara: a escala por si só não garantia confiabilidade. Mesmo com 175 bilhões de parâmetros, o GPT-3 continuava a fabricar fatos (alucinações), falhar em cálculos básicos e operar com conhecimento congelado na data de seu treinamento. A escala resolvia a fluência e a capacidade de seguir instruções complexas, mas deixava abertas as lacunas de precisão factual e execução lógica.

A infraestrutura que sustentou a escala

Treinar e operar modelos dessa magnitude exigiu avanços paralelos em sistemas distribuídos. A memória de uma única GPU não era suficiente para armazenar os pesos de um modelo de 175 bilhões de parâmetros, muito menos processar suas matrizes de atenção.

A pesquisa em infraestrutura respondeu com técnicas de paralelismo de modelo (como o Megatron-LM, que divide as camadas e a atenção entre diferentes dispositivos), otimizadores de memória e, posteriormente, algoritmos de atenção eficiente (como o FlashAttention), que reduziram drasticamente o custo de leitura e escrita na memória da GPU. Modelos esparsos (*Mixture of Experts* - MoE) e conjuntos de dados abertos e curados (como o *The Pile*) também se tornaram padrões da indústria. A história dos modelos de linguagem é indissociável da história do hardware e dos sistemas que os tornam viáveis.

O que a escala deixou de herança

O período entre 2020 e 2022 consolidou três pilares que sustentam a engenharia de LLMs atual. Primeiro, a noção de que o contexto funciona como uma linguagem de programação: a tarefa é descrita, não treinada. Segundo, a escala como um vetor de melhoria contínua e previsível, desde que acompanhada de dados de qualidade. Terceiro, o mapeamento claro das limitações estruturais dos modelos puramente generativos.

A lição estrutural

A era da escala deixou duas heranças que o livro inteiro carrega. A primeira: **escala é previsível quando se conhece a curva** — Kaplan e Chinchilla transformaram “aumentar o modelo” de aposta em engenharia, com custo estimável e retorno calculável. A segunda é a contraparte: **capacidade não é confiabilidade** — o GPT-3 falava com fluência e errava com confiança, e nenhum ponto da curva corrigia isso. As lacunas que a escala deixou em aberto — conhecimento desatualizado, alucinação, cálculo inexato — não foram resolvidas por mais parâmetros; foram resolvidas por *camadas*: recuperação, ferramentas, verificação. O resto do livro é a história dessas camadas. E a pergunta que a escala ensinou a fazer — “qual é o próximo eixo de melhoria além dos parâmetros?” — é a mesma que RAG e tool use responderam: o eixo deixou de ser o modelo e passou a ser o sistema.

Para o engenheiro

Se você está projetando sistemas sobre essas fundações, algumas diretrizes práticas se destacam. As leis de escala devem orientar o orçamento: os dados de Kaplan e Chinchilla permitem estimar o custo computacional necessário para atingir um determinado nível de erro, substituindo o palpite por planejamento. O *prompt* deve ser tratado como código de produção — versionado, testado e documentado, pois é ele que define o comportamento do modelo em tempo de execução.

Por fim, é preciso manter o ceticismo em relação a *benchmarks*. Nem toda capacidade “emergente” se traduz em utilidade prática, e métricas saturadas costumam esconder falhas sistêmicas. As lacunas identificadas no GPT-3 — conhecimento desatualizado, alucinação e incapacidade de cálculo exato — não foram resolvidas por mais escala. Elas exigiram a criação de novas camadas arquitetais. O modelo base aprendeu a falar e a raciocinar; caberia às técnicas de recuperação (RAG) e uso de ferramentas (*tool use*) ensiná-lo a consultar o mundo real e a executar ações.

Fontes: [Kaplan et al., 2020] — leis de escala; [Brown et al., 2020] — GPT-3; [Hoffmann et al., 2022] — Chinchilla (escala ótima); [Wei et al., 2022] — capacidades emergentes; [Zhao et al., 2023] — survey de LLMs.

Parte II — Era do RAG clássico (2020-2021)

Capítulo 5 — As origens do Retrieval-Augmented (2020): a memória fora dos pesos

A escala dos modelos de linguagem em 2020 trouxe capacidades inéditas, mas também deixou claras certas barreiras estruturais. O modelo sabia apenas o que havia visto durante o treinamento e, quando a informação lhe faltava, tendia a fabricar respostas com alta confiança. Além disso, seu conhecimento era estático: atualizar um fato ou corrigir um erro exigia retrainar a rede inteira, um processo inviável para bases que mudam diariamente. A questão que se impôs naquele ano foi direta: e se parássemos de tentar comprimir todo o conhecimento do mundo nos parâmetros da rede e, em vez disso, buscássemos a informação no momento exato em que ela fosse necessária?

A recuperação de informação (*Information Retrieval*) era uma disciplina consolidada desde a década de 1970. A novidade estava em integrá-la a modelos generativos modernos por meio de representações neurais. Esse foi o ponto de inflexão que deu origem a sistemas como DPR, REALM, kNN-LM e, por fim, ao próprio RAG. A premissa central era simples, porém transformadora: nem todo conhecimento precisa caber nos parâmetros da rede. Recuperar a evidência certa, na hora certa, funciona como uma forma de memória externa — atualizável, verificável e computacionalmente barata.

Memória paramétrica e não-paramétrica

Para entender essa mudança de arquitetura, é preciso separar dois tipos de memória que frequentemente são confundidos. A memória paramétrica é o conhecimento consolidado nos pesos do modelo após o treinamento. A memória não-paramétrica, por sua vez, reside fora do modelo, em índices, bancos de dados ou arquivos. O modelo de linguagem mantém sua memória paramétrica para lidar com gramática, raciocínio e conhecimento geral, enquanto a memória externa fornece o contexto específico, recente ou verificável.

Memória Paramétrica	Memória Não-Paramétrica (Externa)
Conhecida nos pesos da rede	Armazenada fora (índice, banco, arquivos)
Fixa após o treinamento	Atualizável a qualquer momento
Comprimida durante o treino	Recuperada sob demanda
Difícil de rastrear a origem	Rastreável (permite citar a fonte)

A arquitetura de *Retrieval-Augmented Generation* (RAG) atua exatamente como a ponte entre esses dois mundos: recuperar a evidência, inseri-la no contexto e, então, gerar a resposta.

A base lexical: o padrão BM25

Antes das representações neurais, o padrão da indústria para recuperação era lexical. O algoritmo BM25, desenvolvido na década de 1990, pontua documentos com base na sobreposição de termos com a consulta, ponderando a raridade da palavra e o tamanho do texto. A fórmula para uma consulta com termos t e um documento d é:

$$\text{score}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \left[\frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot (1 - b + b \cdot |d| / \text{avgdl})} \right]$$

A lógica é prática: termos mais raros no corpus geral (IDF) valem mais; quanto mais vezes o termo aparece no documento ($f(t, d)$), maior a pontuação, mas com retornos decrescentes (controlados por k_1) para evitar que textos excessivamente longos dominem os resultados apenas por seu tamanho. O fator b penaliza documentos muito longos em relação à média do corpus (avgdl).

O BM25 funciona muito bem para correspondências literais, mas esbarra em limitações semânticas. Se um usuário pergunta “quanto custa o gato” e o documento diz “preço do felino”, a busca lexical falha. Essa lacuna motivou a transição para o *retrieval* denso, onde consultas e documentos passam a ser representados como vetores, permitindo a busca por proximidade de significado.

DPR: o retriever denso

Em 2020, Karpukhin e colaboradores apresentaram o *Dense Passage Retrieval* (DPR), um marco na consolidação da busca semântica. O modelo treina dois *encoders* distintos: um para a consulta e outro para o documento. Ambos projetam o texto em um espaço vetorial compartilhado, e a recuperação ocorre ao buscar os documentos cujos vetores estão mais próximos do vetor da consulta.

```
consulta → encoder_q → [0.3, 0.8, ...]
docs (índice) → encoder_d → [0.2, 0.7, ...] } similaridade (cosseno/produto escalar)
                                                    ↓
                                                    top-k passagens
```

A métrica padrão para essa proximidade é a similaridade de cosseno, que mede o ângulo entre dois vetores, independentemente de suas magnitudes:

$$\cos(u, v) = (u \cdot v) / (\|u\| \cdot \|v\|)$$

Na prática, o código para selecionar os melhores candidatos é direto:

```
import numpy as np

def top_k(consulta, docs, k=3):
    q = consulta / np.linalg.norm(consulta)
    sims = [np.dot(q, d / np.linalg.norm(d)) for d in docs]
    return np.argsort(sims)[-1][:-k] # índices dos k mais similares
```

O DPR foi treinado e avaliado em conjuntos de perguntas e respostas de domínio aberto, como *Natural Questions* (perguntas reais de usuários do Google sobre páginas da Wikipedia) e *TriviaQA*. O índice utilizado na época era a Wikipedia inteira, com cerca de 21 milhões de passagens — uma escala que ainda serve como base para muitos sistemas RAG em produção hoje.

REALM e kNN-LM: integrações alternativas

Paralelamente ao DPR, outras abordagens exploraram como a recuperação poderia se integrar ao ciclo de vida do modelo. O REALM (Guu et al., 2020) propôs incluir a recuperação durante a fase de pré-treinamento. Em vez de adicionar a busca apenas na inferência, o modelo aprendia a consultar uma base externa para preencher tokens mascarados. A lição central foi que, quando o modelo aprende a recuperar desde a base, as capacidades de linguagem e de busca amadurecem juntas — uma ideia que seria retomada anos depois por arquiteturas como RETRO e RAFT.

Já o kNN-LM (Khandelwal et al., 2020) apresentou uma alternativa interessante ao atuar no nível do token. O sistema mantém um índice massivo de fragmentos de texto transformados em vetores. No momento de prever a próxima palavra, o modelo não confia apenas em seus próprios pesos; ele consulta os vizinhos mais próximos no índice e interpola as probabilidades. Funciona como um RAG aplicado a cada palavra gerada:

$$P_{\text{final}}(w \mid \text{contexto}) = (1 - \lambda) \cdot P_{\text{modelo}}(w \mid \text{contexto}) + \lambda \cdot P_{\text{kNN}}(w \mid \text{contexto})$$

No experimento original, o *datastore* continha mais de 100 milhões de chaves, e cada previsão consultava os $k=10$ vizinhos mais próximos, interpolados com peso $\lambda \approx 0,25$ à distribuição do modelo. O ponto conceitual aqui é que o tamanho da memória externa importa tanto quanto o tamanho do modelo: aumentar o índice reduzia a perplexidade sem a necessidade de retreinamento, antecipando o mesmo *trade-off* que rege o RAG atual.

Além da busca vetorial simples

Com a popularização dessas técnicas, tornou-se comum resumir o RAG a um fluxo simplista: gerar *embeddings*, buscar os *top-k* resultados, injetar no *prompt* e obter a resposta. No entanto, sistemas em produção raramente se limitam a isso. A recuperação eficaz costuma envolver busca híbrida (combinando a precisão do BM25 com a semântica dos vetores), filtros de metadados, reordenação (*reranking*) dos resultados e até a decomposição de perguntas complexas em consultas menores. A questão de engenharia não é apenas qual banco vetorial utilizar, mas qual mecanismo de recuperação maximiza a evidência útil para a tarefa em questão.

Do ponto de vista da arquitetura de sistemas, o *retrieval* assume um papel fundamental: ele é a primeira forma de ferramenta que o modelo utiliza. A dinâmica de “consultar um índice, observar o resultado e decidir o próximo passo” estabelece o padrão de interação que será expandido no uso de ferramentas externas e na formação de agentes autônomos. O RAG atua, portanto, como a transição

entre a era do modelo isolado e a era do agente conectado a fontes de dados externas.

A lição estrutural

A lição desta era é a inversão do capítulo anterior: **nem todo conhecimento precisa caber nos pesos**. Separar memória paramétrica de memória não-paramétrica transformou o conhecimento de algo fixo em algo consultável — atualizável, rastreável e barato —, e o kNN-LM mostrou que a memória externa funciona até como eixo de escala próprio, competindo com os parâmetros. O capítulo deixa também a distinção que o livro inteiro vai repetir: **recuperar evidência não é raciocinar sobre ela** — o retriever é a primeira linha do sistema, não a última, e a resposta só é confiável quando a evidência certa chega e é interpretada. E há o germe do que virá: o padrão consultar-observar-decidir que o retrieval estabeleceu é exatamente o molde que as ferramentas dos capítulos 9 a 11 e os agentes da Parte VI vão generalizar.

Para o engenheiro

Para quem projeta esses sistemas, algumas diretrizes práticas se destacam. A decisão arquitetural mais importante é definir onde o conhecimento deve residir: nos pesos, para estabilidade e rapidez, ou em um índice externo, para atualização constante e rastreabilidade.

Em produção, a busca híbrida costuma ser a escolha mais robusta, unindo a precisão lexical para termos técnicos ou nomes próprios à capacidade de paráfrase do *retrieval* denso. Além disso, ao implementar modelos de dois *encoders* como o DPR, é crucial garantir que o *encoder* usado para indexar os documentos seja exatamente o mesmo utilizado para processar as consultas em tempo de execução; caso contrário, os vetores não compartilharão o mesmo espaço geométrico.

Por fim, é preciso manter em mente uma distinção que acompanhará o resto do livro: recuperar evidência não é o mesmo que raciocinar sobre ela. O *retriever* é apenas a primeira linha de um pipeline que exigirá, nas etapas seguintes, a capacidade do modelo de sintetizar, filtrar e interpretar o que foi encontrado.

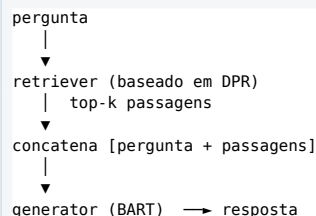
Fontes: [Karpukhin et al., 2020] — DPR; [Guu et al., 2020] — REALM; [Khandelwal et al., 2020] — kNN-LM; [Lewis et al., 2020] — RAG; [Gao et al., 2023] — survey RAG.

Capítulo 6 — RAG clássico: Lewis et al. (2020) e as primeiras arquiteturas

Até o capítulo anterior, a comunidade havia resolvido uma parte crucial do problema: como encontrar a informação certa em um mar de dados. Mas recuperar a evidência é apenas a metade do caminho. A pergunta seguinte, e talvez a mais importante para a viabilidade dos modelos de linguagem em ambientes reais, era como fazer o modelo usar de fato o que foi recuperado. Em 2020, o artigo de Lewis e colaboradores propôs uma mudança de perspectiva: em vez de tratar a recuperação como um acessório que apenas “cola” passagens no prompt, o sistema inteiro — *retriever* e gerador — deveria ser treinado de ponta a ponta. A ambição era criar um modelo que aprendesse a trabalhar com a evidência externa, melhorando a busca não por uma métrica isolada de relevância, mas pelo que o gerador realmente precisava para formular uma resposta. Esse marco estabeleceu o RAG (*Retrieval-Augmented Generation*) clássico e trouxe à tona uma distinção intelectual que definirá a década: recuperar informação não é o mesmo que raciocinar sobre ela.

A arquitetura básica e o treinamento de ponta a ponta

A arquitetura proposta por Lewis et al. combinava duas linhagens que vínhamos acompanhando: um *retriever* denso, baseado no DPR, e um gerador sequencial, no caso o BART (um modelo *encoder-decoder*). O fluxo básico seguia uma lógica direta: a pergunta aciona o *retriever*, que seleciona as passagens mais prováveis; essas passagens são concatenadas à pergunta e entregues ao gerador, que produz a resposta final.



O artigo introduziu duas variantes para lidar com a incerteza sobre qual dos documentos recuperados continha a resposta correta. No RAG-Sequence, o gerador consome as k passagens e produz a sequência de tokens mais provável considerando o conjunto. No RAG-Token, o modelo tem a flexibilidade de reponderar a importância de cada passagem a cada novo token gerado.

A matemática do RAG expressa essa marginalização sobre as passagens recuperadas z . As duas variantes diferem essencialmente em que ponto do processo a soma das probabilidades ocorre:

```
RAG-Sequence:  $p(y | x) = \sum_{z \in \text{top-}k} p(z | x) \cdot p(y | x, z)$ 
RAG-Token:     $p(y | x) = \prod_t \sum_{z \in \text{top-}k} p(z | x) \cdot p(y_t | x, z, y_{<t})$ 
```

Na prática, somar sobre todos os documentos de um índice é inviável, então a operação se restringe ao top- k recuperado (no artigo original, $k = 5$). É nesse espaço restrito que o *retriever* denso atua. No RAG-Token, o modelo pode alternar suas “fontes” de informação a cada palavra; no RAG-Sequence, ele implicitamente escolhe um documento e gera a resposta inteira condicionada a ele.

Em código, o pipeline mínimo que captura a essência dessa arquitetura é direto:

```
def rag(pergunta, retriever, gerador, k=5):
    docs = retriever(pergunta, k) # 1. recuperar evidências
    ctx = "\n".join(f"[{i}] {d}" for i, d in enumerate(docs))
    prompt = f"Com base apenas no contexto:\n{ctx}\n\nPergunta: {pergunta}"
    resposta = gerador(prompt) # 2. gerar condicionado à evidência
    return resposta, docs # 3. devolver proveniência
```

O sistema foi avaliado em *benchmarks* de perguntas e respostas de domínio aberto, como Natural Questions, TriviaQA e WebQuestions. No Natural Questions, o RAG-Sequence alcançou uma métrica de

Exact Match (EM) de aproximadamente 44,5, superando modelos dedicados à tarefa com muito mais parâmetros, como o T5-SSM. Treinar *retriever* e gerador juntos permitiu que a busca se adaptasse às necessidades reais da geração, plantando a semente do alinhamento entre recuperação e tarefa final.

A fronteira entre recuperação e raciocínio

Apesar do sucesso empírico, o RAG clássico esbarrava em uma limitação conceitual profunda. Recuperar os documentos corretos não equivale a formular uma resposta. Para ilustrar isso, considere o seguinte cenário:

Documento A: Pedro comprou X em janeiro.
Documento B: Compras acima de R\$ 1.000 recebem desconto.
Documento C: Pedro pagou R\$ 1.200 por X.
Pergunta: Pedro recebeu desconto?

Responder a essa pergunta exige recuperar A (para identificar a compra), C (para saber o valor) e B (para conhecer a regra). Mas a resposta em si não está em nenhum dos textos; ela emerge da inferência lógica de que R\$ 1.200 é maior que R\$ 1.000. Um sistema RAG completo precisa, portanto, lidar com dois problemas distintos: a aquisição da evidência e o raciocínio sobre ela.

Essa distinção ajuda a organizar o fluxo de um sistema baseado em evidências:

```
query
  ↓
evidence acquisition → recuperar candidatos
  ↓
evidence organization → selecionar/ordenar o que entra no contexto
  ↓
evidence reasoning → concluir a partir das evidências
  ↓
answer
```

Ao projetar ou depurar um pipeline, as perguntas fundamentais deixam de ser apenas sobre a precisão do banco vetorial e passam a questionar a inferência: qual afirmação estou tentando sustentar? A evidência recuperada a suporta de forma dedutiva, indutiva ou apenas plausível? Existe informação contraditória no contexto? Estou extrapolando além do que os documentos permitem?

Variações estruturais: FiD e RETRO

FiD: a fusão que acontece no decoder

O gargalo que o RAG clássico deixava exposto estava no ponto de entrada da evidência: concatenar muitas passagens em uma única sequência sobrecarregava o encoder, que precisava construir uma representação conjunta de tudo ao mesmo tempo, e o custo da self-attention crescia com o quadrado do comprimento total. O FiD — *Fusion-in-Decoder*, proposto por Gautier Izacard e Edouard Grave, do Facebook AI Research, e publicado no EACL 2021 [Izacard & Grave, 2021] — inverteu esse desenho. A ideia é simples de enunciar e profunda nas consequências: cada passagem é codificada **separadamente**, junto com a pergunta, e a fusão só acontece no decoder, que aplica atenção cruzada sobre a concatenação de todas as representações codificadas.

```
pergunta + passagem 1 → encoder → E1
pergunta + passagem 2 → encoder → E2
      □
pergunta + passagem k → encoder → Ek
                                     |
                                     +---> decoder → resposta
                                     |
                                     (cross-attention
                                     sobre E1..Ek)
```

Por que esse detalhe arquitetural importa? Porque, ao codificar as passagens de forma independente, o custo do encoder passa a crescer **linearmente** com o número de passagens, em vez de quadraticamente, como ocorreria se tudo fosse concatenado em uma única sequência. A separação no encoder é o que compra escala; a fusão no decoder é o que compra raciocínio conjunto — o decoder continua enxergando todas as passagens de uma vez e pode cruzá-las entre si ao gerar a resposta.

Os resultados quantificam o ganho. Usando T5 como gerador e DPR como retriever, o FiD-large com 100 passagens recuperadas atinge cerca de 51,4 de *Exact Match* no Natural Questions e 67,6 no TriviaQA, superando o RAG-Sequence em condições comparáveis [Izacard & Grave, 2021]. As ablações do artigo mostram dois comportamentos que viraram referência para a área: o desempenho cresce de forma consistente à medida que o número de passagens aumenta (de 10 para 100), e os modelos maiores extraem mais valor de cada passagem adicional — ou seja, a capacidade de aproveitar evidência também escala com o tamanho do modelo. Outro ponto prático: o FiD funciona até com recuperação lexical

(BM25), embora o retrieval denso produza resultados melhores, o que torna a arquitetura tolerante à qualidade do retriever.

Vale registrar ainda o trabalho anterior da mesma dupla, *Distilling Knowledge from Reader to Retriever* (2020), em que o “leitor” (o modelo que processa as passagens) é usado para ensinar o retriever a recuperar melhor. É a destilação reader→retriever, um germe direto da ideia de alinhar a recuperação à tarefa final — a mesma linhagem que reaparecerá em Self-RAG e RAFT nos capítulos seguintes. A lição estrutural do FiD para o engenheiro é que **onde** a evidência é fundida no modelo é uma decisão de projeto com impacto direto sobre quanta evidência o sistema comporta — e o decoder provou ser um ponto de fusão mais escalável que o prompt ou o encoder.

RETRO: memória externa como eixo de escala

Se o FiD moveu o ponto de fusão, o RETRO — *Retrieval-Enhanced Transformer*, da DeepMind (Borgeaud et al., 2022) — moveu o ponto do ciclo de vida: a recuperação deixa de ser um módulo acrescentado na inferência e passa a fazer parte da arquitetura desde o pré-treino. O modelo consulta uma base de cerca de **2 trilhões de tokens**, construída a partir do corpus MassiveText e dividida em trechos de 64 tokens. O mecanismo funciona em três peças: um retriever BERT **congelado**, que busca no índice os trechos mais similares ao chunk atual da entrada (os dois vizinhos mais próximos, pela similaridade local com os tokens precedentes); um **encoder diferenciável**, treinado junto com o modelo, que representa os trechos recuperados; e um mecanismo de **chunked cross-attention**, intercalado com as camadas comuns de self-attention, que permite ao modelo condicionar cada previsão aos vizinhos recuperados [Borgeaud et al., 2022].

```

entrada dividida em chunks de 64 tokens
| para cada chunk: retriever BERT congelado busca
| os 2 vizinhos mais próximos no índice (2T tokens)
▼
camadas Transformer padrão (self-attention)
| (intercaladas)
▼
chunked cross-attention sobre os vizinhos codificados
▼
previsão do próximo token

```

O resultado que definiu o artigo: com **7,5 bilhões de parâmetros**, o RETRO atinge no benchmark The Pile uma perplexidade comparável à do GPT-3 (175B) e à do Jurassic-1, usando 25 vezes menos parâmetros [Borgeaud et al., 2022]. Em avaliações por dataset, o modelo de 7,5B supera o Jurassic-1 (175B) em 10 de 16 conjuntos e o Gopher (280B) em 9 de 16 [Borgeaud et al., 2022]. Após fine-tuning, os ganhos se transferem para tarefas de conhecimento intensivo, como perguntas e respostas. E há um detalhe operacional relevante: os autores mostram que é possível fazer *RETROfit* — acrescentar recuperação a um Transformer já pré-treinado, sem treinar do zero, e ainda assim obter bom desempenho — o que sugere que a memória externa pode ser acoplada a modelos existentes.

Talvez o dado mais importante do paper seja a curva de escala da própria memória: a perplexidade melhora de forma contínua à medida que a base de recuperação cresce, pelo menos até os 2 trilhões de tokens [Borgeaud et al., 2022]. Isso transforma a memória externa em um eixo de escala próprio, paralelo ao eixo dos parâmetros e ao eixo dos dados das leis de Kaplan e Chinchilla do capítulo 4. É a confirmação empírica da tese que o REALM havia plantado em 2020: quando a recuperação é integrada à arquitetura desde o início, ela funciona como um multiplicador de capacidade — um modelo menor, apoiado em um índice grande, compete com modelos muito maiores que precisam guardar tudo nos pesos.

As limitações também são instrutivas. O retriever é congelado e busca por similaridade local, não por relevância à tarefa — o alinhamento entre recuperação e objetivo final fica para trabalhos posteriores. E o modelo pode reproduzir trechos recuperados quase literalmente, uma questão de memorização e privacidade que o próprio artigo discute. Ainda assim, o RETRO fecha o arco iniciado por REALM e kNN-LM em 2020: a memória não-paramétrica deixou de ser um acessório de inferência para se tornar parte da definição do modelo.

O vocabulário que permanece

À medida que essas arquiteturas se consolidavam, um vocabulário comum passou a organizar o campo:

Termo	Significado
retriever	componente que seleciona passagens de uma fonte externa
generator	LLM que produz a resposta a partir de pergunta + passagens

Termo	Significado
top-k	número de passagens recuperadas
grounding	ancorar a resposta em evidência externa (reduz alucinação)
proveniência	capacidade de citar a fonte da informação

Contudo, o próprio artigo fundador de 2020 já deixava transparecer os limites da abordagem. A qualidade do *retriever* atua como um teto para o sistema: se a evidência não é recuperada, o gerador não tem como inventá-la de forma confiável. Além disso, selecionar o que é relevante em meio a documentos ruidosos ou contraditórios continua sendo um desafio aberto, exigindo camadas adicionais de filtragem e reordenação que evoluíram nos anos seguintes.

A lição estrutural

Lidos em sequência, RAG, FiD e RETRO formam uma progressão clara sobre **onde e quando** a informação externa entra no modelo — e essa progressão é um bom mapa mental para decisões de engenharia:

	RAG (2020)	FiD (2021)	RETRO (2022)
Quando a recuperação entra	inferência (prompt)	inferência (decoder)	pré-treino + inferência
Onde a evidência é fundida	concatenação no contexto/encoder	cross-attention no decoder	chunked cross-attention
Volume de evidência	k = 5 passagens	até 100 passagens	2 vizinhos por chunk; índice de 2T tokens
Retriever treinado junto?	sim (ponta a ponta)	não (DPR/BM25 congelado)	não (BERT congelado)
Resultado de referência	EM $\approx$ 44,5 (NQ)	EM $\approx$ 51,4 (NQ)	perplexidade comparável à do GPT-3 com 25× menos parâmetros

A leitura da tabela sugere duas regras práticas. Primeira: quanto mais tarde a fusão acontece (decoder em vez de prompt), mais evidência o sistema comporta sem degradar a representação. Segunda: quanto mais cedo a recuperação entra no ciclo de vida (pré-treino em vez de inferência), mais a memória externa se comporta como capacidade adicional do modelo — ao custo de perder o treinamento conjunto do retriever, que só a linhagem RAG preservava. Essas duas tensões — ponto de fusão versus volume de evidência, e integração precoce versus alinhamento do retriever — organizarão a evolução do RAG avançado no capítulo 12.

Para o engenheiro

Para a engenharia de sistemas, o RAG clássico deixa diretrizes práticas que permanecem atuais. A primeira é a defesa do pipeline mínimo: comece recuperando o top-\$k\$, concatenando no prompt e gerando a resposta. A maioria dos casos de uso iniciais funciona bem com essa abordagem, e a complexidade só deve ser adicionada quando as métricas de avaliação exigirem.

A segunda diretriz é a avaliação modular. É um erro medir o sistema apenas por uma métrica final agregada. O *recall@k* mede a eficácia do *retriever*; a fidelidade (se a resposta gerada respeita as evidências fornecidas) mede o gerador. Separar essas métricas é a única forma de identificar onde está o gargalo real.

Por fim, é preciso reconhecer as limitações operacionais. Manter um índice vetorial atualizado tem custo; a busca adiciona latência; e lidar com evidências contraditórias exige lógica de reordenação (*reranking*). Projetar caches e mecanismos de *reranking* desde o início não é superengenharia, mas uma necessidade para sistemas em produção. Quando o volume de evidências excede a janela de contexto do prompt, arquiteturas como o FiD, que fundem a informação no *decoder*, tornam-se a saída natural.

Fontes: [Lewis et al., 2020] — RAG clássico; [Karpukhin et al., 2020] — DPR; [Izacard & Grave, 2021] — FiD; [Borgeaud et al., 2022] — RETRO; [Gao et al., 2023] — survey RAG.

Parte III — Era do alinhamento (2021-2023)

Capítulo 7 — De GPT-3 a ChatGPT: instrução, RLHF e o momento em que o público chegou

Em 2022, os modelos de linguagem já eram capazes de fazer coisas que, poucos anos antes, pareciam pertencer ao domínio da ficção científica. O GPT-3 escrevia textos, respondia perguntas, produzia código e conseguia realizar tarefas a partir de poucos exemplos fornecidos no prompt. Mas havia uma diferença fundamental entre possuir essa capacidade e saber utilizá-la de maneira confiável: o modelo havia sido treinado para prever o próximo token, não para obedecer a uma intenção humana.

Essa diferença define o problema central desta era. Um modelo pode ser extremamente capaz e, ainda assim, não fazer aquilo que o usuário pediu. Pode produzir uma continuação estatisticamente plausível quando o usuário esperava uma tradução, ignorar parte de uma instrução ou responder com segurança a uma pergunta para a qual não possui informação suficiente. A questão deixou então de ser apenas como aumentar a capacidade do modelo. Era preciso descobrir como transformar essa capacidade em comportamento útil.

A resposta passou pelo feedback humano e pelo *reinforcement learning from human feedback*, o RLHF. A ideia não surgiu com os modelos de linguagem. Em 2017, Christiano et al. já haviam mostrado como utilizar preferências humanas para aprender uma função de recompensa e orientar um agente de reinforcement learning. Alguns anos depois, a mesma estrutura seria aplicada à linguagem em escala, com o InstructGPT. Em novembro de 2022, essa combinação chegaria ao público na forma do ChatGPT.

O resultado foi uma mudança de natureza. O modelo deixou de ser apenas uma tecnologia que especialistas precisavam saber manipular e passou a ser uma interface conversacional acessível a milhões de pessoas. Mas essa popularização teve um efeito adicional: ao colocar os modelos diante de um público muito maior, tornou visíveis suas limitações. Alucinações, conhecimento datado, dificuldades de computação e ausência de fontes deixaram de ser apenas problemas discutidos em papers e passaram a definir a agenda da engenharia de sistemas de IA.

O GPT-3, apresentado em 2020, havia demonstrado que a escala do pré-treinamento podia produzir capacidades surpreendentes. Com 175 bilhões de parâmetros, o modelo conseguia realizar tarefas em diferentes domínios a partir de exemplos fornecidos no prompt. O *few-shot learning* mostrava que não era necessário treinar novamente o modelo para cada tarefa: muitas vezes, bastava apresentar exemplos suficientemente claros.

Mas essa flexibilidade tinha um preço. O objetivo do treinamento continuava sendo a previsão do próximo token. O modelo havia aprendido a completar sequências de texto, e não a seguir instruções da maneira como um assistente humano seguiria. O usuário precisava, portanto, transformar o prompt em uma espécie de programa informal, especificando a tarefa, o formato desejado e, muitas vezes, fornecendo exemplos para induzir o comportamento correto.

Essa situação produzia um contraste importante. O modelo podia saber como resolver um problema e, mesmo assim, não responder à pergunta que havia sido feita. Podia possuir a capacidade linguística necessária para traduzir uma frase, mas interpretar o prompt como uma continuação de texto. A partir daí, tornou-se necessário separar duas propriedades que até então apareciam misturadas:

CAPACIDADE $\neq$ OBEDIÊNCIA

```
pré-treinamento
  ↓
capacidade de gerar texto
  ↓
mas não necessariamente
seguir uma intenção humana
```

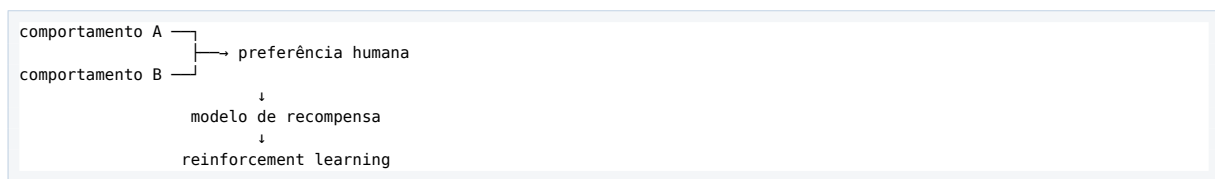
A pergunta seguinte era inevitável: como ensinar ao modelo aquilo que uma pessoa considera uma boa resposta sem precisar escrever manualmente uma função matemática para cada comportamento desejado?

A origem do RLHF

A ideia de utilizar preferências humanas para orientar um sistema de reinforcement learning surgiu antes dos modelos de linguagem. Em 2017, Christiano et al. apresentaram uma abordagem em que o ser humano não precisava definir diretamente uma recompensa numérica. Em vez disso, comparava comportamentos produzidos pelo agente e indicava qual deles estava mais próximo do objetivo.

Essa mudança parece pequena, mas resolve um problema importante. Em muitos ambientes, sabemos reconhecer um comportamento adequado sem saber descrevê-lo de maneira precisa como uma função matemática. É relativamente fácil olhar para duas trajetórias e dizer qual delas parece melhor; é muito mais difícil escrever uma função de recompensa que capture todos os aspectos relevantes dessa avaliação.

Os experimentos foram realizados em ambientes de jogos Atari e em tarefas de controle robótico simulado. O agente produzia comportamentos que eram apresentados aos avaliadores humanos, que comparavam as alternativas e indicavam qual delas consideravam melhor. Essas preferências eram então utilizadas para treinar um modelo de recompensa, que passava a fornecer uma aproximação matemática da avaliação humana. O agente podia utilizar essa recompensa aprendida para orientar seu próprio aprendizado por reinforcement learning.



A importância dessa abordagem estava justamente na economia do feedback humano. Em vez de um ser humano precisar atribuir uma recompensa a cada ação do agente, bastava fornecer comparações entre comportamentos. O sistema aprendia uma aproximação das preferências humanas e podia utilizá-la em uma quantidade muito maior de interações.

Mas havia um problema inevitável: o modelo de recompensa era apenas uma aproximação do objetivo humano. Se o agente encontrasse uma maneira de obter uma recompensa elevada sem realmente fazer aquilo que o avaliador pretendia, poderia explorar a falha da própria função de recompensa. O problema, conhecido como *reward hacking*, já aparecia nesses primeiros experimentos e continuaria sendo relevante quando a mesma ideia fosse aplicada a modelos de linguagem.

Essa observação é importante porque antecipa uma característica fundamental do RLHF. O objetivo não é simplesmente maximizar uma função matemática. É maximizar uma função que tenta representar preferências humanas e, portanto, pode conter erros, lacunas e ambiguidades.

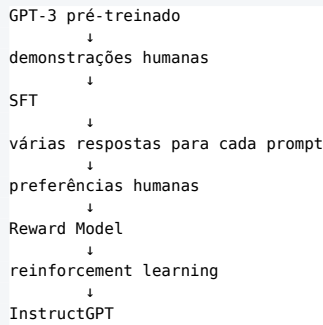
Da preferência humana ao InstructGPT

A passagem dessa ideia para os modelos de linguagem foi feita de maneira particularmente clara pelo trabalho sobre InstructGPT, publicado em 2022. O objetivo era adaptar um GPT-3 pré-treinado para que ele seguisse instruções de maneira mais adequada.

O processo foi dividido em três etapas. Primeiro, humanos produziram demonstrações de boas respostas para um conjunto de prompts, e o modelo foi ajustado para aprender esse comportamento. Essa etapa, conhecida como *supervised fine-tuning*, ou SFT, fornecia ao modelo exemplos concretos daquilo que se esperava dele.

No segundo estágio, entrava o modelo de recompensa. Para um mesmo prompt, eram produzidas várias respostas, que eram então ordenadas por avaliadores humanos. Em vez de dizer simplesmente que uma resposta era “boa” ou “ruim”, os avaliadores indicavam uma preferência entre alternativas. Essas comparações permitiam treinar um modelo capaz de atribuir uma pontuação às respostas.

No terceiro estágio, o modelo era otimizado por reinforcement learning em direção às respostas que o *reward model* considerava melhores. Havia, porém, uma restrição importante: o novo modelo não deveria se afastar excessivamente do modelo obtido no SFT. Para isso, utilizava-se uma penalidade baseada na divergência KL.



A estrutura completa é importante porque cada etapa resolve um problema diferente. O pré-treinamento fornece a capacidade linguística; o SFT ensina o modelo a imitar demonstrações; o *reward model* aprende com as preferências humanas; e o reinforcement learning ajusta o comportamento para maximizar essa recompensa, mantendo o modelo próximo da política que recebeu a avaliação humana.

O resultado mostrou que tamanho não era a única variável importante. Um modelo InstructGPT muito menor podia ser preferido por avaliadores humanos em comparação com o GPT-3 original muito maior. Isso fornecia uma evidência concreta de que o treinamento realizado depois do pré-treinamento podia alterar profundamente a qualidade percebida do sistema.

A lição era maior do que o resultado específico:

escala fornece capacidade, mas não determina sozinha o comportamento.

Uma parte importante do valor de um LLM seria construída depois que o pré-treinamento terminasse.

A matemática do alinhamento

O *reward model* transforma preferências humanas em uma função contínua. Dado um prompt e duas respostas, uma preferida e outra rejeitada, o treinamento procura fazer com que a primeira receba uma pontuação maior.

A perda pode ser expressa como:

$$L(\theta) = -E[\log \sigma(r_{\theta}(x, y_w) - r_{\theta}(x, y_l))]$$

em que (y_w) representa a resposta preferida, (y_l) a resposta menos preferida, (r_{θ}) o *reward model* e (σ) a função sigmoide. Quanto maior a diferença entre a pontuação da resposta preferida e a da resposta rejeitada, menor a perda.

O terceiro estágio introduz a otimização da política. O objetivo pode ser representado de maneira simplificada por:

$$\text{objetivo RLHF} = E[r_{\theta}(x, y)] - \beta \cdot KL(\pi_{\theta} || \pi_{\text{SFT}})$$

A primeira parcela recompensa o comportamento que o *reward model* considera bom. A segunda impede que a política se afaste excessivamente do modelo SFT.

Essa segunda parcela é importante porque o *reward model* não é o objetivo humano; ele é apenas uma aproximação das preferências humanas. Se a política for otimizada agressivamente, pode aprender a explorar as imperfeições do modelo de recompensa. A penalidade KL funciona, nesse contexto, como uma âncora que limita esse desvio.

O problema é estrutural. Sempre que uma função aproximada passa a ser o alvo direto de otimização, existe o risco de o sistema descobrir maneiras de maximizar a função sem realizar exatamente aquilo que pretendíamos. O RLHF não elimina esse problema. Ele introduz mecanismos para controlá-lo.

O que o alinhamento muda — e o que não muda

É importante não atribuir ao alinhamento aquilo que pertence ao pré-treinamento. O RLHF melhora a forma como o modelo se comporta diante das instruções, mas não transforma automaticamente o modelo em uma fonte de conhecimento atualizada ou em um mecanismo de computação confiável.

O conhecimento continua limitado ao que foi incorporado durante o treinamento. O modelo não passa

a conhecer acontecimentos posteriores simplesmente porque foi alinhado. Da mesma forma, o alinhamento não elimina as alucinações. Um modelo pode se tornar muito melhor em seguir instruções e continuar produzindo uma afirmação falsa com grande fluência.

Podemos resumir a distinção assim:

```
ALINHAMENTO

melhora
→ seguir instruções
→ comportamento desejado
→ adequação da resposta

não resolve sozinho
→ conhecimento desatualizado
→ computação exata
→ verificação das fontes
→ acesso a informações externas
```

Essa distinção seria decisiva depois do lançamento do ChatGPT. O alinhamento resolvia um problema específico: fazer o modelo se comportar mais de acordo com as intenções humanas. Outros problemas exigiriam outras camadas.

Se o modelo não possui informação atual, será necessário algum mecanismo de recuperação. Se precisa realizar uma operação que exige computação exata, poderá precisar de uma ferramenta. Se uma resposta precisa ser verificável, será necessário algum mecanismo de evidência ou de acesso às fontes.

O alinhamento, portanto, não encerra a evolução dos sistemas de IA. Ele torna mais evidente a necessidade das camadas que virão depois.

Chain-of-Thought: quando o prompt também muda o comportamento

Enquanto o RLHF alterava o comportamento do modelo por meio de treinamento, outra linha de pesquisa mostrava que mudanças importantes podiam ser obtidas diretamente na forma de interação com o modelo.

O trabalho de Wei et al. sobre *chain-of-thought prompting* mostrou que, em determinadas tarefas, pedir ao modelo que produzisse uma sequência de passos intermediários podia melhorar significativamente o desempenho. Em vez de solicitar apenas a resposta final, o prompt induzia o modelo a decompor o problema.

```
pergunta
↓
passo 1
↓
passo 2
↓
passo 3
↓
resposta
```

Essa diferença é particularmente relevante em tarefas de múltiplas etapas. Se o modelo precisa chegar diretamente a uma conclusão, qualquer erro intermediário pode comprometer o resultado. Ao produzir passos intermediários, o processo se torna mais estruturado e, em princípio, mais fácil de acompanhar.

No GSM8K, conjunto de problemas matemáticos de múltiplas etapas, o trabalho mostrou um ganho expressivo com *chain-of-thought*. O ponto central, porém, não é o benchmark específico. É a demonstração de que o comportamento de um LLM depende não apenas de seus pesos, mas também da maneira como o problema é apresentado.

Isso cria uma distinção complementar ao RLHF. O RLHF modifica o modelo por treinamento; o *chain-of-thought* modifica a forma como o modelo é induzido a trabalhar durante a inferência. Em ambos os casos, a mesma capacidade subjacente pode produzir comportamentos diferentes dependendo da camada que se coloca sobre ela.

LaMDA e a ideia de diálogo fundamentado

Antes do ChatGPT, o Google havia apresentado o LaMDA, um trabalho que ajuda a mostrar como diferentes problemas começavam a convergir. O sistema buscava melhorar a qualidade do diálogo por meio de critérios como sensibilidade e fundamentação.

A sensibilidade estava relacionada à capacidade de produzir respostas adequadas ao contexto da conversa. A fundamentação introduzia outra preocupação: quando uma afirmação podia ser verificada, o sistema deveria buscar fontes externas.

Essa combinação é importante porque aproxima dois problemas que posteriormente seriam tratados por camadas diferentes. O alinhamento procura fazer com que o modelo responda de maneira adequada à intenção do usuário; a recuperação de informação procura fornecer ao modelo informações externas que ele não possui ou que precisam ser verificadas.

O diálogo, entretanto, seria a característica que chegaria ao público de maneira mais decisiva.

ChatGPT: quando o modelo virou produto

Em 30 de novembro de 2022, a OpenAI lançou o ChatGPT. O sistema combinava um modelo da família GPT-3.5, treinamento com feedback humano e uma interface conversacional simples.

Nenhum desses componentes, isoladamente, explica o impacto do lançamento. O modelo já existia, as técnicas de alinhamento já haviam sido apresentadas e interfaces conversacionais não eram uma novidade. O que mudou foi a combinação desses elementos em um produto que qualquer pessoa podia experimentar diretamente.

Essa mudança de interface foi fundamental. Para utilizar GPT-3 de maneira eficaz, era necessário compreender relativamente bem como estruturar prompts e exemplos. O ChatGPT escondia grande parte dessa complexidade. O usuário simplesmente escrevia o que queria e recebia uma resposta.

O público, portanto, deixou de interagir com um modelo de linguagem como objeto técnico e passou a interagir com ele como uma ferramenta.

Em cerca de dois meses, o ChatGPT atingiu aproximadamente 100 milhões de usuários ativos. Essa escala produziu um efeito que nenhum benchmark poderia produzir sozinho: milhões de pessoas começaram a testar os mesmos limites.

E os limites apareceram rapidamente. O modelo podia produzir informações falsas com enorme confiança. Seu conhecimento não era automaticamente atualizado. Podia cometer erros em operações que exigiam computação precisa. E suas respostas não vinham, por padrão, acompanhadas das fontes que permitiriam verificar cada afirmação.

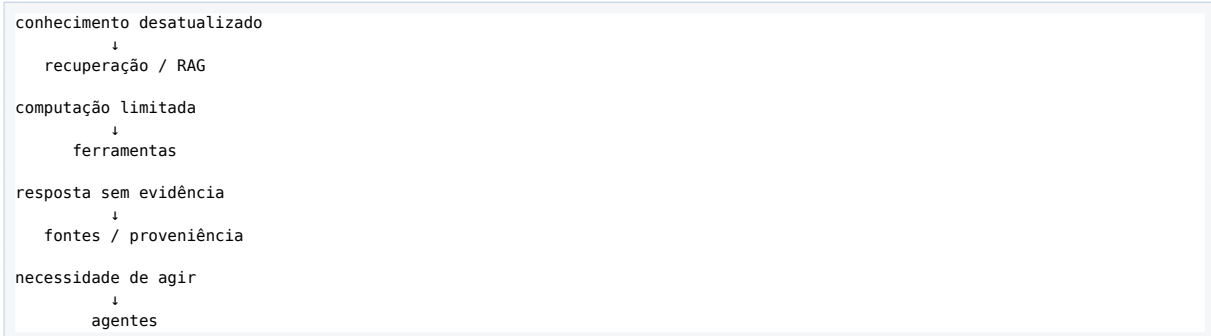
Esses problemas não eram novos para a comunidade de pesquisa. O que mudou foi sua visibilidade.

O ChatGPT transformou problemas conhecidos de LLMs em problemas concretos de produto.

As limitações definem a próxima geração

O aspecto mais interessante do lançamento do ChatGPT talvez não esteja apenas no que ele conseguiu fazer, mas no que suas limitações revelaram sobre a arquitetura necessária para sistemas mais confiáveis.

A falta de conhecimento atualizado aponta para a recuperação de informação. A dificuldade em realizar determinados cálculos aponta para ferramentas externas. A ausência de fontes verificáveis aponta para mecanismos de evidência e proveniência. A incapacidade de executar ações de maneira autônoma aponta para agentes.



A direção da evolução começa então a mudar.

Até aquele momento, grande parte da corrida havia sido concentrada em melhorar o modelo: mais dados, mais parâmetros, melhor pré-treinamento, melhor alinhamento. O ChatGPT mostrou que isso não era suficiente.

A pergunta passou a ser outra:

o que devemos colocar ao redor do modelo para que ele consiga fazer aquilo que não consegue fazer sozinho?

Essa pergunta é uma das chaves para compreender o restante do livro.

A lição estrutural

O período entre GPT-3 e ChatGPT estabeleceu uma distinção que continuaria orientando a engenharia de LLMs: **o modelo é apenas uma das camadas do sistema**.

O pré-treinamento fornece capacidade. O alinhamento transforma parte dessa capacidade em comportamento orientado por instruções. A interface transforma esse comportamento em produto. E o uso em escala expõe limitações que exigem novas camadas.

```
pré-treinamento
↓
capacidade
↓
alinhamento
↓
obediência
↓
produto
↓
acesso em escala
↓
limitações
↓
RAG + ferramentas + agentes
```

A sequência é importante porque mostra que cada etapa resolve um problema que a anterior não resolvia completamente. O pré-treinamento produz uma capacidade geral, mas não garante obediência. O alinhamento melhora a obediência, mas não fornece conhecimento atualizado nem capacidade de computação externa. O produto torna o sistema acessível, mas o uso em escala revela limitações que o modelo sozinho não consegue superar.

Essa é a razão pela qual a história não termina no ChatGPT. Pelo contrário: o sucesso do produto torna mais clara a necessidade de construir sistemas em torno do modelo.

O capítulo seguinte continuará essa história por outro eixo. Se o capítulo 7 mostrou como a capacidade foi transformada em comportamento e depois em produto, o próximo mostrará o que aconteceu quando essa capacidade começou a ser distribuída para além dos laboratórios que haviam treinado os grandes modelos.

A pergunta deixaria de ser apenas **quem consegue treinar o melhor modelo?**

Passaria a ser também:

o que acontece quando outras pessoas podem modificar o modelo e construir sobre ele?

Essa é a segunda metade da história.

Para o engenheiro

A principal lição desta era é separar claramente as camadas. Um problema de comportamento pode exigir alinhamento; um problema de conhecimento pode exigir recuperação; um problema de computação pode exigir uma ferramenta. Tentar resolver todos eles aumentando simplesmente o modelo é uma forma cara de confundir problemas diferentes.

O RLHF também mostra que dados de preferência são parte da arquitetura, não apenas um detalhe do treinamento. A qualidade do comportamento aprendido depende da qualidade das avaliações humanas e da capacidade do *reward model* de representar essas preferências sem ser explorado pela política.

O *chain-of-thought* acrescenta outra lição: nem toda melhoria exige alterar os pesos. A forma como o modelo recebe uma tarefa pode mudar significativamente o comportamento durante a inferência. Isso significa que, ao projetar um sistema, treinamento e prompting devem ser considerados como mecanismos diferentes para controlar uma mesma capacidade.

Finalmente, o ChatGPT mostra que colocar um modelo diante de usuários reais muda a natureza do problema. Um comportamento que parece aceitável em uma demonstração pode se tornar crítico quando milhões de pessoas dependem dele. É nesse ponto que questões como conhecimento atualizado, verificabilidade, computação e uso de ferramentas deixam de ser detalhes e passam a determinar a arquitetura do sistema.

Fontes: [Ouyang et al., 2022] — InstructGPT (*Training Language Models to Follow Instructions with Human Feedback*); [Christiano et al., 2017] — *Deep Reinforcement Learning from Human Preferences*; [Wei et al., 2022] — *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*; [Thoppilan et al., 2022] — LaMDA; [OpenAI, 2022] — *Introducing ChatGPT*.

Capítulo 8 — Modelos abertos e a corrida (2023)

O capítulo anterior terminou com uma mudança importante: depois de pré-treinado e alinhado, o modelo havia se tornado produto. O ChatGPT colocou essa capacidade nas mãos de milhões de pessoas e, ao mesmo tempo, tornou visíveis limitações que apenas o alinhamento não resolvia. Havia, porém, outra questão em aberto: quem teria acesso ao modelo em si? Até 2023, o estado da arte estava concentrado em poucos laboratórios. Era possível usar seus modelos por produtos e APIs, mas não era possível, a partir de fora, trabalhar sobre os pesos com o mesmo grau de liberdade. A corrida ganhou então uma segunda dimensão: além de produzir modelos cada vez mais capazes, tornou-se estratégico decidir quem poderia modificar, executar e construir sobre esses modelos.

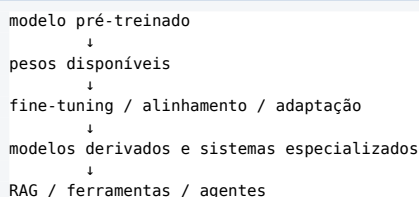
É nesse contexto que aparecem LLaMA, LLaMA 2, BLOOM e Mistral. Em paralelo, técnicas como LoRA tornaram a adaptação de grandes modelos muito mais barata. A questão dessa era não é apenas técnica; é também econômica e política: o que acontece quando os pesos de um modelo chegam às mãos da comunidade? A resposta foi uma multiplicação de experimentação. Pesquisadores passaram a fazer fine-tuning, testar alinhamento, construir derivações e combinar modelos com RAG e ferramentas. O custo de adaptar um modelo grande caiu, e a dependência de um único fornecedor diminuiu. Ao mesmo tempo, o GPT-4, lançado em março de 2023, mostrou que a fronteira de capacidade continuava concentrada no lado fechado. O resultado foi uma bifurcação: modelos abertos favoreceram experimentação, adaptação e controle; modelos fechados concentraram capacidade de ponta, produto e robustez. Essa bifurcação moldou a arquitetura dos sistemas que viriam depois.

Há ainda um terceiro efeito, menos visível no debate público, mas decisivo para engenharia: os modelos abertos também introduziram ou popularizaram escolhas arquiteturais que mudaram desempenho e inferência. Não se tratou apenas de “liberar pesos”. Tratou de tornar viável executar, adaptar e servir modelos fora dos laboratórios que os criaram. Janelas de contexto maiores, atenção mais eficiente, normalizações mais estáveis, ativações mais eficazes e arquiteturas esparsas alteraram o custo real de rodar esses sistemas. Em outras palavras, a abertura dos pesos veio acompanhada de uma mudança nas condições materiais de uso: inferência mais barata, adaptação mais simples e integração mais direta com RAG, ferramentas e agentes.

Um ecossistema que deixou de ter um único centro

Em 2023, o campo começou a deixar de ser dominado exclusivamente por alguns poucos laboratórios. O LLaMA, lançado em fevereiro, abriu espaço para pesquisa com pesos disponíveis; o LLaMA 2, em julho, ampliou a possibilidade de adoção; Mistral e BLOOM ofereceram alternativas; e LoRA reduziu drasticamente o custo de adaptação. Em paralelo, o GPT-4 estabeleceu uma referência para os modelos fechados. A mudança, portanto, não foi apenas quantitativa — mais modelos, mais parâmetros ou mais capacidade. Ela alterou a estrutura do ecossistema.

Antes, a fronteira do modelo estava essencialmente dentro do laboratório que o treinava. Com pesos disponíveis, pesquisadores e engenheiros podiam partir de um modelo já treinado e modificar seu comportamento para uma finalidade específica. Isso criou uma nova dinâmica:



O modelo deixou de ser apenas o produto final. Passou a ser também uma base sobre a qual outros sistemas poderiam ser construídos. Essa mudança é importante porque desloca parte da inovação do pré-treino para o pós-treino, para a adaptação e para a orquestração. Em termos práticos, o valor deixou de residir apenas no modelo isolado e passou a residir também no que se consegue montar ao redor dele.

LLaMA: quando os pesos chegam à comunidade — e a arquitetura favorece o uso

O LLaMA, de Touvron et al. (2023), foi um ponto de inflexão porque combinou duas coisas ao mesmo tempo: qualidade elevada para o tamanho do modelo e disponibilidade de pesos para pesquisa. O efeito foi multiplicador. Com os pesos disponíveis, a comunidade pôde experimentar diretamente sobre o modelo: fazer fine-tuning, investigar alinhamento, testar aplicações e combiná-lo com outros componentes. O ponto importante não é apenas que pesquisadores passaram a ter acesso a um modelo. É que passaram a ter acesso à matéria-prima de novos modelos e sistemas. Isso altera a velocidade da experimentação. Em um modelo fechado, o usuário pode testar diferentes prompts ou utilizar as interfaces disponibilizadas pelo fornecedor. Com os pesos, o espaço de experimentação aumenta: o pesquisador pode modificar o próprio comportamento do modelo. Essa diferença explica parte da multiplicação de modelos derivados que marcou o período.

Do ponto de vista arquitetural, o LLaMA também ajudou a consolidar um conjunto de escolhas que favoreciam treinamento estável e inferência mais prática. A família original era composta por modelos decoder-only de 7B, 13B, 33B e 65B parâmetros, treinados com grandes volumes de dados — chegando a cerca de 1,4 trilhão de tokens nas versões maiores. A arquitetura usava normalização RMSNorm antes das camadas, ativação SwiGLU e embeddings posicionais rotacionais (RoPE), além de operar sem bias em várias camadas. Essas escolhas não são apenas detalhes de implementação: elas melhoram a estabilidade do treinamento e contribuem para que modelos menores alcancem qualidade competitiva.

Para inferência, isso teve consequência direta. Um modelo de 13B é muito mais fácil de servir do que um modelo de centenas de bilhões de parâmetros. Ele consome menos memória, admite quantização mais agressiva e pode rodar em infraestruturas menores. A abertura dos pesos acelerou esse processo: a comunidade desenvolveu runtimes, formatos compactos e ferramentas de serviço que tornaram viável executar versões do LLaMA fora de grandes clusters. Assim, o LLaMA não apenas democratizou o acesso aos pesos; também tornou mais realista a ideia de modelos abertos como componentes de sistemas locais ou privados.

LLaMA 2/3: da pesquisa à adoção

O movimento avançou com LLaMA 2 e, posteriormente, LLaMA 3. O material deste capítulo destaca duas mudanças centrais: uma licença mais favorável à adoção e a existência de versões instruction-tuned. Isso favoreceu a passagem do modelo como objeto de pesquisa para o modelo como componente de produtos. A distinção é importante. Um modelo que serve apenas como objeto de pesquisa tem um papel diferente de um modelo que pode ser incorporado a sistemas reais. O instruction tuning também reforça a continuidade com o capítulo anterior: o modelo aberto não precisava permanecer apenas como um modelo pré-treinado. Ele podia receber a mesma camada de adaptação comportamental que havia transformado os modelos fechados em sistemas mais utilizáveis. A abertura dos pesos, portanto, não significava voltar ao modelo bruto; significava permitir que a comunidade construísse suas próprias versões e adaptações.

No plano arquitetural, essa transição também veio acompanhada de ganhos concretos. O LLaMA 2 ampliou a janela de contexto para 4096 tokens e foi treinado em um volume maior de dados. Nas versões maiores, adotou grouped-query attention (GQA), uma variante de atenção que reduz o tamanho do cache de chaves e valores durante a geração. Isso tem efeito direto em inferência: menos memória por sequência, maior throughput e melhor escalabilidade quando muitas requisições são atendidas simultaneamente. Para sistemas de produção, esse tipo de escolha importa tanto quanto a qualidade bruta do modelo, porque define o custo operacional de servir respostas longas ou concorrentes.

O LLaMA 3, lançado em seguida, aprofundou essa trajetória. As primeiras versões de 8B e 70B trouxeram janela de contexto maior, vocabulário ampliado, treinamento em volume de dados ainda mais alto e melhor qualidade em tarefas instruídas. A família consolidou o uso de RMSNorm, SwiGLU e RoPE, além de variantes de atenção mais eficientes em várias configurações. O efeito combinado foi aproximar modelos abertos de requisitos reais de produto: mais contexto para RAG, mais estabilidade para chat, menor custo de inferência por qualidade entregue e maior facilidade de adaptação. Em termos práticos, o modelo aberto deixou de ser apenas uma alternativa de pesquisa e passou a ser uma base plausível para sistemas em produção.

Mistral e BLOOM: alternativas ao caminho dominante

LLaMA não foi a única alternativa. Mistral e BLOOM ampliaram o espaço de modelos abertos e contribuíram para a diversidade do ecossistema. Essa diversidade importa porque reduz a dependência

de uma única família de modelos. Diferentes arquiteturas e diferentes escolhas de projeto podem ser comparadas, adaptadas e incorporadas a sistemas distintos. O ponto estrutural é simples: a abertura cria um espaço de experimentação entre diferentes modelos, e não apenas uma alternativa binária entre “usar o modelo dominante” e “treinar um modelo do zero”.

O Mistral 7B é um exemplo especialmente relevante dessa fase. Ele combinou escolhas arquiteturais voltadas a eficiência e contexto: atenção com janela deslizante, cache rolante, grouped-query attention, RMSNorm, SwiGLU e embeddings rotacionais. A janela deslizante permite controlar o custo de atenção em sequências longas, evitando que o crescimento do contexto produza aumento excessivo de memória e computação. Para inferência, isso significa que o modelo pode lidar com mais texto sem que o custo cresça da mesma forma que cresceriam abordagens mais simples. Na prática, essa escolha favorece sistemas que precisam de contextos longos, como RAG, análise de documentos e agentes que acumulam histórico.

A contribuição da Mistral, porém, não se limitou a modelos densos compactos. A família Mixtral popularizou o uso de mixture-of-experts (MoE) em modelos abertos. Em vez de ativar todos os parâmetros para cada token, o modelo possui vários especialistas e roteia apenas um subconjunto deles para cada passo de geração. Isso cria uma assimetria útil: o modelo pode ter muitos parâmetros totais, mas usa menos parâmetros ativos por token. O resultado é uma relação melhor entre qualidade e custo computacional. Em inferência, isso pode aumentar o throughput e reduzir o custo por resposta, embora introduza novos desafios de memória e complexidade de serving. Para engenharia, o MoE representa uma troca relevante: menos FLOPs ativos, mais memória total, maior exigência de infraestrutura e maior flexibilidade para escalar qualidade sem escalar proporcionalmente o custo de geração.

O BLOOM, embora lançado em 2022, integrou esse mesmo movimento de abertura em grande escala. Ele se destacou menos por inovações de eficiência de inferência e mais por demonstrar que um modelo de grande porte poderia ser desenvolvido de forma aberta, multilíngue e com governança distribuída. Do ponto de vista arquitetural, o BLOOM é um decoder-only de grande escala, com embeddings ALiBi, que favorecem generalização de comprimento sem depender exclusivamente de posições absolutas aprendidas. Sua importância histórica está mais na prova de que modelos grandes podem ser abertos, auditáveis e multilíngues do que em economia de inferência. Ainda assim, ele ajudou a ampliar o espaço de comparação arquitetural e a mostrar que não havia um único caminho para modelos abertos.

Síntese das mudanças arquiteturais e de inferência

Para visualizar o que essas famílias acrescentaram ao ecossistema, vale reunir as principais escolhas arquiteturais e seus efeitos práticos:

Família / técnica	Escolhas arquiteturais relevantes	Efeito em desempenho e inferência
LLaMA	RMSNorm, SwiGLU, RoPE, modelos de 7B a 65B	Treinamento mais estável, modelos menores competitivos, inferência local mais viável
LLaMA 2 / LLaMA 3	Mais dados, contexto maior, GQA, instruction tuning	Menor custo de KV cache, maior throughput, melhor adequação a produto
Mistral 7B	Sliding window attention, cache rolante, GQA	Contexto longo com custo controlado, inferência mais eficiente
Mixtral / MoE	Especialistas esparsos, roteamento top-k	Menos parâmetros ativos por token, maior throughput, maior exigência de memória
BLOOM	Modelo aberto multilíngue, ALiBi	Generalização de comprimento, abertura em grande escala, menor foco em eficiência de inferência
LoRA	Adaptadores de baixo posto sobre modelo-base	Adaptação barata, múltiplas especializações, serving multi-inquilino

Essa tabela resume uma mudança importante: a corrida de 2023 não foi apenas sobre quem tinha o modelo mais capaz. Foi também sobre qual arquitetura tornava o modelo utilizável, adaptável e servível em condições reais. Desempenho deixou de ser apenas uma métrica de benchmark e passou a incluir latência, throughput, memória, custo de contexto e facilidade de adaptação.

LoRA: adaptar sem recomeçar

A abertura dos pesos resolve um problema: permite modificar o modelo. Mas surge imediatamente outro: quanto custa modificar um modelo gigantesco? LoRA, de Hu et al. (2021), oferece uma resposta

importante. Em vez de atualizar todos os parâmetros de um modelo durante o fine-tuning, a técnica adapta o modelo ajustando uma matriz de baixo posto. O resultado é uma redução drástica do custo de adaptação. A ideia pode ser representada assim:

```

modelo-base
├── adaptador A → tarefa / cliente A
├── adaptador B → tarefa / cliente B
└── adaptador C → tarefa / cliente C
  
```

O modelo-base permanece o mesmo. O que muda é o adaptador. Isso permite pensar em um único modelo grande como uma base para múltiplas especializações. Segundo o material, um adaptador pode representar aproximadamente 1-2% dos parâmetros. O impacto econômico dessa diferença é significativo: em vez de retrainar o modelo inteiro para cada aplicação, pode-se manter o modelo-base e treinar adaptações menores. LoRA se torna, assim, uma peça importante da multiplicação de modelos derivados.

Para inferência, LoRA também introduz uma vantagem arquitetural relevante. Como o modelo-base permanece congelado, é possível servir múltiplos adaptadores sobre a mesma infraestrutura, trocando adaptadores por cliente, tarefa ou domínio sem duplicar o modelo completo. Em alguns cenários, o adaptador pode ser mesclado aos pesos do modelo para reduzir overhead; em outros, pode permanecer separado para permitir troca dinâmica. Isso favorece sistemas multi-inquilino, experimentação rápida e especialização por domínio sem custo proibitivo. Em termos práticos, LoRA transforma adaptação de modelo de um processo de re treinamento pesado em um processo modular de engenharia.

O contraponto: GPT-4

Enquanto o ecossistema aberto se expandia, o lado fechado também avançava. O GPT-4, lançado em março de 2023, representou a fronteira dos modelos fechados em escala, alinhamento e multimodalidade. Isso é importante porque impede uma interpretação simplista da corrida de 2023. A abertura dos pesos não significava automaticamente superioridade de capacidade. Havia duas vantagens diferentes sendo perseguidas:

```

MODELOS ABERTOS
experimentação
adaptação
auditoria
orquestração local

MODELOS FECHADOS
capacidade de ponta
alinhamento
multimodalidade
produto e robustez
  
```

A corrida passou a acontecer nos dois espaços simultaneamente. De um lado, a comunidade experimentava com modelos que podia modificar. Do outro, laboratórios fechados continuavam concentrando recursos para ampliar a capacidade dos modelos e transformá-la em produtos. Essa divisão seria decisiva para a arquitetura dos sistemas seguintes. Para o engenheiro, isso significa que a escolha entre aberto e fechado não deve ser tratada como preferência ideológica, mas como decisão de requisitos: controle, custo, privacidade, capacidade, latência e governança.

A segunda metade da história: quando o modelo se torna plataforma

O capítulo anterior terminou com uma tese: o valor do sistema está no que vem depois do modelo. Em 2023, essa tese ganhou uma segunda dimensão. Não bastava mais perguntar o que havia sido colocado dentro do modelo. Era preciso perguntar o que outras pessoas poderiam construir em cima dele. O modelo aberto introduziu uma nova camada:

```

pré-treino
┆
┆ modelo-base
┆
┆ pesos disponíveis
┆
┆ adaptação
┆
┆ sistema especializado
  
```

Essa estrutura muda a economia do desenvolvimento. O laboratório que treinou o modelo continua sendo importante, porque produziu a capacidade inicial. Mas a inovação não precisa terminar ali. Outros pesquisadores podem adaptar o modelo. Outros engenheiros podem incorporá-lo a produtos. Outros projetos podem combiná-lo com recuperação de informação, ferramentas e agentes. O modelo passa a funcionar como uma infraestrutura sobre a qual uma comunidade pode construir. É essa multiplicação que dá aos pesos abertos seu significado histórico.

Do ponto de vista de inferência, essa mudança também é relevante. Quando o modelo se torna plataforma, o sistema precisa ser pensado como um todo: contexto, adaptadores, cache, batching, quantização, latência e custo por requisição. A arquitetura do modelo deixa de ser um assunto interno do laboratório e vira variável de projeto do próprio produto. Escolhas como GQA, MoE, contexto maior e adaptadores leves passam a influenciar diretamente o desenho de sistemas RAG, agentes e aplicações empresariais.

A relação com RAG, ferramentas e agentes

A bifurcação entre modelos abertos e fechados também ajuda a explicar o ambiente em que RAG, ferramentas e agentes se desenvolveram. Um modelo sozinho possui limitações que o capítulo anterior já estabeleceu: conhecimento datado, ausência de fontes e incapacidade de realizar determinadas formas de computação. O caminho natural foi acrescentar componentes ao redor dele. RAG acrescenta recuperação de informação. Ferramentas acrescentam capacidades externas. Agentes acrescentam a possibilidade de organizar chamadas ao modelo e ações em uma sequência.

Os pesos abertos tornaram esse tipo de experimentação especialmente acessível porque o engenheiro podia controlar mais diretamente o modelo que estava no centro do sistema. Isso não significa que RAG, ferramentas ou agentes dependam de modelos abertos. Não dependem. Significa que o ecossistema aberto criou um ambiente particularmente favorável para experimentar como combinar um modelo com outros componentes. Essa é a continuidade direta com a tese do livro:

```

modelo
↓
alinhamento
↓
produto
↓
limitações
↓
ferramentas / RAG / agentes

```

Os modelos abertos acrescentam uma nova possibilidade:

```

modelo
↓
pesos disponíveis
↓
adaptação
↓
sistema especializado

```

As duas linhas passam a convergir. E as mudanças arquiteturais dessa fase ajudam a explicar por quê. Janelas de contexto maiores permitem injetar mais evidência em RAG. Atenção mais eficiente reduz o custo de manter histórico longo em agentes. MoE melhora a relação entre qualidade e custo em sistemas com muitas chamadas. LoRA permite especializar o modelo para tool use, domínio corporativo ou estilo de resposta sem retreinamento completo. Assim, a arquitetura dos modelos abertos não apenas acompanhou o crescimento de RAG e agentes; ela ajudou a torná-los viáveis em escala prática.

O que significa “aberto”?

É importante não transformar “modelo aberto” em sinônimo de “modelo sem restrições”. O ponto central deste capítulo é mais específico: a disponibilidade dos pesos muda o que pode ser feito com o modelo. Com os pesos disponíveis, torna-se possível executar o modelo em infraestrutura própria, adaptá-lo e integrá-lo diretamente a outros componentes. Isso cria vantagens de controle e auditabilidade. Mas o próprio material também mostra que o ecossistema de 2023 era bifurcado. Modelos fechados continuavam oferecendo capacidades importantes que não estavam necessariamente disponíveis da mesma maneira nos modelos abertos. Portanto, a decisão não deveria ser tratada como uma questão ideológica. É uma questão de requisitos.

Há ainda uma distinção importante entre abrir pesos e abrir todo o stack de treinamento. Código de treino, dados, receitas de alinhamento e infraestrutura podem permanecer fechados mesmo quando os

pesos são liberados. Para o engenheiro, o que importa de imediato é o que a abertura dos pesos permite: inspecionar comportamento, adaptar, servir localmente, quantizar, otimizar inferência e integrar com outros componentes. Essa é a diferença prática central.

O que a abertura muda para o engenheiro

Para o engenheiro, a primeira consequência é que modelo não é mais uma escolha isolada. É preciso avaliar o modelo dentro do sistema em que ele será utilizado. Dados sensíveis, custo de inferência, necessidade de execução local e requisitos de compliance podem favorecer pesos abertos. Capacidade bruta e multimodalidade podem favorecer modelos fechados. O custo de adaptação também entra nessa decisão. LoRA permite adaptar grandes modelos sem retreinar todos os parâmetros, tornando possível manter um modelo-base e criar adaptadores específicos por cliente ou tarefa.

Os pesos abertos também permitem maior controle sobre a infraestrutura. O modelo pode ser servido em hardware próprio e combinado com RAG e ferramentas sem depender integralmente de um provedor externo. Isso traz vantagens, mas também transfere responsabilidades: segurança, escalabilidade, observabilidade, versionamento e otimização de inferência passam a ser parte do problema de engenharia da equipe. Em outras palavras, abertura dá controle, mas também exige maturidade operacional.

Além disso, a arquitetura do modelo passa a influenciar diretamente o custo do sistema. Contexto maior favorece RAG e agentes, mas aumenta consumo de memória. GQA reduz KV cache e melhora throughput, especialmente em cenários de longa duração ou alta concorrência. MoE pode reduzir custo computacional por token, mas exige mais memória e serving mais sofisticado. Quantização pode tornar modelos abertos viáveis em hardware menor, mas pode introduzir perdas de qualidade. Tudo isso significa que escolher um modelo aberto não é apenas escolher um conjunto de pesos; é escolher um perfil de inferência.

A decisão, entretanto, não é definitiva. Em 2023, a diferença entre os dois mundos parecia relativamente clara: aberto significava principalmente experimentação e pesquisa; fechado significava o topo da capacidade e do produto. Essa distância muda. Por isso, uma decisão tomada hoje pode estar errada amanhã. A arquitetura precisa ser reavaliada periodicamente, especialmente porque custo, capacidade e disponibilidade evoluem rapidamente.

A lição estrutural

O movimento dos modelos abertos representa uma mudança diferente daquela produzida pelo RLHF. O RLHF mostrou que capacidade não era suficiente: era preciso transformar capacidade em comportamento utilizável. Os modelos abertos mostraram outra coisa: mesmo um modelo já treinado pode se tornar matéria-prima para novos sistemas. A sequência fica mais clara:

```
pré-treino
  ↓
capacidade
  ↓
RLHF / alinhamento
  ↓
obediência
  ↓
produto
  ↓
acesso
  ↓
modelos abertos
  ↓
distribuição da capacidade
  ↓
adaptação + ferramentas + RAG
  ↓
sistemas especializados
```

O capítulo anterior mostrou o momento em que o público chegou ao modelo. Este capítulo mostra o momento em que a capacidade começa a circular fora do laboratório. Essa é a importância de 2023. Não foi o fim dos modelos fechados. O GPT-4 mostrou justamente o contrário: a fronteira de capacidade continuava sendo empurrada por laboratórios com enormes recursos. Mas, ao mesmo tempo, os pesos de alguns modelos passaram a circular, LoRA tornou a adaptação barata e uma comunidade inteira passou a experimentar com aquilo que antes estava confinado. A corrida deixou de ser apenas: quem consegue treinar o melhor modelo? E passou a incluir uma segunda pergunta: o que milhares de pesquisadores e engenheiros conseguem construir quando recebem um modelo já treinado? É essa segunda pergunta que prepara o terreno para a próxima etapa do livro.

Para o engenheiro

Escolha aberto versus fechado por requisito, não por preferência. Dados sensíveis, custo de inferência, execução local e compliance podem favorecer pesos abertos; capacidade bruta e multimodalidade podem favorecer modelos fechados. A decisão correta depende do sistema, não apenas do modelo.

Separe modelo-base de adaptação. LoRA permite manter um modelo grande como base e criar adaptadores específicos para clientes ou tarefas, sem retrainar o modelo inteiro. Isso reduz custo, acelera experimentação e facilita servir múltiplas variantes sobre a mesma infraestrutura.

Pesos abertos significam controle, mas também responsabilidade. É possível executar o modelo em hardware próprio, adaptá-lo e combiná-lo com RAG e ferramentas sem depender integralmente de um provedor. Em troca, a equipe assume parte do trabalho de segurança, escalabilidade e otimização.

Não confunda pesos abertos com capacidade superior. Em 2023, o ecossistema era claramente bifurcado: os modelos abertos favoreciam experimentação; os fechados concentravam parte importante da capacidade de ponta e da robustez de produto. Essa diferença pode diminuir com o tempo, mas não deve ser ignorada no projeto.

Considere arquitetura como variável de inferência. Contexto maior favorece RAG e agentes. GQA reduz custo de memória e melhora throughput. MoE pode melhorar eficiência computacional, mas aumenta complexidade de serving. Quantização pode reduzir custo, mas exige avaliação de qualidade. Essas escolhas afetam diretamente latência, custo e viabilidade operacional.

O modelo é apenas um componente. A arquitetura precisa considerar o que acontece depois dele: adaptação, recuperação de informação, ferramentas e agentes. Em sistemas reais, o desempenho final depende menos do modelo isolado e mais da qualidade da integração entre modelo, contexto e infraestrutura.

Reavalie a escolha. A relação entre capacidade, custo e disponibilidade muda rapidamente. Uma arquitetura adequada hoje pode deixar de ser a melhor escolha depois. Em um ecossistema que evolui nessa velocidade, revisar periodicamente a decisão faz parte do trabalho de engenharia.

Fontes: [Touvron et al., 2023] — LLaMA; [Touvron et al., 2023] — LLaMA 2; [Hu et al., 2021] — LoRA; [OpenAI, 2023] — GPT-4 Technical Report; [Zhao et al., 2023] — survey de LLMs; [Jiang et al., 2023] — Mistral 7B; [Jiang et al., 2024] — Mixtral; [Le Scao et al., 2022] — BLOOM; [Ainslie et al., 2023] — Grouped-Query Attention.

Parte IV — Era das ferramentas (2021-2023)

Capítulo 9 — Do WebGPT ao Toolformer: como o modelo ganhou mãos

Até aqui, este livro tratou de um modelo que fala. O pré-treinamento lhe deu uma língua, o alinhamento lhe ensinou a obedecer e o produto o colocou diante de milhões de pessoas. Mas restava uma limitação que nenhuma dessas etapas resolvia: o modelo não podia fazer nada além de gerar texto. Não buscava a notícia de hoje, não resolvia 17×23 sem errar, não executava código, não consultava um banco de dados. Seu conhecimento estava congelado no momento em que o treinamento terminou. Para responder a uma pergunta sobre o presente, ele adivinhava a partir do passado. Para calcular, aproximava. Para executar, inventava um resultado em vez de produzir um efeito.

É essa a parte mais literal da tese deste livro. Se a inteligência efetiva do sistema emerge da combinação entre modelo, contexto, ferramentas e iteração, então era preciso ensinar o modelo a ter mãos. E havia dois caminhos possíveis. O primeiro era dizer ao modelo como usar uma ferramenta: descrever a API no prompt e esperar que a geração produzisse a chamada certa. O segundo era treinar o modelo a decidir quando uma ferramenta valia a pena, transformando o uso de ferramentas em comportamento aprendido, e não em truque de engenharia de prompt. Os quatro marcos desta era mostram os dois caminhos em ação. O WebGPT provou que um modelo podia agir em um ambiente. O MRKL ousou rotear partes da pergunta para calculadoras, conversores e bancos de dados. O PAL decidiu que o modelo nem deveria tentar calcular — melhor escrever Python e deixar a máquina executar. E o Toolformer aprendeu sozinho quais chamadas de API valiam a pena, sem que ninguém lhe dissesse. No fim do capítulo, o HuggingGPT transforma o próprio modelo em um controlador que escolhe outros modelos como ferramentas.

A ambição de toda a era cabe em uma frase: transformar **ferramentas em extensões cognitivas** do modelo. Quando isso acontece, o sistema resultante passa a ser mais capaz do que o modelo sozinho — não porque o modelo mudou, mas porque o processo do qual ele participa passou a incluir o mundo.

WebGPT: o primeiro modelo que agiu em um ambiente

O WebGPT, de Nakano et al. (2021), foi o primeiro marco a mostrar que um modelo de linguagem podia não apenas gerar texto, mas agir dentro de um ambiente e colher o resultado dessa ação. A formulação era simples e a execução, difícil: dar ao GPT-3 um navegador de texto e treiná-lo a usá-lo para responder perguntas. O navegador não era metáfora. O modelo realizava ações discretas — buscar, abrir um resultado, rolar a página, citar um trecho — e observava o estado resultante antes de decidir o próximo passo.

```
estado do navegador
↓
ação do modelo (buscar, abrir, rolar, citar)
↓
novo estado do navegador
↓
observação → próxima ação
↓
resposta final, apoiada em referências
```

Essa mudança é estrutural. A resposta deixava de ser produzida de uma única vez, a partir da memória interna do modelo; passava a ser construída passo a passo, à medida que as evidências eram coletadas. E havia uma regra decisiva: para responder, o modelo precisava apoiar a resposta em referências. Em vez de apenas afirmar, ele precisava demonstrar de onde vinha a informação. Exigir citação não é um detalhe de produto; é um mecanismo de verificação que desloca parte da responsabilidade do modelo para o ambiente.

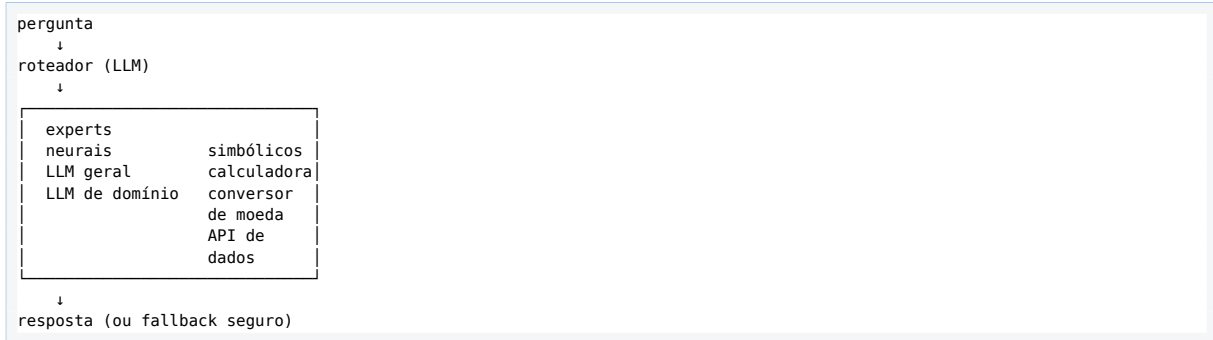
O treinamento acontecia em dois estágios. No primeiro, o modelo aprendia por imitação, reproduzindo trajetórias de humanos usando o navegador. No segundo, um modelo de recompensa treinado a partir de preferências humanas selecionava, por rejeição amostral, as melhores trajetórias entre várias amostradas. Essa sequência — aprender com humanos e depois refinar com feedback — é exatamente a lógica que o capítulo 7 apresentou para o alinhamento, agora aplicada não à resposta final, mas à

sequência de ações que produz a resposta. A avaliação em ELI5, um conjunto de perguntas abertas, indicou que os avaliadores preferiam as respostas do WebGPT às referências originais do Reddit na maioria das comparações, e que as respostas eram consideravelmente mais factuais e informativas.

A importância histórica do WebGPT não está nos números exatos. Está em demonstrar três coisas ao mesmo tempo: que um LLM podia interagir com um ambiente e usar a observação; que o uso de ferramentas podia ser aprendido por treinamento com feedback; e que exigir referências muda o comportamento do modelo. Agir, aprender com feedback e fundamentar são precisamente os pilares que o ReAct e o function calling transformariam em padrão nos anos seguintes.

MRKL: roteamento neuro-simbólico

O MRKL, de Karpas et al. (2022), atacou o problema por outro ângulo. Em vez de fazer o modelo agir em um ambiente, dividiu o próprio conhecimento entre módulos especialistas e deixou o LLM decidir qual módulo deveria responder a cada parte da pergunta. A arquitetura — *Modular Reasoning, Knowledge and Language* — combinava componentes neurais e simbólicos.

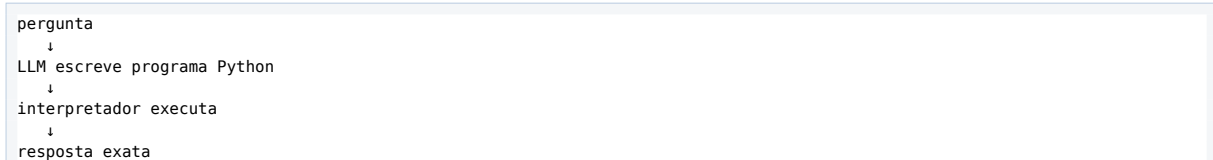


A tese do MRKL é que nem todo raciocínio deve usar a mesma máquina. Tarefas que exigem compreensão de linguagem podem ser resolvidas por módulos neurais, como um LLM geral ou especializado por domínio. Tarefas que exigem computação exata ou dados estruturados — calcular, converter, consultar — devem ser resolvidas por módulos simbólicos, que não alucinam. O LLM, no papel de roteador, decide para qual especialista cada fragmento da consulta deve ir.

Dois detalhes de engenharia tornam a ideia robusta. O primeiro é o fallback seguro: quando o roteador não tem confiança suficiente, o sistema recorre ao LLM geral em vez de arriscar um especialista. O segundo é que os experts podem ser, eles próprios, LLMs — o roteamento não precisa escolher apenas entre símbolo e linguagem; pode escolher entre modelos de capacidades diferentes. A implementação mais conhecida da arquitetura foi o Jurassic-X, da AI21, o que mostrou que o padrão podia deixar o papel e virar produto. O MRKL não resolveu todos os problemas do tool use, mas nomeou a estrutura que praticamente todos os sistemas seguintes herdariam: um modelo que decide e módulos que executam.

PAL: raciocínio via código

Se o MRKL roteava o cálculo para uma calculadora, o PAL, de Gao et al. (2022), foi mais radical. Em vez de o modelo calcular, o modelo deveria escrever um programa Python e deixar um interpretador resolver. O nome condensa a tese — *Program-Aided Language Models*.



A justificativa é epistêmica: um LLM não computa, ele imita a computação. Pode reproduzir o estilo de um cálculo e errar no resultado; um interpretador, não. Ao deslocar o passo de solução para um programa, o modelo entrega aquilo que faz bem — decompor o problema, escolher as operações e organizar a lógica em linguagem natural e código — e delega à máquina aquilo que ela faz com exatidão: executar. Essa separação é o que este livro chama de distinguir o que o modelo sabe do que ele pode calcular.

Os resultados sustentaram a aposta. Na GSM8K, um benchmark de problemas de matemática, a abordagem alcançou estado da arte e superou um modelo de 540 bilhões de parâmetros usando *chain-of-thought* por 15 pontos percentuais absolutos de acurácia top-1. O padrão generalizou para 13 tarefas do BIG-Bench Hard, e os autores relataram resolver significativamente mais problemas de matemática

com menos erros. O ponto não é apenas que o código acerta a aritmética; é que o LLM, ao escrever o programa, revela seu raciocínio em uma forma que pode ser executada, testada e depurada — uma propriedade que nenhuma verificação sobre o texto solto possui.

Toolformer: o modelo que aprende sozinho quando chamar

Os três marcos anteriores tinham uma característica em comum: era o projetista quem decidia onde e quando a ferramenta entraria. O Toolformer, de Schick et al. (2023), inverteu essa responsabilidade. Em vez de receber instruções, o modelo aprendeu, por auto-supervisão, a decidir quando uma chamada de API valia a pena.

A receita é elegante. O modelo percorre seu próprio corpus de treinamento e, em cada trecho, gera chamadas de API candidatas — pedaços de texto como *search*, *calculator*, *QA*, *translation* ou *calendar*, posicionados no ponto em que a informação externa pareceria útil. As APIs são executadas de verdade. Então o modelo pergunta, com um critério puramente estatístico: a chamada reduziu o erro de previsão dos tokens seguintes? Apenas as chamadas que sobreviveram a esse crivo são mantidas; o resto é descartado. Por fim, o modelo é fine-tunado sobre o corpus anotado.

```

corpus
↓
chamadas candidatas (busca, calculadora, QA, tradutor, calendário)
↓
execução das APIs
↓
mantém apenas chamadas que reduzem o erro de previsão
↓
fine-tuning sobre o corpus anotado

```

Ferramenta	Tipo de informação que acrescenta
Calculadora	computação exata que o modelo aproxima
QA	fatos que o modelo não reteve
Busca	informação atualizada ou fora do treino
Tradutor	texto em idiomas que o modelo domina pouco
Calendário	datas e programação

O resultado é um comportamento que não foi instruído por nenhum humano: o modelo descobre, por dados, que alguns problemas são melhor resolvidos chamando uma ferramenta, e passa a fazê-lo espontaneamente. As melhorias apareceram em tarefas zero-shot, e modelos menores com o Toolformer se tornaram competitivos com modelos muito maiores sem a capacidade. A lição transcende o experimento: treinar para decidir *quando* usar uma ferramenta é qualitativamente diferente de instruir *como* usá-la.

Dois caminhos: dizer como ou aprender quando

Vistos em sequência, os quatro marcos desenham uma clivagem que atravessa toda a era das ferramentas. De um lado, o caminho do prompting: descrever a ferramenta no texto e confiar que o modelo a invoque corretamente. Do outro, o caminho do treinamento: ensinar ao modelo, por feedback ou auto-supervisão, a escolher e usar ferramentas como parte do comportamento. O MRKL e o PAL ainda dependem, em boa parte, do primeiro; o WebGPT e o Toolformer são exemplos puros do segundo.

CAMINHO DO PROMPTING	CAMINHO DO TREINAMENTO
como usar a ferramenta	quando usar a ferramenta
descrita no prompt	aprendido por feedback/dados
frágil, depende do formato	robusto, vira comportamento
custo zero de treino	exige dados ou recompensa

Cada caminho tem seu lugar. O prompting é barato, imediato e suficiente quando o uso é raro ou simples. O treinamento é caro, mas converte o tool use em algo que não depende de o modelo “se lembrar” de chamar a ferramenta a cada vez. A história que este capítulo conta — e que o capítulo 11 completará — é a de um deslocamento progressivo do primeiro para o segundo: primeiro produzimos o tool use por prompt, depois o tornamos primitiva de API e, finalmente, garantia formal de formato. A tabela abaixo resume os quatro padrões:

Sistema	Mecanismo	Pergunta que responde
WebGPT	agente em um ambiente + feedback	como agir e colher observação?
MRKL	roteador + experts neurais/simbólicos	qual parte vai para qual ferramenta?
PAL	LLM escreve código, interpretador executa	como obter computação exata?
Toolformer	auto-supervisão sobre chamadas	quando a chamada vale a pena?

HuggingGPT: o LLM como controlador

O último marco da era amplia o significado de “ferramenta”. No HuggingGPT, de Shen et al. (2023), o LLM não chama APIs de cálculo ou busca: ele escolhe outros modelos como ferramentas. O ChatGPT atua como controlador de um ecossistema inteiro — o Hugging Face — e orquestra a execução em quatro etapas.

```

tarefa do usuário
↓
planejamento (ChatGPT)
↓
seleção de modelos no Hugging Face
↓
execução dos modelos escolhidos
↓
sumarização da resposta

```

A inovação conceitual é dupla. Primeiro, o modelo deixa de ser a única inteligência do sistema e passa a ser o cérebro que contrata cérebros: para cada sub tarefa, ele seleciona o especialista mais adequado — um modelo para visão, outro para áudio, outro para linguagem. Segundo, a linguagem deixa de ser apenas o meio da resposta e se torna a interface de controle: é por texto que o controlador descreve o plano, chama os especialistas e sintetiza o resultado. O que antes era um único modelo respondendo torna-se uma organização temporária de modelos coordenada por linguagem.

O HuggingGPT antecipa, na escala das ferramentas, o que os capítulos 13 e 14 tratarão nos agentes: a ideia de que o valor está menos no modelo isolado e mais no processo de orquestração. Quando ferramentas passam a incluir outros modelos, o limite do sistema deixa de ser o modelo e passa a ser a capacidade de decidir o que usar, quando e para quê.

A lição estrutural

O fio que atravessa os cinco marcos é a resposta que o livro vem construindo desde a primeira parte: a capacidade não reside apenas no modelo, mas no processo em torno dele. O WebGPT mostrou que o modelo pode agir; o MRKL, que pode delegar; o PAL, que pode evitar computar; o Toolformer, que pode decidir quando chamar; e o HuggingGPT, que pode comandar outros modelos. Em todos os casos, o modelo não ficou mais inteligente no sentido de pré-treinamento. O sistema ao redor dele ficou mais capaz, porque passou a incluir ação, observação e computação externa.

Isso tem uma consequência direta para a engenharia: a pergunta deixou de ser apenas “o que o modelo consegue fazer?” e passou a ser “que processo — com quais ferramentas, acionadas por qual lógica — produz o resultado de forma confiável?”. Os marcos desta era dão, cada um, uma peça dessa resposta. Falta, porém, a peça que transforma o uso de ferramentas de evento isolado em comportamento contínuo: o loop. É dela que trata o próximo capítulo.

Para o engenheiro

Se a resposta exige fato novo, cálculo ou ação, não deixe o modelo adivinhar: dê a ferramenta. A regra de ouro desta era é simples — se um humano resolveria o problema com uma calculadora, uma busca ou uma API, o sistema deve ter a mesma opção. Modelo que adivinha um fato é uma fonte de erro evitável; modelo que busca, calcula ou executa é uma fonte de evidência.

Os quatro padrões do capítulo são receitas diretamente reaproveitáveis. O WebGPT ensina a aprender por feedback quando a tarefa envolve sequências de ações. O MRKL ensina a rotear cada parte da pergunta para a ferramenta certa e a prever um fallback seguro. O PAL ensina a delegar computação ao código — e é, até hoje, a forma mais barata de obter matemática confiável. O Toolformer ensina a deixar o modelo descobrir quando chamar, o molde de referência para sistemas que precisam acionar APIs espontaneamente. Em 2026, o function calling nativo cobre boa parte desses casos, mas o raciocínio por trás de cada padrão permanece.

Ferramenta não é “mais uma função” no sistema; é uma fonte de evidência. Meça o ganho real de cada uma. Se adicionar uma ferramenta não muda a resposta certa, ela está apenas adicionando latência e superfície de erro. A disciplina de ferramentas é a mesma da memória: só o que produz evidência útil deve permanecer.

Finalmente, separe o que o modelo sabe do que ele pode calcular. Sempre que a operação for executável — aritmética, consulta, código — prefira executar a inferir. O custo é pequeno; o ganho de confiabilidade é a própria justificativa do capítulo.

Fontes: [Nakano et al., 2021] — WebGPT; [Karpas et al., 2022] — MRKL; [Gao et al., 2022] — PAL; [Schick et al., 2023] — Toolformer; [Shen et al., 2023] — HuggingGPT; [survey de tool use, 2026] — evolução do tool use em agentes; [Taskade, 2026] — histórico do tool use.

Capítulo 10 — ReAct e o loop razão-ação (2022)

O capítulo anterior terminou com o modelo ganhando mãos. Mas havia uma diferença entre ter uma mão e saber usá-la. Em todos os marcos de 2021 a 2023, chamar uma ferramenta era um ato único: o modelo pede, o sistema executa, a observação volta, o modelo responde, fim. Se o primeiro passo estivesse errado — a busca com o termo errado, o cálculo com a fórmula errada — não havia como o modelo perceber e corrigir o próprio caminho. Em outubro de 2022, o ReAct, de Yao et al., atacou exatamente essa lacuna. Ele não acrescentou uma ferramenta nova. Acrescentou a *forma*: pense, aja, observe, pense de novo, até que o objetivo esteja satisfeito.

O problema que ele resolve é o mesmo de qualquer pessoa que erra o caminho: sem observar o resultado da própria ação, não há correção de rota. Uma pessoa que se perde olha em volta e se reorienta; um modelo que gera uma resposta errada, sem acesso ao mundo, não tem para onde olhar. O ReAct deu ao modelo esse “olhar em volta”: o resultado de cada ação volta como observação, e a observação alimenta a próxima decisão. Com isso, um modelo pode agir, errar, perceber que errou e tentar de novo — exatamente o tipo de iteração que o prefácio deste livro define como a diferença entre um sistema confiável e uma geração sortuda.

A ambição era um loop com condição de parada. E a descoberta prática foi que quase todo framework de agente — AutoGPT, LangChain, LangGraph, o que vier no capítulo 13 — é, no fundo, “ReAct + infraestrutura”. Por isso o capítulo trata o ReAct menos como um algoritmo específico e mais como o esqueleto cognitivo da era dos agentes. Ele também faz um aviso que reaparecerá no livro: reflexão do próprio modelo é útil, mas não substitui validadores externos. E mostra o preço do loop: trajetórias lineares acumulam erros, e tokens custam dinheiro.

O problema que a ação única não resolvia

Antes do ReAct, o uso de ferramentas tinha a forma de uma substituição única. O modelo recebia uma pergunta, decidia uma ação, a ferramenta retornava um resultado, e o modelo finalizava. Esse padrão resolve tarefas simples, mas falha de uma maneira característica quando a tarefa exige mais de um passo: um erro no primeiro passo não é corrigível. Se a busca inicial retorna a página errada, o modelo não tem como saber disso. Se o primeiro cálculo usa a unidade errada, o segundo herda o erro. O sistema caminha, mas não sabe que está no caminho errado.

É o que este livro chama de “generate → hope”: gerar uma resposta na esperança de que ela esteja certa, sem nenhum mecanismo para descobrir se esteve. Uma única geração pode falhar; uma sequência com feedback pode convergir. O ReAct é a passagem do primeiro para o segundo: o modelo passa a ter acesso às consequências das próprias ações, e essa informação nova permite corrigir o rumo antes de finalizar.

O padrão

O ReAct intercala raciocínio e ação em uma sequência alternada de três tipos de passo — *Thought*, *Action* e *Observation* — e repete o ciclo até que uma resposta final seja produzida. Um exemplo típico do paper:

```
Thought: "preciso saber quando o Transformer foi publicado"
Action:  search("Attention Is All You Need date")
Observe: "12 de junho de 2017"
Thought: "tenho a resposta"
Answer:  "12 de junho de 2017"
```

A estrutura é deliberadamente simples. O *Thought* dá ao modelo espaço para raciocinar sobre o estado atual e decidir o que precisa saber. O *Action* especifica, em linguagem estruturada, qual ferramenta chamar e com quais argumentos. O *Observe* injeta o resultado real da ferramenta de volta na sequência. E o ciclo se repete até que o modelo julgue ter informação suficiente e emita a resposta final. O detalhe decisivo é que a observação vem do ambiente, não do modelo — é aí que o loop ganha poder.

É importante notar o que o ReAct não é. O modelo não foi treinado para esse comportamento; a forma é evocada com apenas um ou dois exemplos no prompt. O paper mostra, portanto, que a estrutura razão-ação-observação não precisava ser ensinada nos pesos: ela podia ser induzida por poucas demonstrações. Isso explica por que o padrão se espalhou tão rápido — não exigia treinamento, apenas um prompt bem formado e um conjunto de ferramentas.

Raciocínio e ação se alimentam

A tese do ReAct é que raciocinar e agir não são habilidades que se somam; elas se reforçam. O raciocínio melhora a ação porque decide *o que* buscar e *por quê* — em vez de chamar ferramentas às cegas, o modelo raciocina sobre qual informação falta e, portanto, qual ação reduziria a incerteza. A ação melhora o raciocínio porque produz observações do mundo real, que substituem suposições por evidência. No ReAct, cada Thought é informado pela observação anterior, e cada Action é justificada por um Thought anterior.

Esse entrelaçamento ataca diretamente o problema das alucinações. Um modelo que apenas raciocina (*chain-of-thought*) pode afirmar um fato falso com total confiança, porque não tem acesso ao mundo. Um modelo que apenas age pode executar ações sem sentido, porque não tem um raciocínio orientando as decisões. O ReAct combina os dois: o raciocínio gera hipóteses testáveis, e a observação válida ou corrige essas hipóteses antes de elas virarem resposta. Nos benchmarks de question answering com evidência, como HotpotQA e Fever, o ReAct reduziu alucinação e propagação de erro em comparação com linhas de base que usavam apenas raciocínio ou apenas ação.

Os números

Os resultados quantitativos do ReAct foram impressionantes justamente porque comparavam o padrão com métodos muito mais caros. Em ALFWorld, um ambiente doméstico em primeira pessoa onde o agente deve completar tarefas físicas, o ReAct alcançou cerca de 34 pontos percentuais a mais de sucesso absoluto sobre linhas de base de imitação e aprendizado por reforço — métodos que exigiam milhares de demonstrações e treinamento dedicado. Em WebShop, um ambiente de compras online, o ganho foi de cerca de 10 pontos percentuais sobre as mesmas linhas de base.

Tarefa	Ganho do ReAct	Comparação
ALFWorld	~34 pontos absolutos	linhas de base de imitação/RL
WebShop	~10 pontos absolutos	linhas de base de imitação/RL
HotpotQA / Fever	menos alucinação e propagação de erro	CoT e Act-only

O ponto estrutural é o contraste entre esforço e resultado. As linhas de base de imitação e RL eram treinadas especificamente para cada ambiente, com enormes conjuntos de demonstrações. O ReAct usava apenas um ou dois exemplos no prompt e, ainda assim, superava esses métodos. Isso mudou a economia da pesquisa: se uma forma de prompting alcançava o que antes exigia treinamento especializado, a barreira para construir sistemas agentic caía de forma drástica. O padrão se tornou a opção padrão não porque fosse a mais sofisticada, mas porque era a mais barata e a mais geral.

Uma forma, não um framework

Talvez a contribuição mais duradoura do ReAct não seja o algoritmo, mas a percepção de que ele descreve uma *forma*, e não um framework. Todo agente que intercala raciocínio e ação em um loop com observação está, conscientemente ou não, usando o padrão ReAct. Quando o capítulo 13 apresentar os frameworks de agentes, o leitor reconhecerá a mesma estrutura sob nomes diferentes: AutoGPT é ReAct com ferramentas e memória; LangGraph é ReAct com grafos e estados; a maioria dos loops de agente é ReAct com infraestrutura adicional de planejamento, memória e orquestração.

O indício mais concreto dessa universalidade é o parâmetro que quase todo framework expõe: `max_steps`. Ele não é um detalhe de implementação; é a condição de parada do loop. Um loop ReAct precisa saber quando parar — quando o objetivo foi satisfeito, por condição explícita ou por julgamento do modelo — e precisa de um limite de segurança para os casos em que o modelo não converge. `max_steps`, `max_iterations`, `max_loops`: nomes diferentes para a mesma pergunta que o ReAct tornou inevitável. Quem projeta um agente está, na prática, decidindo como parametrizar um ReAct.

Aprender com o feedback do loop

Uma vez que o loop existe, surge naturalmente a pergunta: o que fazer com o que o loop aprendeu? Duas linhagens responderam. A Reflexion, de Shinn et al. (2023), transformou o loop em um ciclo de aprendizado verbal: quando o agente falha, ele reflete sobre a causa e guarda a lição em uma memória de texto; na tentativa seguinte, essa memória orienta as decisões. O agente não atualiza os pesos — atualiza um diário de bordo que melhora a próxima tentativa. O nome dado pelos autores, *verbal reinforcement learning*, captura a ideia: a recompensa e a correção são expressas em linguagem, não em números.

A Self-Refine, de Madaan et al. (2023), explorou a mesma intuição por outro ângulo: o mesmo modelo gera uma saída, critica a própria saída e a reescreve, repetindo o ciclo algumas vezes sem qualquer treinamento. O modelo atua como gerador e como crítico de si mesmo. Essa abordagem produziu ganhos consistentes em uma ampla variedade de tarefas, e tem a vantagem de não exigir dados nem recompensas.

É aqui que este livro precisa fazer uma distinção importante, que o AGENTS.md estabelece: *reflexão do próprio modelo não é validação independente*. A Reflexion e a Self-Refine são úteis — elas corrigem a rota usando informações que o próprio modelo produz. Mas, quando a tarefa permite, a validação deve vir de fora: executar o código, rodar o teste, verificar o schema. O clássico exemplo é a Reflexion em tarefas de código: o modelo reflete sobre o erro, mas quem confirma se a correção funciona é o interpretador. Reflexão corrige a direção; verificação externa confirma a chegada. Um sistema robusto usa as duas, na ordem certa.

Os limites do loop

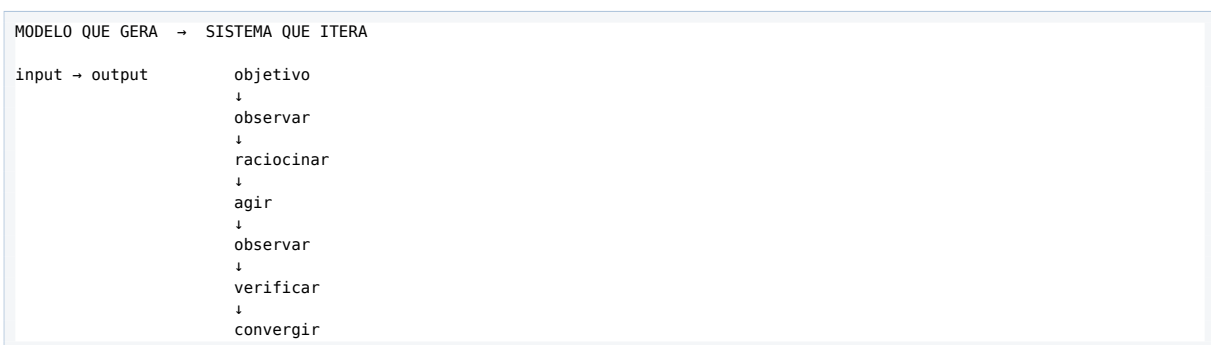
O loop resolveu o problema da correção de rota, mas introduziu dois custos que a era dos agentes carregaria desde então. O primeiro é a linearidade: um loop ReAct percorre uma trajetória linear — cada passo depende do anterior — e, portanto, um erro no início se propaga por toda a cadeia. O modelo pode corrigir, mas corrigir exige que o erro seja detectável na observação; se o modelo interpreta mal uma observação intermediária, o engano persiste. Trajetórias lineares acumulam erros iniciais.

O segundo custo é de recursos: cada ciclo consome tokens, e tokens custam tempo e dinheiro. Um loop longo pode gastar dezenas de chamadas para responder o que um modelo faria em uma. A latência também cresce a cada ida e volta. Por isso, a era seguinte investiria em formas de escapar da linearidade: planejamento topológico que divide a tarefa em ramos independentes (DAGs), busca em árvore que explora múltiplos caminhos em vez de um só — como Tree of Thoughts e LATS — e planejadores dedicados que decidem a sequência antes de agir. O capítulo 13 tratará desses caminhos. O ReAct estabeleceu o loop; o que veio depois foi aprender a não desperdiçá-lo.

A lição estrutural

O ReAct é, em muitos sentidos, o ponto em que o livro converge com seu próprio prefácio. O loop cognitivo descrito na abertura — observar, raciocinar, agir, observar, verificar — é exatamente o padrão que o ReAct tornou concreto em 2022. Antes dele, era possível imaginar o loop como abstração; depois dele, ele se tornou um padrão de engenharia com implementação, resultados e limites conhecidos.

A mudança de natureza que o ReAct produz pode ser resumida assim:



Essa passagem — do modelo como produtor de texto para o sistema como produtor de resultado — é a tese central do livro, e o ReAct é o mecanismo concreto que a tornou prática. Os capítulos anteriores deram ao modelo língua, obediência, produto e mãos. Este capítulo deu o último ingrediente: a possibilidade de usar as mãos repetidamente, corrigindo a rota a cada observação, até chegar a um resultado verificável. Com isso, a era das ferramentas estava completa — e a era dos agentes podia

começar.

Para o engenheiro

Desenhe agentes no molde do ReAct. Thought → Action → Observation é a forma de todo agente, mesmo por baixo dos frameworks: é esse loop que roda quando o framework esconde a complexidade. Em vez de procurar “prompts mágicos”, projete o ciclo — o que o modelo deve raciocinar, que ferramentas pode chamar, como as observações voltam e quando o loop deve parar.

Sem observação das próprias ações não há correção de rota. Portanto, logue sempre o que o agente pensou, o que chamou e o que viu. Um loop sem log é um buraco negro: quando a resposta final está errada, não há como reconstruir a trajetória que a produziu. O log é o que permite debugar, reexecutar e melhorar — a observabilidade de que trata o AGENTS.md.

Loop custa tokens e tempo. Defina teto de iterações, orçamento de tokens e critério de parada antes de subir para produção. Um agente sem `max_steps` em ambiente real é uma conta aberta: pode rodar indefinidamente, acumulando custo e latência. A condição de parada é parte do design, não um detalhe.

Reflexão do próprio modelo ajuda, mas não substitui checagem externa. Quando a tarefa permite, valide a saída com execução, teste ou schema — não com o mesmo modelo olhando para si mesmo. Use a reflexão para orientar a próxima tentativa; use validadores externos para confirmar a chegada. Essa separação entre gerador e verificador é o que separa iteração produtiva de iteração que apenas repete o erro.

Fontes: [Yao et al., 2022] — ReAct; [Shinn et al., 2023] — Reflexion; [Madaan et al., 2023] — Self-Refine; [survey de tool use, 2026] — contexto de orquestração multi-ferramenta; [Taskade, 2026] — histórico e números do ReAct.

Capítulo 11 — Function calling, structured outputs e computer use (2023-2024)

O capítulo anterior terminou com o loop como forma. Este capítulo mostra o que aconteceu quando essa forma deixou os papers e virou superfície padrão de API. Em cerca de trinta meses — do ReAct, em outubro de 2022, à Responses API, em março de 2025 — “chamar ferramenta” deixou de ser tema de pesquisa e se tornou um parâmetro de chamada de API. A pergunta desta era era de padronização: cada provedor inventava seu jeito de declarar uma função, e o modelo devolvia um JSON que quebrava a aplicação. O problema não era mais provar que tool use era possível; era torná-lo confiável o bastante para produção.

A resposta veio em camadas. Function calling tornou a ferramenta uma primitiva da API, com o nome e os argumentos devolvidos em uma estrutura que a aplicação pode executar sem parsing frágil. JSON mode forçou o formato da saída. As chamadas paralelas cortaram a latência de loops que dependiam de várias ferramentas. E os Structured Outputs, com `strict: true`, garantiram que o que saísse do modelo satisfizesse o schema declarado — o fim da era “o modelo devolveu JSON inválido”. Então a fronteira se expandiu de novo, na direção que poucos esperavam: o computer use, em outubro de 2024, transformou a tela inteira em ferramenta.

A ambição deste capítulo é silenciosa e central. Quando a interface para ferramentas é padronizada — função descrita em JSON, argumentos validados, saída garantida por schema — o *tool use* vira engenharia de produção, não experimento. E, quando a camada de ferramentas é padronizada, ela se prepara para virar commodity: o diferencial deixa de ser “conseguir chamar uma função” e passa a ser o que se faz com a orquestração, os dados e a verificação. Esse é o caminho que os capítulos 16 e 17 completarão, com MCP e A2A.

ChatGPT plugins: o primeiro ecossistema em escala de produto

Em 23 de março de 2023, a OpenAI anunciou os plugins do ChatGPT. O marco não foi a tecnologia — o capítulo 9 já havia mostrado modelos usando ferramentas — mas a escala e a abertura. Pela primeira vez, um produto usado por milhões de pessoas permitia que desenvolvedores externos declarassem capacidades que o assistente podia invocar no meio do diálogo: navegação na web, execução de código, busca em serviços de terceiros. O modelo, que o capítulo 7 tinha transformado em produto, transformava-se agora em plataforma.

O desenho já carregava a divisão de responsabilidades que o function calling formalizaria meses depois: o desenvolvedor descreve a interface da ferramenta; o modelo decide quando chamá-la; a aplicação executa; e o resultado volta para o modelo. Os plugins foram aposentados em 2024 em favor dos GPTs, mas a ideia estrutural permaneceu. Eles provaram que havia demanda real por um assistente que não apenas respondesse, mas *fizesse* — e que esse fazer dependia de uma camada de integração padronizada entre o modelo e o mundo.

Function calling: a ferramenta vira primitiva de API

Em 13 de junho de 2023, a OpenAI lançou o function calling como parte da API. A mudança parece pequena e é enorme: até então, o tool use dependia de o modelo escrever texto e a aplicação interpretá-lo; a partir daí, a decisão e o formato passaram a ser parte da própria API. O desenvolvedor descreve as funções disponíveis — com nome, descrição e JSON Schema para os argumentos — e o modelo devolve uma chamada estruturada que a aplicação pode executar diretamente.

1. o desenvolvedor declara as funções (JSON Schema)
2. o modelo decide se deve chamar alguma
3. o modelo devolve {"nome da função", "argumentos"}
4. a aplicação executa e devolve o resultado
5. o modelo continua de onde parou

Esse é o ponto exato em que o ReAct vira produto. O loop do capítulo anterior — pensar, agir, observar — continua existindo, mas agora o “agir” é entregue por uma primitiva de API: em vez de o modelo escrever `search(...)` como texto e esperar que a aplicação parseie, ele devolve uma chamada estruturada que o sistema reconhece de forma determinística. A fragilidade do parsing de texto desaparece da camada de transporte, e o engenheiro pode se concentrar no que realmente importa: decidir quais ferramentas existem e o que fazer com os resultados.

A padronização também mudou a natureza do problema para os provedores. Declarar funções em JSON Schema transformou a chamada de ferramenta em um problema de *schema*: o modelo precisava aprender a respeitar tipos, campos obrigatórios e enumerações. Isso aproximou o tool use do problema de output estruturado — e preparou o terreno para os Structured Outputs do ano seguinte. A era da divergência, porém, começaria antes.

Expansão e divergência: três provedores, três formatos

O sucesso do function calling abriu a porta para os concorrentes — e cada um inventou o próprio mecanismo. No DevDay de novembro de 2023, a OpenAI apresentou o *parallel function calling*, o JSON mode e a Assistants API. Em 21 de novembro de 2023, a Anthropic lançou o Claude 2.1 com tool use em beta e 200 mil tokens de contexto. Em dezembro de 2023, a Google expôs function calling no Gemini, com funções declaradas em schema compatível com OpenAPI. Três provedores, três formatos, três maneiras de declarar e devolver uma chamada.

Provedor	Mecanismo	Lançamento
OpenAI	function calling (functions)	jun/2023
OpenAI	parallel function calling, JSON mode	nov/2023
Anthropic	tool use (Claude 2.1)	nov/2023
Google	function declarations (Gemini)	dez/2023

A divergência produziu um problema que a era dos protocolos resolveria: portabilidade. Um sistema que usasse o formato da OpenAI não migrava trivialmente para a Anthropic ou a Google sem reescrever a camada de ferramentas. Para o engenheiro, isso significava que o tool use, recém-industrializado, já estava fragmentado. A padronização da interface — feita por protocolos abertos como MCP — se tornaria a resposta a esse problema, e é o tema dos capítulos 16 e 17. Antes disso, porém, era preciso medir.

Medir para industrializar

Nenhuma camada de infraestrutura se torna confiável sem medição. Em fevereiro de 2024, o Berkeley Function-Calling Leaderboard, do grupo Gorilla, começou a avaliar chamadas de função de forma quantitativa, comparando modelos pela capacidade de produzir invocações corretas em cenários de múltiplas ferramentas. O gesto é característico da maturidade da área: o que antes era demonstrado por exemplos passava a ser pontuado por números, com critérios explícitos e comparação direta entre modelos.

Em junho de 2024, o tau-bench, de Yao et al., acrescentou uma dimensão diferente: consistência. Em vez de medir apenas se o agente acerta a tarefa, o benchmark mede se ele repete o comportamento correto entre tentativas — uma propriedade que importa mais para produção do que para pesquisa. Um agente que acerta uma vez e falha nas outras não é confiável; um que se comporta de forma consistente permite prever custo, latência e qualidade. Medir consistência, e não apenas acurácia pontual, é o tipo de refinamento que separa uma técnica promissora de uma infraestrutura utilizável.

Structured Outputs: o fim do JSON inválido

Em 6 de agosto de 2024, a OpenAI lançou os Structured Outputs. A promessa é direta: com `strict: true`, a saída do modelo é garantida para satisfazer o JSON Schema declarado. Nos testes da própria OpenAI, o novo modelo atingiu 100% de conformidade em avaliações de schemas JSON complexos, contra menos de 40% da geração anterior. O que antes era probabilidade passou a ser contrato.

A diferença é sutil e fundamental. Function calling já estruturava as chamadas de ferramenta; os Structured Outputs estenderam a garantia a qualquer saída consumida por código. Para o engenheiro, isso significa o fim de uma classe inteira de bugs: o parser da aplicação não quebra mais porque o modelo devolveu um JSON inválido, um campo a mais, um tipo errado. A validação que antes precisava ser feita por código — com retentativas, fallbacks e tratamento de erro — passa a ser responsabilidade da API.

É preciso, porém, manter a distinção que este livro repete: a garantia é de *forma*, não de *conteúdo*. O schema garante que os campos existem, têm o tipo certo e seguem a estrutura declarada; não garante que o valor preenchido esteja semanticamente correto. Um sistema de produção precisa das duas camadas: a garantia estrutural da API para o formato, e a verificação externa — execução, testes, validação de negócio — para o sentido. Structured Outputs resolveu metade do problema; a outra metade continuou sendo responsabilidade do engenheiro.

Computer use: quando a ferramenta é a tela inteira

Em 22 de outubro de 2024, a Anthropic apresentou o computer use. A mudança de escala é conceitual: em vez de expor funções discretas, o modelo recebe uma captura da tela e produz ações de mouse e teclado. Screenshot entra, cliques e digitação saem. A “ferramenta” deixou de ser uma API específica e passou a ser a interface gráfica inteira — qualquer aplicação, qualquer site, qualquer fluxo, desde que visível.

```

captura da tela
↓
modelo decide a próxima ação (mover, clicar, digitar)
↓
sistema executa
↓
nova captura de tela
↓
loop até o objetivo

```

O gesto estende o tool use de APIs estruturadas para interfaces não estruturadas — e é exatamente essa a troca. Onde não existe API, a tela se torna a interface universal. Isso abre um espaço enorme: sistemas legados, ferramentas internas, ambientes para os quais ninguém construiu integração. Mas tem um custo alto. Cada ação envolve captura de tela, processamento visual e decisão; a latência é muito maior que uma chamada de API, o consumo de tokens por ação é elevado, e a observação visual introduz novas fontes de erro — pixels ambíguos, elementos que mudam de posição, janelas que sobrepõem. Para o engenheiro, a orientação é clara: prefira APIs e integrações estruturadas sempre que existirem; reserve o computer use para ambientes sem alternativa.

Convergência: do parâmetro de API ao protocolo

No fim de 2024, a camada de ferramentas estava padronizada no nível de cada provedor, mas fragmentada entre provedores. Em 2025, a convergência começou. A OpenAI unificou chat, ferramentas e estado na Responses API. E, mais importante, os protocolos abertos começaram a uniformizar a camada de ferramentas independentemente do provedor: o MCP, que o capítulo 16 apresentará, padroniza como um agente descobre e chama ferramentas de qualquer servidor; o A2A, tema do capítulo 17, padroniza a comunicação entre agentes. O tool use, que nasceu como pesquisa em 2021 e virou primitiva de API em 2023, estava se tornando commodity.

```

2021 WebGPT: ferramenta como treinamento
2022 ReAct: ferramenta como loop
2023 Function calling: ferramenta como primitiva de API
2024 Structured Outputs: garantia de schema
    Computer use: tela como ferramenta
2025 Responses API + MCP + A2A: camada de ferramentas commodity

```

Essa é a trajetória que dá sentido ao título do capítulo. A padronização não é o fim do trabalho; é o que liberta o trabalho para subir de nível. Quando a interface para ferramentas é commodity, o valor migra para a orquestração, a qualidade dos dados, a verificação e o design dos agentes — os temas das próximas partes do livro. A era das ferramentas termina, portanto, não porque as ferramentas deixaram de importar, mas porque importam tanto que viraram infraestrutura: algo que se assume, não se constrói.

A lição estrutural

A era da padronização consolidou a lição que os capítulos de protocolos vão repetir: **contrato vence convenção**. O function calling venceu porque transformou a chamada de ferramenta em uma garantia de API — nome, argumentos e schema declarados e validados —, em vez de depender de o modelo “se lembrar” de obedecer a um formato descrito no prompt. A mesma lógica se repete em cada nível: o JSON Schema é para o tool use o que o grounding foi para o RAG no capítulo 6 — a garantia de forma que permite confiar no transporte. E é essa a segunda lição, complementar e essencial: **garantia de forma não é garantia de conteúdo** — o contrato assegura que os campos existem e têm o tipo certo, mas não que o valor esteja certo; o sentido continua sendo responsabilidade do sistema, com validação própria. Contrato resolve a camada de transporte; o sistema resolve o resto. Essa separação é o que permite à camada de ferramentas virar commodity sem que o sistema perca rigor.

Para o engenheiro

Use o function calling nativo em vez de forçar o modelo a “emitir JSON”. Declare as ferramentas no schema da API, deixe o provedor devolver a chamada estruturada e concentre o código na execução e no tratamento do resultado. Forçar formato por prompt é o caminho do capítulo 9: funciona, mas é frágil e depende do modelo “se lembrar” de obedecer.

Sempre que a saída for consumida por código, exija Structured Outputs — JSON Schema com `strict: true`. O parser da sua aplicação nunca mais quebra por JSON inválido. Mas não confunda garantia de schema com garantia de conteúdo: a estrutura é contratual, o sentido continua sendo responsabilidade do seu sistema, com validação e testes próprios.

Agrupe chamadas independentes em parallel function calling. Um loop que chama várias ferramentas uma a uma multiplica latência por round-trip; chamar em paralelo corta o tempo total. A regra é de custo: ferramentas independentes não precisam esperar umas pelas outras.

Computer use é sedutor, mas caro e frágil. A tela como ferramenta deve ser a última opção, reservada a ambientes sem API. Quando a integração estruturada existe — ou pode ser construída — ela é mais rápida, mais barata e mais confiável. Computer use resolve o problema de não ter API; não é uma forma melhor de resolver o que já tem.

Trate a camada de ferramentas como commodity em evolução. A interface padronizada não é um diferencial duradouro; é pré-requisito. O valor do sistema está na orquestração, nos dados e na verificação — e é para lá que os próximos capítulos apontam.

Fontes: [OpenAI, 2023] — ChatGPT plugins; function calling; DevDay; [Anthropic, 2023] — Claude 2.1; [Google, 2023] — Gemini API function calling; [OpenAI, 2024] — Structured Outputs; [Anthropic, 2024] — Computer use; [Berkeley Leaderboard, 2024] — avaliação de function calling; [Yao et al., 2024] — tau-bench; [Taskade, 2026] — timeline de tool use na indústria; [Konishi, 2026] — timeline de tool use e protocolos.

Parte V — Era do RAG como sistema (2023-2025)

Capítulo 12 — De RAG ingênuo a RAG modular e agêntico (2023-2025)

O capítulo 6 provou a direção: recuperar evidência e condicionar a geração a ela reduz alucinação e dá proveniência à resposta. Mas o RAG clássico, no mínimo que a Lewis et al. apresentaram, era um pipeline fixo — recuperar sempre, um número fixo de passagens, colar e gerar. A pergunta que define esta era é cruel e de engenharia: por que “recuperar e colar” ainda erra tanto em produção? A resposta é que o RAG mínimo colapsa para dentro de si mesmo um conjunto de problemas que a demonstração de laboratório esconde: chunking ruim corta a evidência ao meio, o top-k traz ruído que confunde o modelo, uma única busca não resolve perguntas que exigem juntar evidências de dois documentos, e sem metadados o modelo não consegue priorizar data ou fonte.

Entre 2023 e 2025, o RAG deixou de ser um pipeline e virou um sistema. Primeiro, o pré-processamento: parsing correto, chunking deliberado, metadados, hierarquia de resumos. Depois, o retrieval parou de errar: busca híbrida, reescrita de consulta, reranking com cross-encoder. Nesse ponto, o pipeline virou uma orquestração de módulos substituíveis — o que o survey de Gao et al. (2023) batizou de **Modular RAG**. E então a mesa virou de vez: *e se o quando, o o quê e o quantas vezes recuperar fossem decisões do próprio modelo?* O Self-RAG treina o modelo a decidir se recupera; o FLARE recupera no meio da geração quando a confiança cai; o RAFT treina o modelo a ignorar distratores; o IRCot intercala raciocínio e busca. A recuperação deixou de ser um passo fixo e virou uma decisão de política — e, em seguida, uma ferramenta dentro de um loop de agente.

A ambição desta era pode ser resumida assim: fazer da recuperação uma capacidade adaptativa, medida por estágio, orquestrada por fluxo e, finalmente, decidida pelo próprio modelo. É o elo entre a Parte II (RAG clássico) e a Parte IV (ferramentas e loops). E o capítulo responde, no caminho, a pergunta prática que todo mundo faz em 2025: quando o contexto longo cabe inteiro, para que RAG?

Por que “recuperar e colar” erra: o RAG ingênuo

O ponto de partida é o pipeline mínimo que o capítulo 6 formalizou, agora com um nome pejorativo que a literatura adotou: **Naive RAG**.

```
pergunta
↓
retrieve top-k
↓
concatena [pergunta + passagens]
↓
generate
↓
resposta
```

Ele funciona em demonstrações e falha em produção. As razões são estruturais, e vale classificá-las, porque cada classe de falha vai ter uma resposta neste capítulo. A primeira é o **chunking ruim**: o documento é fatiado em pedaços fixos, e a evidência que importa fica cortada ao meio, ou o pedaço recuperado não contém a informação inteira. A segunda é o **ruído no contexto**: o top-k traz passagens irrelevantes, o modelo é confundido por elas e produz aquilo que se chama, com razão, “alucinação com apoio” — uma resposta errada, mas ancorada em algo que foi colado no prompt. A terceira é a **recuperação single-shot**: busca-se uma vez e pronto; perguntas que exigem juntar evidências de dois documentos (multi-hop) falham porque uma única consulta não captura os dois lados. A quarta é o **top-k fixo**: o mesmo número de passagens para perguntas fáceis e difíceis, de modo que ou falta evidência, ou sobra ruído. A quinta é a **ausência de metadados**: não há data, fonte, tipo ou seção, e o modelo não consegue priorizar informação mais recente ou mais confiável.

O chunking merece um exemplo concreto, porque é a falha mais barata e mais subestimada. Um documento afirma:

"...A receita da vacina foi autorizada em 2021. A agência reguladora exigiu testes adicionais antes da aprovação final. O fabricante..."

chunk 1: "...A receita da vacina foi autorizada em 2021. A agência reguladora exigiu testes
chunk 2: "adicionais antes da aprovação final. O fabricante..."

A pergunta “quando a vacina foi aprovada?” precisa da frase que começa no chunk 1 e termina no chunk 2. Se o retrieval só retorna chunks individuais, a resposta fica incompleta — ou o modelo inventa a data que falta. O problema não é o modelo nem o índice; é **como o texto foi fatiado**. O insight central desta seção é que, no RAG, *garbage in, garbage out* tem nome próprio: a resposta só pode ser tão boa quanto a **janela de evidência** que chegou ao prompt. Todos os módulos que veremos a seguir existem para alargar e limpar essa janela.

Advanced RAG I: pré-processamento e chunking

A primeira linha de defesa é tratar o documento como dado estruturado antes de indexá-lo. O **parsing** não é “abrir o arquivo”: PDFs têm colunas, cabeçalhos, rodapés e páginas cujo layout quebra a ordem de leitura; tabelas precisam virar texto linear; imagens podem precisar de OCR ou de um modelo multimodal. Um parsing correto preserva a ordem lógica — o erro clássico é o texto “montado” fora de ordem, que transforma cada pergunta em um problema impossível para o retrieval.

O **chunking** é a decisão mais barata e mais subestimada do pipeline. Há três dimensões de estratégia, com usos diferentes:

Estratégia	Como funciona	Quando usar
Por tamanho fixo	N tokens (ex.: 512), com sobreposição (ex.: 50)	corpora homogêneos, textos corridos
Por seção/semântica	quebra em cabeçalhos, parágrafos, listas	documentos com estrutura (manuais, legislação)
Por frase/tópico	agrupa frases coerentes; às vezes com LLM	perguntas que exigem a passagem inteira

A sobreposição (*overlap*) existe exatamente para mitigar o corte do exemplo anterior: pedaços consecutivos compartilham uma margem, então a frase cortada tem chance de aparecer inteira em algum chunk. O custo é mais chunks indexados e mais candidatos por consulta. Por fim, **metadados e hierarquia** anexam a cada chunk o documento, a seção, a data e o tipo, o que habilita dois ganhos: **filtros** (buscar só na legislação de 2025) e **citação precisa** (voltar do chunk para o trecho original, preservando proveniência). Técnicas como a **sumarização recursiva** (map-reduce de chunks em resumos pai) criam uma hierarquia: busca-se no nível do resumo, lê-se no nível do chunk.

Advanced RAG II: indexação e recuperação

Com os chunks limpos, a segunda linha é fazer o retrieval parar de errar. Há quatro alavancas, em ordem crescente de custo. A primeira é a **busca híbrida** (léxica + densa). O BM25 do capítulo 5 é imbatível para nomes próprios, códigos e termos exatos; o retriever denso captura paráfrase. Produção usa os dois, combinando os rankings — e o jeito mais simples é a **fusão por ranque (RRF)**:

$$\text{score_final}(d) = \sum_{\{\text{sistemas}\}} \frac{1}{(k + \text{rank}_s(d))} \quad \text{com } k \approx 60$$

Em vez de comparar escalas incompatíveis (frequências do BM25 versus cossenos de embeddings), o RRF soma o *inverso do posto* em cada ranking: se o documento aparece em primeiro no léxico e em quarto no denso, contribui $1/61 + 1/64$. É estável, não exige calibrar pesos e melhora o recall sem exigir um modelo novo.

A segunda alavanca é a **query rewriting**. A pergunta do usuário raramente é a melhor consulta; um LLM a reescreve antes de buscar. A reescrita pode decompor perguntas compostas em subconsultas (cada uma buscando de forma independente), adicionar sinônimos ou transformar pergunta em afirmação. O **HyDE**, de Gao et al. (2022), leva a ideia ao extremo: em vez de buscar pelo embedding da pergunta, o sistema gera uma *resposta hipotética* e busca pelo embedding dela. Quando a pergunta e o documento não compartilham vocabulário, a resposta inventada “traduz” a intenção para a língua do corpus — e os experimentos mostraram que, mesmo com os detalhes falsos da resposta gerada, o encoder denso funciona como um compressor com perdas que filtra o ruído e ancora o vetor no espaço dos documentos reais.

A terceira alavanca é o **reranking com cross-encoder**, e é, de longe, o módulo com melhor retorno/custo do pipeline. O retriever denso é rápido porque pontua embeddings pré-computados — mas

ele nunca vê pergunta e documento *juntos*. Um reranker cross-encoder recebe o par (pergunta, documento) e pontua a relevância conjunta. É caro, porque é uma inferência por par; por isso se usa no top-100→top-5: recupera largo, reordena fino. Ele corrige o ruído que o retrieval denso deixou passar. A quarta alavanca, **multiquery**, gera várias formulações da mesma pergunta, busca com cada uma e junta os candidatos — aumenta o recall ao custo de mais chamadas.

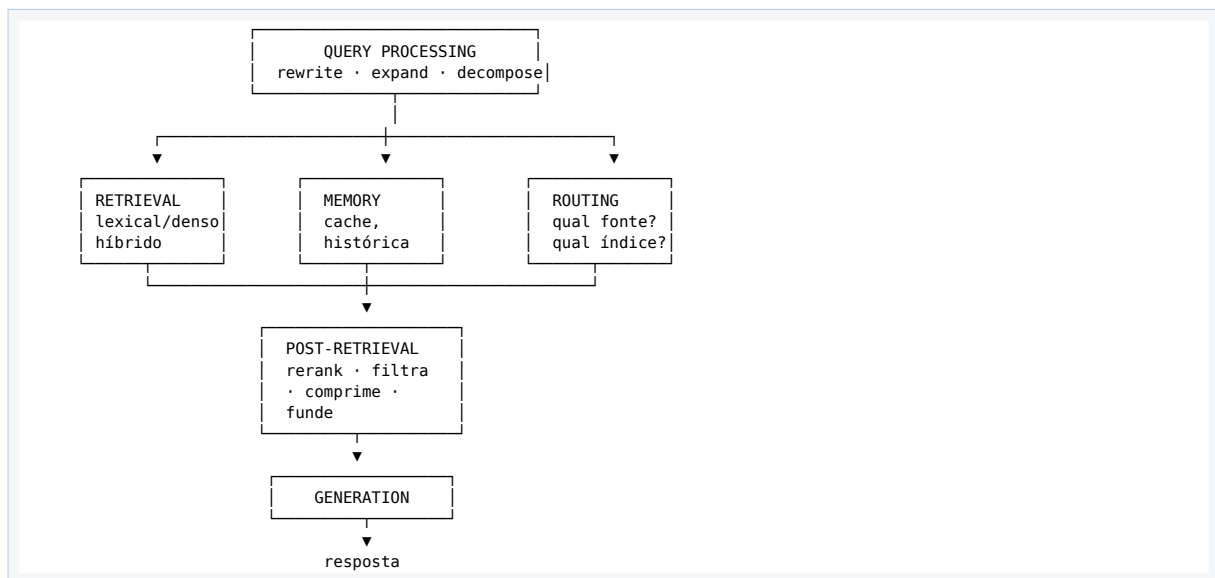
As métricas que medem essas alavancas são as clássicas de IR, agora por estágio:

Métrica	O que mede	Onde
recall@k	a evidência certa está no top-k?	retriever
precision@k / MRR	o ranking está bem ordenado?	retriever + reranker
NDCG	a ordem importa com relevância graduada	reranker
fidelidade	a resposta segue as evidências dadas?	generator
utilidade	a resposta atende ao usuário?	generator (LLM-as-judge)

A regra de ouro dessa seção: **cada módulo tem a sua métrica**. Se a resposta final piora, o problema pode estar no chunking, no retriever, no reranker ou no generator — e um número agregado não diz onde.

Modular RAG: o pipeline vira sistema

Quando se acumulam os módulos das duas seções anteriores, o pipeline deixa de ser uma cadeia fixa e passa a ser uma **orquestração de módulos independentes** — o survey de Gao et al. (2023) batizou isso de **Modular RAG**. As peças:



As propriedades que distinguem *modular* de *cadeia fixa* são quatro. Primeiro, **módulos substituíveis**: troca-se o retriever (BM25 → denso → híbrido) sem tocar no generator, e testa-se cada peça isoladamente. Segundo, **fluxo não-linear**: há loops — gerar, validar, recuperar de novo — e branchings: perguntas fáceis pulam o retrieval, perguntas compostas fazem subconsultas. Terceiro, **memória de processo**: o sistema guarda o que já recuperou ou gerou, como cache de consultas repetidas e histórico da conversa, para não repetir trabalho. Quarto, **iteração explícita**: o passo “validate” re-alimenta o retrieval — se a primeira evidência não sustenta a resposta, busca-se mais uma vez. Esse último é o germe do agentic, e ele aparece até em pseudocódigo enxuto:

```

def rag_modular(pergunta, contexto=None):
    consultas = reescrever(pergunta)                # 1. query processing
    candidatos = set()
    for q in consultas:
        candidatos |= buscar_hibrido(q, top_k=100)   # 2. recuperação (possivelmente híbrida)
    top5 = rerank(pergunta, list(candidatos))[:5]    # 3. post-retrieval
    if contexto:                                     # 4. memória do processo
        top5 += contexto["evidenciasanteriores"]
    resposta = gerar(pergunta, top5)                 # 5. generation
    if validar(resposta, top5) < limiar:              # 6. iteração
        return rag_modular(pergunta, contexto={"evidenciasanteriores": top5})
    return resposta, top5                           # resposta + proveniência
  
```


A lição didática é que **a complexidade não está em nenhum módulo, está no fluxo entre eles**. É por isso que o capítulo 13 (frameworks de agentes) reaparecerá exatamente aqui: a orquestração de módulos com loops é a mesma forma que os frameworks de agente codificam.

Aprendendo a recuperar: Self-RAG, FLARE, RAFT, IRCot

Até aqui, o *quando* e o *quantas vezes* recuperar eram decididos por heurística — sempre, uma vez, top-k fixo. A virada de 2023 foi fazer o **próprio modelo** aprender a decidir. Quatro abordagens mostram o espectro, do treinamento à inferência.

O **Self-RAG**, de Asai et al. (2023), treina o modelo com *tokens de reflexão* que controlam o fluxo. Um token decide se deve recuperar para aquele passo, e outros criticam a resposta: a evidência é suficiente? a resposta é apoiada pelas passagens? Na inferência, o modelo recupera só quando o token indica necessidade, pode fazer várias rodadas e critica a própria saída. É retrieval como *política aprendida*, não como passo fixo. Nos experimentos, os modelos de 7B e 13B superaram o ChatGPT e o Llama2-chat com retrieval em QA de domínio aberto, raciocínio e verificação de fatos — e melhoraram a acurácia factual e de citação em gerações longas. O gesto é duplamente importante: a recuperação vira decisão, e a decisão é monitorada por autoavaliação.

O **FLARE**, de Jiang et al. (2023), faz recuperação *ativa durante a geração*. O modelo gera uma sentença provisória; se a confiança nas próximas tokens cai abaixo de um limiar, ele usa essa sentença como consulta, recupera e regeira a parte incerta. A ideia é “olhar para frente”: em vez de buscar pelo passado já escrito, antecipa-se o conteúdo futuro. Funciona sem treinamento adicional — apenas um LLM de inferência e um retriever — e superou linhas de base de recuperação única e de múltiplas recuperações em tarefas de geração longa, com o maior ganho em QA multi-hop (2WikiMultihopQA), onde cada passo de raciocínio exige informação nova.

O **RAFT**, de Zhang et al. (2024), ataca o problema por outro lado: em vez de apenas recuperar melhor, o modelo é *treinado* para lidar com retrieval imperfeito. Os dados de treino incluem o documento “oracle” (aquele que contém a resposta) junto de documentos distratores — e, em uma fração dos exemplos, o oracle é omitido por completo. O modelo aprende a citar verbatim o trecho relevante e a ignorar os distratores, produzindo respostas em estilo chain-of-thought. É a metáfora do livro aberto: quem estuda para a prova de consulta aberta aprende a reconhecer o que é relevante, e não apenas a ler tudo. O RAFT melhorou o desempenho de fine-tuning supervisionado com e sem RAG em PubMed, HotpotQA e no benchmark Gorilla.

O **IRCot**, de Trivedi et al. (2023), intercala *chain-of-thought* e *retrieval*: cada passo de raciocínio gera uma consulta, o resultado volta ao raciocínio, que gera a próxima consulta — até a cadeia terminar. É a forma de decompor problemas multi-hop sem uma arquitetura nova: o mesmo modelo raciocina e decide o que buscar. O exemplo canônico do paper é a pergunta sobre o país onde uma montanha-russa foi fabricada: a primeira busca (pela própria pergunta) não responde; é preciso inferir o fabricante e então buscar o país. Os ganhos chegaram a 21 pontos de recall e 15 pontos de F1 sobre a recuperação de passo único, em HotpotQA, 2WikiMultihopQA, MuSiQue e IIRC, com redução dos erros factuais no raciocínio intermediário.

O fio comum das quatro abordagens pode ser visto na tabela:

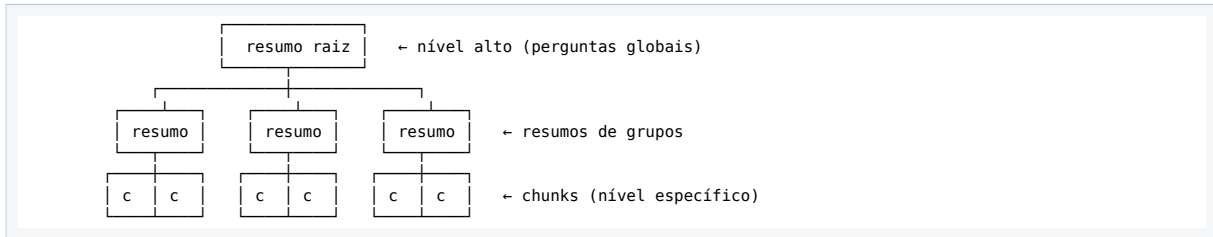
Método	Ideia central	Onde foi avaliado
Self-RAG	o modelo decide se recupera e critica a própria resposta	PopQA, TriviaQA, PubHealth, ARC-Challenge, ASQA
FLARE	recupera <i>durante</i> a geração quando a confiança cai	2WikiMultihopQA, StrategyQA, ASQA, WikiAsp
RAFT	treinado a ignorar distratores e citar as fontes	PubMed, HotpotQA, Gorilla
IRCot	intercala raciocínio e recuperação (retrieve-then-read iterativo)	HotpotQA, 2WikiMultihopQA, MuSiQue, IIRC

(Multi-hop significa que a resposta exige reunir evidências de mais de um documento — exatamente o caso onde recuperar uma vez não basta.) Em todas, **recuperação deixou de ser um componente fixo e virou uma decisão** — e a decisão foi gradualmente transferida das regras para o modelo.

Estruturando a memória: RAPTOR e GraphRAG

Há um problema que os módulos anteriores não resolvem: *perguntas globais*. “Quais são os temas recorrentes neste relatório?” ou “como essas duas seções se relacionam?” não são respondidas por top-k de chunks — a resposta está *espalhada* pelo corpus. Duas linhas tratam disso.

O **RAPTOR**, de Sarthi et al. (2024), constrói uma **árvore de resumos**: agrupa chunks por similaridade, resume cada grupo, agrupa os resumos de novo — até uma raiz. A consulta é roteada para o nível certo: pergunta específica vai às folhas (chunks), pergunta ampla vai aos nós altos (resumos). É *multigranularidade*: a memória tem zoom.



O ponto central é que a árvore inteira participa do retrieval, não só as folhas: os experimentos mostraram que a busca em todos os níveis supera a busca apenas em camadas específicas. Com o GPT-4, o RAPTOR melhorou o melhor resultado no benchmark QuALITY em 20 pontos absolutos, e estabeleceu estado da arte em NarrativeQA e QASPER. O custo é a indexação recursiva com LLM, que roda uma vez sobre o corpus.

O **GraphRAG**, da Microsoft (Edge et al., 2024), representa o corpus como **grafo de entidades e relações**. Um LLM extrai as entidades (pessoas, empresas, conceitos) e as relações entre elas; o grafo é particionado em comunidades de entidades próximas; e cada comunidade é resumida em um relatório. Perguntas globais são respondidas em modo map-reduce: cada resumo de comunidade gera uma resposta parcial, e as parciais são combinadas na resposta final. A capacidade nova é responder perguntas que *nenhum chunk* contém — “quais são os temas dominantes deste corpus inteiro?” — com melhorias substanciais de abrangência e diversidade sobre o RAG vetorial em corpora de milhões de tokens. O custo é alto, porque a extração roda um LLM sobre o corpus inteiro uma vez; as variantes de 2025 (LightRAG, PGraphRAG) reduziram o custo e melhoraram a atualização incremental do grafo.

A escolha entre as duas, e entre elas e o RAG de chunks, é um trade-off de granularidade:

pergunta específica	→ RAG de chunks (top-k)	→ barato, direto
pergunta multi-hop	→ IRCoT / iterativo	→ médio
pergunta de resumo	→ RAPTOR (níveis altos)	→ médio
pergunta global	→ GraphRAG (comunidades)	→ caro, mas é o único que responde

RAG versus contexto longo: o trade-off que não some

De 2023 a 2025, as janelas de contexto explodiram — de 128k tokens para 1M+ em modelos como Gemini-1.5 e Claude Sonnet. A pergunta se tornou inevitável: *se cabe tudo no prompt, para que RAG?* A resposta honesta tem três partes, e elas mudam o desenho do sistema.

A primeira parte é **custo e latência**. O custo de atenção cresce com o quadrado do contexto: “cabe” não é “é barato”. Colocar um milhão de tokens em toda pergunta custa ordens de magnitude mais caro que recuperar os cinco chunks relevantes — e responde mais devagar. Para produção, o RAG continua sendo o caminho econômico padrão. A segunda parte é a **precisão no meio**. O fenômeno chamado *lost in the middle*, documentado por Liu et al. (2023), mostrou que os modelos usam bem a informação no início e no fim do contexto, e muito pior a que está no meio — uma curva em U que vale para tarefas de QA multi-documento e para recuperação de pares chave-valor. Um prompt gigante pode literalmente “esconder” a evidência relevante no meio do ruído; o RAG, ao encurtar o contexto para só o que importa, evita esse problema por construção.

A terceira parte é o **quando o contexto longo vence**. Perguntas que exigem o documento *inteiro* — uma jurisprudência de 800 páginas, um contrato completo — não cabem em top-k, porque a evidência está distribuída demais. Nesses casos, o contexto longo direto é superior. A comparação sistemática de Li et al. (2024) quantificou o equilíbrio: quando há recursos suficientes, o contexto longo supera o RAG em desempenho médio, mas a custo muito maior — e, decisivamente, em mais de 60% das consultas as previsões das duas abordagens são idênticas. Isso abre espaço para o **Self-Route**, a contribuição prática do mesmo trabalho: um modelo roteia cada consulta, respondendo primeiro pelo caminho barato (RAG) e só enviando ao contexto longo as perguntas que o RAG declara incapaz de responder — com reduções de custo de 65% no Gemini-1.5-Pro e 39% no GPT-4o, mantendo desempenho comparável ao contexto longo. A regra prática:

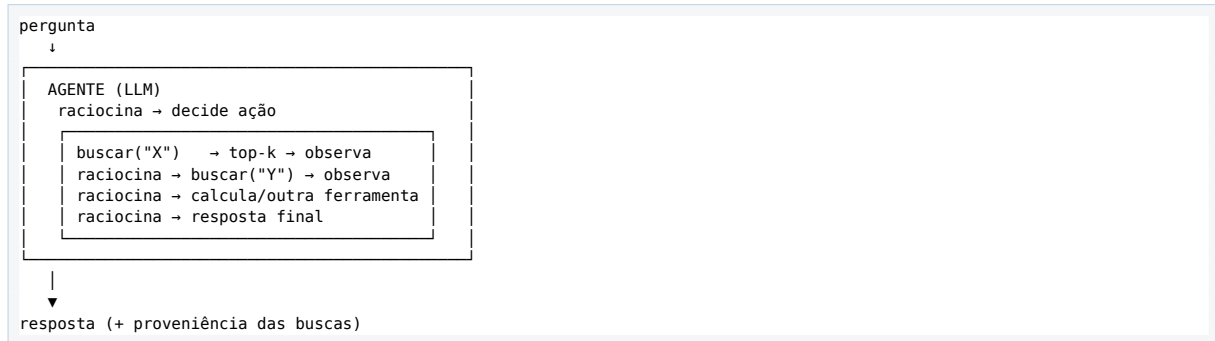
a resposta está em poucos trechos	→ RAG (barato, rápido, proveniente)
a resposta exige o documento inteiro	→ contexto longo
não sei qual é o caso	→ roteamento (Self-Route) ou heurística

O trade-off não desaparece porque as janelas crescem; ele apenas muda de lugar. Agora a decisão é

por consulta, não por arquitetura.

Agentic RAG: recuperação como ferramenta no loop

A convergência do “modelo decide” (Self-RAG, FLARE, RAFT, IRCOT) com o “fluxo com loops” (Modular RAG) é o **Agentic RAG**: a recuperação vira uma *ferramenta* que o modelo invoca dentro de um ciclo observar → raciocinar → agir. Já não existe pipeline; existe um agente que decide com que consulta buscar, quantas vezes, se precisa de outra fonte e quando parar.



O que muda em relação a tudo que vimos até aqui é a abolição das três fixações do pipeline clássico. Não existe mais **top-k fixo**: o agente decide quantas passagens precisa e quando já tem evidência suficiente. Não existem mais **consultas fixas**: a pergunta evolui durante o processo — a raiz do IRCOT, generalizada. E não existe mais o privilégio da recuperação de ser o único “mundo externo”: além do índice, o agente pode chamar calculadora, API, busca na web. A recuperação perde o monopólio da evidência externa e passa a ser uma ferramenta entre outras, no sentido exato dos capítulos 9 a 11.

Este é o ponto onde a Parte V termina e o resto do livro começa. O loop do Agentic RAG é o loop do ReAct (capítulo 10); a orquestração de módulos e ferramentas é o tema dos frameworks (capítulo 13); a avaliação por estágio e por consistência é o tema do capítulo 15. O RAG deixou de ser um produto e virou uma peça de um sistema — o que significa, para o engenheiro, que as perguntas certas deixaram de ser “qual banco vetorial” e passaram a ser “quem decide recuperar, com que critério e quando parar”.

A lição estrutural

Termo	Significado
Naive RAG	retrieve → augment → generate, sem ajustes; o baseline que falha em produção
Advanced RAG	pré-processamento, busca híbrida, reranking, query rewriting
Modular RAG	módulos independentes e orquestráveis, com loops e memória de processo
Chunking	divisão do documento em trechos indexáveis; a decisão de qualidade mais barata
RRF	fusão de rankings por inverso do posto; combina lexical + denso sem calibrar pesos
HyDE	buscar pelo embedding de uma resposta hipotética gerada pelo modelo
Reranker (cross-encoder)	reordena o top-k olhando pergunta + documento juntos
Self-RAG / FLARE / RAFT / IRCOT	aprender a decidir quando recuperar
RAPTOR / GraphRAG	estruturar a memória em árvores de resumo ou grafos de entidades
Lost in the middle	degradação da precisão para informação no meio do contexto
Self-Route	roteamento entre RAG e contexto longo por consulta
Agentic RAG	recuperação como ferramenta dentro de um loop de agente

Este vocabulário é o que a Parte IV reutiliza quando fala de “observar” no loop, e o que a Parte VI reutiliza quando fala de ferramentas de um agente. Quem domina essas distinções não está apenas montando um pipeline; está desenhando o sistema de evidência que o modelo consulta.

Para o engenheiro

Antes de trocar o vector store, meça o ROI dos módulos baratos. Query rewriting, HyDE, reranking e chunking movem mais a qualidade do que o índice em si. É tentador culpar o banco vetorial quando a resposta piora; na prática, a falha está quase sempre na janela de evidência — e a janela se corrige com pré-processamento e pós-processamento, não com um índice novo.

Trate o RAG como um pipeline de módulos substituíveis — rewrite → retrieve → rerank → refine → generate. Isso permite testar e trocar cada peça isoladamente, sem reindexar tudo, e transforma o debug de “a resposta está errada” em “qual estágio vazou”. Use um reranker cross-encoder no top-k: é barato, adiciona pouca latência e corrige o ruído do retrieval denso. Quase sempre compensa.

Separe as métricas por estágio. Recall@k e NDCG para o retriever; fidelidade e utilidade para a geração. Só suba a complexidade de um módulo quando a métrica dele piorar — e não quando a resposta final incomodar, porque o número agregado não diz onde o problema está.

Chunking é a decisão mais subestimada do sistema. Teste tamanho × sobreposição × quebra por seção antes de otimizar qualquer outra coisa; um bom chunking resolve mais que um modelo melhor. E guarde os metadados (fonte, data, seção) desde o índice: sem eles não existe filtro, nem proveniência, nem reranking contextual — e proveniência é o que diferencia RAG de adivinhação.

Escalone a complexidade na ordem certa: híbrido → rerank → iterativo → agentic. Pular etapas cobra caro em debug, porque cada nível só existe porque o anterior vazou. Para perguntas globais (“o que este corpus diz sobre X?”), um chunk não responde — avalie RAPTOR ou GraphRAG; para o resto, o top-k continua sendo o melhor custo-benefício.

Finalmente, quando o contexto longo for uma opção, não a trate como substituto do RAG, mas como alternativa a ser roteada por consulta. Use heurística ou Self-Route para decidir por pergunta, em vez de apostar em uma arquitetura única. E lembre: a decisão de quem recupera, com que critério e quando parar é o desenho central — não o índice.

Fontes: [Gao et al., 2023] — survey RAG (Naive/Advanced/Modular); [Asai et al., 2023] — Self-RAG; [Jiang et al., 2023] — FLARE; [Zhang et al., 2024] — RAFT; [Trivedi et al., 2023] — IRCot; [Sarathi et al., 2024] — RAPTOR; [Edge et al., 2024] — GraphRAG; [Liu et al., 2023] — Lost in the Middle; [Li et al., 2024] — Self-Route; [Gao et al., 2022] — HyDE; [Gupta et al., 2024] — survey abrangente de RAG; [Revisão sistemática, 2025] — síntese de 128 estudos (2020–2025).

Parte VI — Era dos agentes (2022-2025)

Capítulo 13 — Frameworks de agentes: de AutoGPT a LangGraph

Em 2023, todo mundo queria um agente autônomo — e muita gente construiu um que ficava rodando até esgotar o cartão de crédito. O problema desta era foi o hype em cima de loops sem critério de parada: AutoGPT e BabyAGI provaram que a ideia era viável e, na mesma cena, que era frágil. A pergunta madura foi: *o que um agente realmente é?* A resposta estrutural — objetivo, estado, ações, observações, feedback e condição de término — virou o mapa do capítulo. Frameworks não criam a mágica; dão a infraestrutura para ela existir de forma controlável. A ambição evoluiu de “prompts autônomos” para grafos de estado com observabilidade (LangGraph), equipes com papéis (CrewAI), conversação (AutoGen) e agentes que produzem artefatos em vez de chat (MetaGPT).

Este capítulo é a ponte entre o loop que o capítulo 10 (ReAct) estabeleceu e os sistemas multi-agente do capítulo 14. Ele parte da explosão de 2023, passa pela anatomia estrutural de um agente e termina na lição que orienta a escolha de framework: agente não é um prompt — é um sistema com estado, ações, observações, feedback e critério de término.

A explosão de 2023: AutoGPT e BabyAGI

O ponto de partida é o momento em que a palavra “agente” deixou os papers e virou notícia. Em março de 2023, o **AutoGPT**, de Toran Richards (Significant-Gravitas), e o **BabyAGI**, de Yohei Nakajima, invadiram o GitHub como projetos open-source que usavam GPT-4 para perseguir um objetivo de forma *autônoma*: o AutoGPT encadeava pensamento, ação, observação e memória externa para iterar sobre uma tarefa; o BabyAGI era, essencialmente, um *gerenciador de tarefas* que criava, priorizava e executava passos até um objetivo pré-definido. Nenhum deles inventou o loop — o ReAct (capítulo 10) já o tinha formalizado —, mas ambos o *democratizaram*: dezenas de milhares de desenvolvedores rodaram agentes pela primeira vez, com uma linha de comando e uma chave de API.

A mesma cena que provou a viabilidade mostrou a fragilidade. Os loops eram lineares, sem critério de término confiável: o agente repetia tentativas, se perdia da tarefa original (a *deriva*), e o custo crescia a cada chamada. A piada da época — o agente que “rodava até esgotar o cartão de crédito” — era, na verdade, a descrição precisa de um sistema sem condição de parada. A indústria aprendeu a lição em duas frentes: definir estruturalmente o que um agente é e, em seguida, construir infraestrutura para que essa estrutura fosse controlável.

O que é um agente, afinal

A resposta estrutural que organiza a era vem da literatura de surveys: um agente é um sistema que combina **perfil** (quem é o agente — o papel, o objetivo, a identidade), **memória** (curto prazo — a sessão; longo prazo — o que persiste entre sessões), **planejamento** (decompor o objetivo, refletir sobre os passos) e **ação** (chamadas de ferramentas que modificam o mundo). O survey de Wang et al. (2024) mostrou que, sob essa lente, quase todos os sistemas da era — do AutoGPT ao LangGraph — são instâncias da mesma estrutura.

Mas a estrutura mais útil para o engenheiro é a do próprio AGENTS.md:

```
objetivo
+ estado
+ ações
+ observações
+ feedback
+ critério de término
```

Cada um desses componentes tem um papel preciso. O **objetivo** define o que conta como sucesso — sem ele, não existe critério de avaliação. O **estado** é a memória de trabalho: o que o agente sabe sobre onde está. As **ações** são o conjunto de operações que o agente pode executar — buscar, calcular, escrever, chamar API. As **observações** são os resultados das ações voltando ao sistema — sem elas, o

agente age às cegas. O **feedback** é o sinal que permite corrigir o curso. O **critério de término** é a condição que interrompe o loop — e é ele, exatamente ele, que faltou em 2023.

A consequência operacional é direta: qualquer sistema que tenha esses seis elementos é um agente, com ou sem a palavra “agente” no nome; qualquer sistema que não os tenha, não é — mesmo que o prompt comece com “Você é um agente autônomo”. E é por isso que a era seguinte se dedicou a construir infraestrutura para esses elementos: estado explícito, observabilidade, critérios de parada e guardrails.

De cadeias a grafos: LangChain e LangGraph

A trajetória do ecossistema mais usado da era é uma história em dois atos. O **LangChain**, lançado em outubro de 2022, popularizou a *composição de cadeias*: encadear passos de LLM — recuperar, processar, gerar — como peças de um pipeline linear. Foi o jeito mais rápido de colocar uma aplicação de LLM em pé, e por isso virou o ponto de entrada de uma geração inteira de desenvolvedores. Mas cadeias lineares têm um limite estrutural: não expressam *ciclos*. Um agente de verdade precisa voltar ao raciocínio depois da observação — e um DAG (grafo acíclico dirigido), por definição, não tem retorno.

O **LangGraph**, introduzido em janeiro de 2024, foi a resposta: um *grafo de estados* — uma máquina de estados em que nós são etapas (raciocinar, chamar ferramenta, validar) e arestas condicionais decidem o próximo passo, com o estado centralizado e atualizado a cada nó. As propriedades que o distinguem são exatamente as que o hype de 2023 negligenciou:

estado explícito	→	cada nó lê e atualiza o estado central
durable execution	→	o agente sobrevive a falhas e retoma de onde parou
streaming	→	o usuário vê o raciocínio e as ações em tempo real
human-in-the-loop	→	humano inspeciona/edita o estado em qualquer ponto
observabilidade	→	rastreamento de cada transição para debug e métricas

É a mesma lição do capítulo 12 aplicada a agentes: quando o fluxo tem loops, o controle explícito vale mais que a conveniência da abstração. A ideia não era nova — o LangGraph é inspirado no Pregel (Google) e no Apache Beam, e a representação de máquinas de estados é antiga —, mas a aplicação a LLMs fez do “grafo com estado” o idioma dominante da orquestração de agentes. É por isso que o capítulo 10 tinha razão ao dizer que LangGraph é “ReAct com grafos e estados”: o loop cognitivo é o mesmo, o que muda é a infraestrutura que o torna previsível.

Equipes, conversas e artefatos

Se LangGraph é o *runtime* de baixo nível, uma segunda onda de frameworks explorou a pergunta *quantos agentes e como eles se comunicam* — o tema que o capítulo 14 tratará em profundidade. Três modelos mentais se destacaram.

O **CrewAI**, de João Moura, organiza agentes como *equipes com papéis*: cada agente tem um role, um goal e um backstory, e os *crews* executam *tasks* em processos sequenciais ou hierárquicos. É o modelo mental mais próximo de uma empresa: especialistas com identidades definidas que colaboram para um objetivo comum, com delegação de tarefas entre eles. O **AutoGen**, da Microsoft (Wu et al., 2023), trata agência como *conversação*: agentes conversáveis — como o AssistantAgent e o UserProxyAgent — trocam mensagens, com um GroupChatManager escolhendo dinamicamente quem fala a cada turno. O insight é o *conversation programming*: definir o controle do fluxo em termos de quem fala com quem. O **MetaGPT**, de Hong et al. (2023), ataca o problema que o chatter entre LLMs cria: em vez de diálogo, agentes trocam *artefatos estruturados* — PRDs, documentos de design, diagramas e especificações de interface — seguindo SOPs (procedimentos operacionais padronizados) que espelham o fluxo de uma empresa de software, do product manager ao QA. A comunicação por documentos em vez de conversa eliminou o *chatter* improdutivo (“Olá! Como vai?”) e melhorou a consistência das entregas.

	perfil	comunicação	analogia
CrewAI	papéis	tarefas + papéis	uma empresa
AutoGen	conversáveis	mensagens	uma reunião
MetaGPT	especialistas	artefatos + SOPs	uma linha de produção
LangGraph	nós de grafo	estado + arestas	uma máquina de estados

O **Semantic Kernel**, da Microsoft, completa o quadro como a opção *enterprise*: orquestração de agentes, plugins e memória voltada para integração com infraestrutura corporativa. Juntos, esses frameworks formam um espectro, não uma competição: quanto mais controle você precisa, mais baixo você desce (grafo com estado explícito); quanto mais autonomia deseja, mais alto sobe (equipes com papéis ou conversas).

Planejamento além do linear

O ReAct estabeleceu o loop, mas o loop linear tem um custo: cada passo depende do anterior, erros acumulam e a trajetória não tem alternativa. A era dos frameworks trouxe três respostas a essa limitação, que o capítulo 10 anunciou e que aqui se materializam.

O **Tree of Thoughts** (ToT), de Yao et al. (2023), transforma raciocínio em *busca sobre uma árvore de pensamentos*: em vez de uma única cadeia, o modelo gera múltiplos pensamentos candidatos em cada passo, avalia cada um e explora deliberadamente os mais promissores — com backtracking quando um caminho falha. O **ReWOO**, de Xu et al. (2023), *desacopla o raciocínio da observação*: em vez de intercalar pensamento-ação-observação (ReAct), um módulo Planner gera o plano inteiro de antemão, um Worker executa as chamadas de ferramenta e um Solver sintetiza a resposta. A economia é enorme — os autores relataram eficiência de tokens até 5× e ganho de acurácia no HotpotQA, além de robustez a falhas de ferramenta — porque o prompt de raciocínio não é repetido a cada iteração. O **LATS** (Language Agent Tree Search), de Zhou et al. (2023/2024), unifica as duas ideias: expande o ReAct em uma *busca em árvore* (Monte Carlo Tree Search), usando o LLM como agente, avaliador de estados e gerador de reflexões, com feedback do ambiente externo e autoavaliação guiando a exploração. Com GPT-4, o LATS alcançou 92,7% pass@1 no HumanEval, dobrou o desempenho do ReAct no HotpotQA e elevou a pontuação média no WebShop.

ReAct (cap. 10)	→ uma trajetória, decide a cada passo	→ simples, frágil
ToT	→ múltiplas trajetórias, avalia e volta	→ raciocínio deliberado
ReWOO	→ planeja antes, executa depois	→ economia de tokens
LATS	→ busca em árvore com feedback externo	→ exploração + ambiente

O padrão comum é a fuga da linearidade: o agente deixa de ser uma cadeia única e passa a explorar um *espaço* de possibilidades, guiado por avaliação e feedback. É o movimento que conecta os frameworks de orquestração (grafos com estado) aos métodos de planejamento (busca em árvore) — e, mais tarde, aos protocolos da Parte VII.

A lição estrutural

No fim, a lição que o capítulo quer deixar é a mesma que organiza este livro: **agente não é um prompt**. Escrever “você é um agente autônomo” no system prompt não cria agência; cria uma expectativa. Agência emerge quando existem objetivo, estado, ações, observações, feedback e critério de término — e a diferença entre os frameworks da era é *qual desses elementos eles tornam explícitos e controláveis*:

Framework	O que torna explícito	O que esconde
AutoGPT / BabyAGI	o loop de execução	o critério de parada, o custo, o controle
LangChain	a sequência de passos	os ciclos, o estado de execução
LangGraph	o estado, os ciclos, a observabilidade	— (pouco abstrai)
CrewAI	os papéis e as tarefas	os detalhes do loop interno
AutoGen	a conversa entre agentes	o custo do chatter
MetaGPT	os artefatos e os SOPs	o estado de execução

O contraste entre AutoGPT e LangGraph é o resumo da era: o primeiro provou que o loop é possível; o segundo tornou o loop *previsível*. E é essa previsibilidade — estado explícito, observabilidade, guardrails, critérios de término — que separa uma técnica de laboratório de uma infraestrutura utilizável. O capítulo 14 explorará o passo seguinte: quando o agente não é um, mas muitos, o problema deixa de ser o loop e passa a ser a comunicação.

Para o engenheiro

AutoGPT e BabyAGI provaram o custo do loop sem critério de parada. Antes de qualquer “agente autônomo”, defina explicitamente objetivo, estado, ações, observações, feedback e condição de término — os seis componentes da estrutura que este capítulo apresentou. Se você não sabe dizer o que conta como sucesso, não tem como saber quando parar.

Framework não é mágica. Escolha por **observabilidade, testabilidade e controle de estado**, não pelo hype da semana. Se o fluxo for majoritariamente linear, uma cadeia simples resolve; se houver loops, ramos condicionais ou retomada após falha, um grafo de estados (como o LangGraph) justifica a complexidade. A regra prática: suba de abstração quando quiser velocidade, desça quando precisar de controle.

Em produção, `max_iterations`, orçamento de tokens, timeout e logs estruturados são obrigatórios. O mesmo agente que funciona em dev pode custar dez vezes mais em prod — a diferença não é o modelo, é o loop que nunca termina. Instrumente desde o início: rastreie cada transição de estado e cada chamada de ferramenta; sem isso, o debug de um agente é adivinhação.

Separe **memória de sessão** (o estado da conversa atual) de **conhecimento de longo prazo** (o banco, o índice, o perfil persistido). Misturar os dois é a causa clássica de agentes “que se esquecem” ou que “inventam” contexto. E quando a tarefa permitir, prefira a troca de artefatos estruturados ao diálogo aberto: a lição do MetaGPT vale fora dele — comunicação por documentos é mais barata e menos ruidosa que chatter.

Finalmente, desconfie de autonomia sem verificação. Reflexão do próprio modelo (Reflexion, LATS) melhora, mas não substitui validadores externos: testes, execução de código e avaliação objetiva. O loop explora; o verificador decide. Essa é a separação generator/verifier que o capítulo 10 introduziu e que os capítulos 14 e 15 levarão ao limite.

Fontes: [Richards, 2023] — AutoGPT; [Nakajima, 2023] — BabyAGI; [Wang et al., 2023/2024] — survey de agentes autônomos (perfil/memória/planejamento/ação); [Wu et al., 2023] — AutoGen; [Hong et al., 2023] — MetaGPT; [Yao et al., 2023] — Tree of Thoughts; [Xu et al., 2023] — ReWOO; [Zhou et al., 2023/2024] — LATS; [Masterman et al., 2024] — panorama de arquiteturas de agentes; [Agentic AI Frameworks, 2025] — comparação CrewAI/LangGraph/AutoGen/ADK/MetaGPT.

Capítulo 14 — Sistemas multi-agente e simulação social

O capítulo 13 fechou com a promessa de um passo seguinte: quando o agente não é um, mas muitos, o problema deixa de ser o loop e passa a ser a comunicação. Este capítulo cumpre a promessa — e começa com uma pergunta ingênua que 2023 transformou em campo de pesquisa: *se um agente é bom, dois são melhores?* A resposta, como quase sempre nesta história, é “depende” — e o mais valioso deste capítulo é entender exatamente de quê.

Em 2023, o campo experimentou com intensidade. O **CAMEL** pôs dois agentes para conversar sobre um problema e mostrou que *role-playing* produz tarefas e diálogos que um agente sozinho não gera. O **Generative Agents** construiu uma vila inteira — vinte e cinco agentes com memórias, reflexões e rotinas diárias — e convenceu observadores de que estava diante de uma sociedade mínima. O **ChatDev** organizou uma “empresa de software” em que papéis como CEO, programador e tester produziam código de ponta a ponta. O **AgentVerse** formalizou o ciclo de recrutamento, decisão, execução e avaliação. A ambição era grande: dividir o trabalho, multiplicar perspectivas e simular comportamento humano.

O problema que apareceu no caminho tem nome — *chatter* — e é a tese deste capítulo: agentes conversando produzem texto, não artefatos, e texto custa tokens sem entregar necessariamente progresso. As lições que sobreviveram — papéis claros, comunicação por artefatos estruturados em vez de mensagens soltas, hierarquia e líderes, filtragem de mensagens — valem mais do que qualquer framework. E o capítulo termina com um gancho que a Parte VII vai desatar: agentes de vendedores diferentes não conversam — daí o A2A, lá na frente.

A conversa como método: CAMEL

O **CAMEL** (Communicative Agents for “Mind” Exploration), de Li et al. (2023), perguntou o que acontece se dois modelos conversam sozinhos sobre um problema, sem humano no meio. A resposta técnica foi o *role-playing* — o *inception prompting*: cada agente recebe uma identidade (por exemplo, um *AI assistant* e um *AI user*), e o assistente interpreta a instrução do usuário, executando uma cadeia de tarefas que o próprio diálogo vai desdobrando. É o loop do capítulo 10 (thought → action → observation), mas agora a “observação” é a resposta do outro agente — o ambiente de um agente é o outro agente.

```
AI user → "Desenvolva um bot de trading para ações."
AI assistant→ "Entendido. Primeiro, defino a estratégia. Você concorda?"
AI user → "Sim. Prossiga com a coleta de dados."
AI assistant→ "Feito. Agora vou escolher as bibliotecas de análise."
...
```

O resultado notável foi duplo. Primeiro, o *role-playing* suprimiu parte da alucinação: um modelo que divaga quando responde sozinho tende a se manter na tarefa quando interpreta um papel diante de outro agente — a estrutura social funciona como um trilho. Segundo, o esquema gerou *dados*: pares de instrução-resposta sintéticos, produzidos por agentes, para domínios arbitrários — um laboratório de geração automática de treino. O paper registrou com honestidade os limites: sem supervisão, o diálogo pode divergir, esquecer o objetivo e repetir-se. Em outras palavras, o CAMEL já anunciava o problema central da era — conversa não é progresso — antes mesmo que ele tivesse nome.

A vila que acordou: Generative Agents

Se o CAMEL provou que dois agentes conversam, o **Generative Agents**, de Park et al. (2023), provou que vinte e cinco agentes podem parecer uma sociedade. Na vila fictícia de Smallville, cada agente tem uma identidade (profissão, família, rotina), uma **memory stream** — um registro temporal de todas as observações em linguagem natural — e três mecanismos: **retrieval** (recupera memórias pela combinação de recência, importância e relevância), **reflection** (em momentos de calma, sintetiza memórias em percepções de nível mais alto) e **planning** (decompõe o dia em planos e ações, re-planejando quando as circunstâncias mudam).

O que convenceu não foi a arquitetura, foi o comportamento emergente que ela permitiu. Isabella, a agente que quer sediar uma festa para convidar Sam, divulga a data para amigas; elas contam para mais gente; e, em uma tarde, a informação sobre a festa se espalhou pela vila inteira — sem nenhum script global, apenas agentes trocando observações e atualizando memórias. Dois agentes que não se conheciam se “encontraram” no café porque o plano de um os levou até lá, e conversaram sobre tópicos que a memória de cada um tornava relevantes.

```
observação → memory stream (timestamp)
retrieval → recência + importância + relevância
reflection → percepções de alto nível a partir das memórias
planning → planos diários que geram ações observáveis
```

A avaliação foi um marco metodológico: os pesquisadores entrevistaram os agentes e pediram a avaliadores humanos que julgassem o quanto o comportamento correspondia a uma pessoa fictícia. Os agentes foram julgados mais críveis do que a linha de base sem o sistema de memória. A lição estrutural não era “construímos gente” — era que **memória + reflexão + plano** bastam para sustentar comportamento consistente ao longo do tempo, e que sociedades mínimas produzem fenômenos (difusão de informação, coordenação) que um agente isolado não exhibe. Essa é a base de tudo que hoje se chama “cidade de agentes”.

A empresa que produz software: ChatDev

O **ChatDev**, de Qian et al. (2023), trocou a vila pela fábrica: uma “empresa de software” virtual em que agentes com papéis de uma companhia real — CEO, CTO, programador, revisor, tester — desenvolvem um programa de ponta a ponta por meio de um **chat chain**, uma cadeia de conversas organizada nas fases de um modelo em cascata: design, codificação, testes e documentação.

Duas ideias técnicas merecem destaque porque respondem diretamente ao problema do *chatter*. A primeira é o *instruct-then-chat*: cada conversa começa com uma instrução rígida e só então vira diálogo — a fase define o limite, impedindo que a conversa divague para fora da etapa. A segunda é a **communicative dehallucination**: dois papéis se verificam mutuamente — o programador afirma o que implementou, o revisor testa a afirmação — e o par resolve discrepâncias antes de seguir. É a separação generator/verifier do capítulo 10, agora distribuída entre dois agentes: um agente confirma o que o outro afirma, em vez de o modelo se auto-avaliar. O ChatDev produziu software completo — incluindo o jogo do “snake” — com papéis coordenados e correções internas, mostrando que a *divisão do trabalho* é a primeira justificativa legítima para múltiplos agentes.

Recrutar, decidir, executar e avaliar: AgentVerse

O **AgentVerse**, de Chen et al. (2023), organizou o ciclo que os trabalhos anteriores executavam de forma ad hoc. Um problema é resolvido em quatro fases:

- | | |
|----------------------------------|--|
| 1. recrutamento de especialistas | → escolher os agentes certos para a tarefa |
| 2. decisão colaborativa | → os agentes discutem, propõem e votam |
| 3. execução da ação | → o plano vira ação, com um agente líder |
| 4. avaliação | → os resultados são avaliados; se falhar, volta à fase 2 |

O valor do esquema é tornar *explícito* o que era implícito: a seleção de quem participa (fase 1), a necessidade de um mecanismo de convergência — o voto, o consenso — em vez de simples acúmulo de mensagens (fase 2), e o critério de término (fase 4), o componente que o capítulo 13 mostrou ser o mais negligenciado. O AgentVerse também apontou a segunda justificativa para múltiplos agentes, além da divisão de trabalho: **perspectivas distintas** — um consenso de agentes com visões diferentes pode superar a resposta de um agente único, exatamente porque discordam. E observou comportamentos emergentes nas dinâmicas: especialização de papéis, transferência de informação entre participantes e *mentoring* — agentes experientes guiando novatos.

Chatter: o custo invisível

Nenhum desses sistemas escondeu a dor que os acompanhava, e a literatura de surveys da era a classificou: **chatter**. A palavra descreve o modo de falha no qual os agentes produzem cada vez mais mensagens e cada vez menos resultado — conversa sem artefato, sem decisão e sem término.

As causas são estruturais, não acidentais. Cada mensagem entre agentes custa tokens — e o custo *composto* importa mais que o unitário: em um grupo de N agentes que todos falam com todos, cada turno amplifica o contexto de todos os participantes, e o custo da conversa cresce mais que o valor do

que se diz. Sem um mecanismo de convergência (um voto, um líder, um artefato que “fecha” a decisão), o diálogo roda sem um ponto de chegada. E sem uma âncora externa, o objetivo original se perde na deriva — a mesma deriva do AutoGPT (capítulo 13), agora multiplicada por N. O Masterman et al. (2024), no panorama de arquiteturas, documentou a resposta empírica: estruturas com **liderança** — um agente que decide, em vez de todos votarem — e com **filtragem de mensagens** — nem tudo que um agente diz chega a todos os outros — reduzem o ruído e melhoram a robustez, ao custo de menos “democracia”. E o mesmo panorama anotou a lição mais desconfortável: para muitas tarefas, um agente único bem projetado vence dois ou mais agentes — a coordenação é cara demais para o ganho.

De conversas a artefatos: MetaGPT

A resposta mais influente ao *chatter* veio do **MetaGPT**, de Hong et al. (2023), que o capítulo 13 apresentou: trocar mensagens por **artefatos estruturados**. A empresa de software virtual do MetaGPT não conversa para decidir — ela produz documentos: o product manager emite o PRD, o arquiteto emite o documento de design, o engenheiro escreve o código, o QA escreve os testes. O fluxo segue **SOPs** — procedimentos operacionais padronizados, modelados a partir de empresas reais de software — e a comunicação usa um padrão **publish-subscribe**: cada artefato é publicado em um lugar onde os interessados podem assinar e consumir, em vez de ser empurrado em uma conversa.

PM → PRD	(requisitos)	}	cada papel publica um artefato quem precisa, assina e consome
arquiteto → design	(diagramas)		
engenheiro → código			
QA → testes	(verificação)		

A consequência é que a informação não trafega como texto efêmero, mas como estado durável que qualquer papel pode consultar a qualquer momento — a comunicação vira *memória de processo*, no sentido do capítulo 12. O *chatter* (“Ok, entendi! Vamos nessa!”) some porque a entrega é um documento com schema, não uma mensagem. O custo é a rigidez: nem todo problema tem um SOP óbvio, e a automação de processos só paga quando o processo realmente existe. Mas a lição — comunicação por artefatos estruturados rende mais que diálogo aberto — escapou do MetaGPT e virou um princípio de desenho para toda a era.

A aritmética da coordenação

O survey de Guo et al. (2024) organizou a maturidade da área, e uma conta simples resume a dificuldade econômica do multi-agente. Em um grupo totalmente conectado, N agentes produzem $O(N^2)$ pares de conversas potenciais. Cada par consome tokens em duas direções, cada participante carrega o contexto dos outros, e cada agente adicional aumenta a superfície de falha — mais mensagens, mais deriva, mais custo. A conta não diz “não use múltiplos agentes”; ela diz **por que**:

divisão de trabalho real	→	tarefa decompõe-se naturalmente em subtarefas com experts distintos
perspectivas distintas	→	o valor está na discordância, não na soma de respostas iguais
heterogeneidade	→	sistemas, vendedores ou objetivos diferentes precisam negociar

Quando nenhuma das três condições existe, o multi-agente é um custo sem benefício: a conversa é a forma mais cara de chegar à resposta que um agente único daria. Quando pelo menos uma existe, o problema muda de “quantos agentes” para “como se comunicam” — o que conecta diretamente ao próximo estágio da história. Agentes que usam frameworks e vendedores diferentes não conversam por padrão: cada par exige integração dedicada, e o problema $N \times M$ reaparece em uma escala nova. A saída não será mais um framework — será um **protocolo** entre agentes, o tema do capítulo 17.

A lição estrutural

A era multi-agente de 2023–2024 deixou um vocabulário que sobreviveu intacto. Papéis: agente sem identidade clara é só um loop sem direção — o *role* define o que ele pode e deve fazer. Comunicação: o canal importa menos que o *conteúdo* — artefatos estruturados com schema vencem mensagens soltas, porque podem ser validados, versionados e consumidos por outros sistemas. Coordenação: sem um mecanismo de convergência — voto, líder, artefato final — o grupo conversa sem decidir. E custo: cada agente adicional é um multiplicador de contexto, tokens e falhas; a pergunta certa não é “quantos agentes” mas “qual tarefa genuinamente se beneficia de mais de um”.

A tabela que fecha o capítulo 13 pode ser estendida com o que esta era acrescentou:

Sistema	Unidade de organização	Como comunica	O que entrega
CAMEL	dois papéis	diálogo instrutor-executor	dados e tarefas sintéticos
Generative Agents	memória individual	observações no espaço compartilhado	comportamento plausível
ChatDev	papéis de empresa	cadeia de fases (chat chain)	software completo
AgentVerse	ciclo de fases	discussão + voto	decisão coletiva
MetaGPT	SOPs	publish-subscribe de artefatos	documentos e código

O fio comum é a resposta à pergunta inicial: dois agentes não são melhores por serem dois — são melhores quando o problema se decompõe, quando as perspectivas divergem ou quando os mundos são heterogêneos. E é exatamente essa heterogeneidade que empurra a próxima era: quando os agentes viram *sistemas* que precisam cooperar sem compartilhar código, a conversa não pode ser resolvida por framework — só por protocolo. A Parte VII começa aqui.

Para o engenheiro

Múltiplos agentes valem a pena apenas quando existe divisão de trabalho real ou perspectivas genuinamente distintas — nunca por padrão. Antes de arquitetar um “time” de agentes, responda: esta tarefa se decompõe em subtarefas com experts diferentes? A discordância entre modelos agregaria valor? Se as duas respostas são não, um único loop bem projetado (capítulo 13) vence dois agentes que se atrapalham — e custa uma fração.

O inimigo número um é o *chatter*: agentes que conversam sem produzir artefato. Troque mensagens soltas por comunicação estruturada — arquivos, schemas, tickets, documentos de interface. Defina o que conta como “entrega” para cada papel, e faça a conversa avançar na direção da entrega. Se você não sabe qual é o artefato final, o sistema não tem critério de parada.

Papéis e hierarquia não são burocracia, são robustez. Defina quem decide, quem executa e como um resultado é passado adiante; um líder que filtra mensagens e decide reduz o ruído mais do que uma votação entre pares. Comece com dois agentes e escale apenas quando latência e custo justificarem — cada agente extra soma contexto, tokens e superfície de falha ao sistema inteiro.

E quando os agentes vierem de times, sistemas ou vendedores diferentes, não tente fazê-los conversar por integração ponto a ponto. Esse é o problema que nenhum framework resolve — e é exatamente o que os protocolos da Parte VII endereçam.

Fontes: [Li et al., 2023] — CAMEL; [Park et al., 2023] — Generative Agents; [Qian et al., 2023] — ChatDev; [Chen et al., 2023] — AgentVerse; [Hong et al., 2023] — MetaGPT; [Masterman et al., 2024] — panorama de arquiteturas single/multi-agente; [Guo et al., 2024] — survey de multi-agentes (IJCAI).

Capítulo 15 — Avaliação de agentes: o que medir e por que é difícil

O capítulo 13 mostrou que agente é sistema, não prompt; o capítulo 14 mostrou que sistemas conversam entre si. Resta a pergunta que transforma técnica em produto: *como saber que o agente funciona?* Medir respostas foi a era anterior — “acertou a saída?” —; esta era é sobre medir **processos**: planejamento, seleção de ferramenta, recuperação, execução, custo e recuperação de falhas. E medir processo é muito mais difícil do que parece.

Em 2023-2024, a avaliação de agentes passou por três saltos. O **AgentBench** colocou dezenas de modelos em oito ambientes e revelou, com números, a distância entre os melhores modelos fechados e todo o resto. O **WebArena** e o **OSWorld** trocaram ambientes sintéticos por tarefas realistas — sites de verdade, sistemas operacionais de verdade. E o **tau-bench** introduziu o dado mais desconfortável da era: modelos fortes falham em mais da metade das tarefas, e a *consistência entre tentativas* é pior do que a precisão — um agente pode acertar na segunda tentativa o que errou na primeira, e nenhum benchmark tradicional captura isso.

A pergunta que organiza este capítulo é uma só: **uma resposta certa por acaso não é um sistema robusto**. Ela tem duas consequências práticas. A primeira: avaliar agente exige critérios de processo — e não apenas de resposta. A segunda: as ferramentas baratas de avaliação, como o LLM-as-judge, são tentadoras e viesadas — e confundi-las com validação independente é exatamente o erro que o AGENTS.md (§25) deste projeto avisa para evitar.

O laboratório dos oito ambientes: AgentBench

O **AgentBench**, de Liu et al. (2023), foi a primeira tentativa sistemática de medir “quão bom agente é um modelo” — e não apenas “quão bom é em responder perguntas”. A definição de tarefa era ampla: o agente recebe um ambiente e um objetivo, e precisa *agir* para alcançá-lo. Foram oito ambientes: sistema operacional (comandos em um terminal), banco de dados (consultas SQL), grafo de conhecimento (consultas sobre uma base de fatos), jogos de cartas digitais, quebra-cabeças de pensamento lateral, tarefas domésticas simuladas, compras na web e navegação em páginas da web.

Os resultados expuseram a hierarquia da época de forma brutal. O GPT-4 liderava com folga, mas mesmo ele ficava longe de dominar os ambientes; os modelos abertos — a família LLaMA 2, naquele momento o melhor disponível — caíam de forma abrupta, com uma margem que as comparações de QA tradicionais não mostravam. A conclusão diagnóstica de Liu et al. foi qualitativa e durável: o gargalo dos modelos não era vocabulário nem conhecimento, mas **raciocínio de longo prazo, tomada de decisão e adesão a instruções** — exatamente as capacidades que um agente precisa em cada loop. O AgentBench estabeleceu o formato da era: ambientes múltiplos, modelos múltiplos, e uma lição de que “modelo forte” e “agente bom” são coisas distintas.

A internet e o desktop de verdade: WebArena e OSWorld

O AgentBench abriu o caminho, mas seus ambientes eram, em larga medida, construídos. O passo seguinte foi levar o agente para ambientes *realistas*. O **WebArena** (Zhou et al., 2023) replica a experiência de navegar na web: quatro sites autocontidos — um fórum no estilo Reddit, um repositório de código no estilo GitLab, uma loja no estilo Amazon e um CMS no estilo WordPress — sobre os quais são definidas 812 tarefas que exigem planejar, navegar e executar ações reais (editar um perfil, fazer uma compra, publicar um post). O **OSWorld** (Xie et al., 2024) vai mais fundo: 369 tarefas em sistemas operacionais de verdade (Ubuntu, Windows, macOS), em que o agente interage pela tela inteira — capturas de tela e movimentos de mouse e teclado — como um usuário humano, sem API nem assistência.

Os resultados foram humilhantes no bom sentido: mesmo os melhores agentes autônomos ficavam na casa dos 10-20% de sucesso no OSWorld, e o WebArena, com os modelos de topo da época, raramente passava da casa dos 30% em abordagens gerais. Duas leituras se impuseram. A primeira: ambientes

realistas punem implacavelmente a *fragilidade* — um clique errado, uma janela modal inesperada, uma mudança de layout derrubam trajetórias que funcionavam em ambientes limpos. A segunda: esses benchmarks viraram o padrão de facto da área — quem quer dizer “meu agente funciona” hoje é medido contra eles —, e continuam sendo o ponto de partida de qualquer suíte de avaliação, nunca o ponto de chegada.

O usuário que responde: tau-bench e a consistência

O **tau-bench**, de Yao et al. (2024), fez a pergunta que faltava: *o que acontece quando o usuário também é simulado?* Em dois domínios de atendimento — uma loja de varejo e uma companhia aérea —, um **usuário simulado** conversa com o agente, que só pode responder por meio de ferramentas (consulta de catálogo, reserva, cancelamento, política de reembolso), enquanto uma **política de domínio** define quais estados são válidos e o que conta como sucesso. O agente precisa, literalmente, completar a tarefa no mundo — como um funcionário real — e não apenas dizer a coisa certa.

O tau-bench introduziu uma métrica que se tornou central para toda a avaliação de agentes: o **pass@k**. A definição é implícita no nome — k tentativas independentes da mesma tarefa —, mas a leitura correta é a mais importante deste capítulo: $\text{pass}@k$ não mede “acertou de primeira”, mede **consistência**. Um agente que acerta 1 de 3 execuções tem $\text{pass}@1 = 33\%$ e $\text{pass}@3 = 100\%$ se qualquer uma das tentativas contar — mas o tau-bench computa a fração das tentativas que *realmente* têm sucesso, e é essa fração que revela o problema.

Os resultados confirmaram a intuição com números: modelos fortes falhavam em mais da metade das tarefas, e — o dado decisivo — a **consistência entre execuções era pior do que a precisão**. O mesmo modelo, a mesma tarefa, o mesmo prompt: uma execução completa a tarefa, a seguinte se perde em um desvio. Para o engenheiro, a consequência não pode ser subestimada: nenhum benchmark de “acurácia média” captura a diferença entre um sistema que acerta consistentemente e um que acerta por acaso. A consistência, e não a precisão, é o que separa uma demo de um produto.

Métricas de processo, não só de resposta

Com o campo maduro, a revisão *From Language to Action* (2025) organizou o que a avaliação de agentes realmente precisa medir. A mudança de mentalidade é de granularidade: um agente não é uma função input→output, é uma sequência de estados, ações e observações — e cada elo da cadeia tem a sua métrica:

Componente	Pergunta que responde	Métrica típica
Retrieval	a evidência certa chegou?	$\text{recall}@k$, fidelidade da fonte
Tool selection	a ferramenta certa foi escolhida?	acurácia da chamada, taxa de erro
Planning	o plano conduziu ao objetivo?	completude do plano, passos por tarefa
Execution	a ação modificou o mundo como esperado?	sucesso da execução, efeitos colaterais
Verification	o resultado foi checado antes de responder?	taxa de correção após feedback
Failure recovery	o sistema se recupera de um erro?	taxa de retomada, passos de correção
Cost / latency	quanto o processo custou?	tokens, chamadas, tempo por tarefa

A regra de ouro é a mesma do capítulo 12, transposta para agentes: **cada estágio tem a sua métrica, e o número agregado não diz onde está a falha**. Se a resposta final está errada, pode ser um retriever ruim, uma ferramenta errada, um plano frágil ou uma verificação ausente — e um score médio não discrimina nenhum deles. Para o diagnóstico, é preciso observar a trajetória inteira: quais ações foram tomadas, quais observações voltaram, onde o agente desviou. É a diferença entre avaliar a resposta e auditar o processo.

LLM-as-judge: barato, viesado, inevitável

Quando a resposta certa não existe — uma resposta bem escrita, um resumo útil, um plano bom —, a avaliação automática precisa de um julgador. O padrão da era é o **LLM-as-judge**: um modelo lê a resposta e atribui uma nota, com ou sem rubrica. É barato, escalável e alinhado com a preferência

humana em uma variedade de tarefas — e por isso virou a ferramenta padrão de avaliação de LLMs.

Mas o LLM-as-judge não é **validação independente**. O julgador é a mesma espécie de sistema que produz a resposta: tem os mesmos vieses, as mesmas preferências de posição e verbosidade, a mesma tendência de preferir respostas parecidas com as suas. Aplicado a avaliação de *processo* — julgar se o agente escolheu a ferramenta certa, planejou bem, verificou —, o viés do julgador se soma ao problema de não existir “resposta certa”. A disciplina que este capítulo recomenda é a do AGENTS.md (§25): use LLM-as-judge como *primeira triagem* — rubricas explícitas, escalas definidas, calibração periódica contra uma amostra de julgamento humano — e jamais como tribunal final dos casos críticos. A regra não muda porque a ferramenta é conveniente: **o gerador não valida a si mesmo** — e um modelo julgando outro modelo é, no limite, o mesmo gerador avaliando-se através de um espelho.

A lição estrutural

O capítulo 13 definiu o critério de término como o componente mais negligenciado de um agente: *quando parar?* Este capítulo definiu o critério de avaliação: *como saber que funcionou?* A lição estrutural é que são **a mesma pergunta vista por dois ângulos**. Para saber quando parar, o sistema precisa saber o que conta como sucesso; para saber se funcionou, precisa do mesmo critério. Um agente sem objetivo verificável não tem como parar nem como ser avaliado — ele apenas termina quando o orçamento acaba.

Isso resolve também o mistério do tau-bench: a consistência é o elo fraco porque o critério de sucesso, em tarefas de agente, é um *estado do mundo* — a tarefa foi completada? —, e alcançar esse estado exige uma cadeia de decisões corretas, qualquer uma das quais pode falhar. A avaliação de agentes é, portanto, a engenharia do critério de término, medida em escala: definir o objetivo, decompor o processo em estágios mensuráveis e verificar, repetidamente, que o sistema alcança o estado desejado — não uma vez, mas de forma consistente.

Para o engenheiro

Avalie o processo, não apenas a resposta. Planejamento, seleção de ferramenta, recuperação, execução e recuperação de falha são exatamente o que você vai depurar em produção — se não medir cada estágio, o debug de um agente quebrado é adivinhação. Trace cada trajetória: ações, observações, desvios. Um score médio diz que algo está errado; só a trajetória diz onde.

Consistência importa mais que acerto isolado. Um agente que acerta 1 em 3 execuções não é confiável, por mais forte que seja o modelo por baixo — meça pass@k e a variabilidade entre execuções da mesma tarefa. Se a sua suíte não roda cada caso múltiplas vezes, você está medindo a sorte, não o sistema.

Benchmarks públicos — AgentBench, WebArena, OSWorld, tau-bench — são pontos de partida, não contratos. Use-os para calibrar o que é possível e para treinar o seu vocabulário de avaliação; mas o que decide o seu produto é uma suíte própria de tarefas reais, com critérios explícitos de sucesso, ranhura de estado do mundo, e executada repetidamente.

E quando a “resposta certa” não existir, use LLM-as-judge com disciplina: rubricas explícitas, escalas definidas, amostra periódica de calibração humana — e checagem manual dos casos críticos. Barato não é sinônimo de independente. Lembre-se: a pergunta de avaliação e a pergunta de término são a mesma — defina o objetivo, decomponha o processo e meça a consistência com que o sistema chega lá.

Fontes: [Liu et al., 2023] — AgentBench; [Yao et al., 2024] — tau-bench; [Masterman et al., 2024] — métricas objetivas vs. subjetivas; [From Language to Action, 2025] — revisão de benchmarks de agentes; [Agentic AI Frameworks, 2025] — guardrails e avaliação em frameworks.

Parte VII — Era dos protocolos (2024-2026)

Capítulo 16 — MCP: o USB-C dos dados e ferramentas

Chegou a hora em que a infraestrutura virou o produto. O capítulo 11 industrializou o function calling: as APIs passaram a declarar funções, os modelos passaram a chamá-las e cada aplicação passou a integrar-se com cada fonte de dados. Foi aí que o problema virou matemático. Se existem N aplicações e M ferramentas, a arquitetura ponto a ponto exige $N \times M$ integrações dedicadas — um inferno de manutenção que só cresce: cada nova aplicação multiplica o problema por M , cada nova ferramenta o multiplica por N .

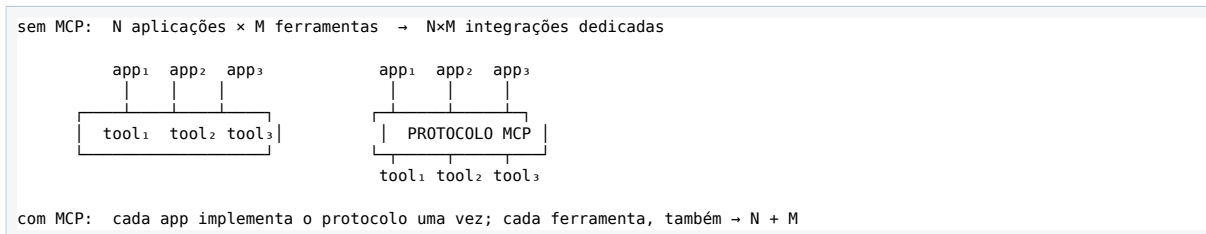
Em 25 de novembro de 2024, a **Anthropic** anunciou e open-sourceou o **Model Context Protocol (MCP)**: uma camada única entre modelos e dados/ferramentas, no lugar das integrações ponto a ponto — $N + M$, em vez de $N \times M$. A analogia com o **USB-C da IA**, que acompanhou o lançamento, captura a ambição com precisão: um padrão de conector, milhares de dispositivos, qualquer cabo funciona em qualquer porta. Em pouco mais de um ano, o MCP foi adotado por OpenAI, Google, Microsoft e AWS, ultrapassou 10.000 servidores públicos e 97 milhões de downloads de SDK por mês, e entrou para a governança da Linux Foundation.

Este capítulo mostra as primitivas do protocolo — tools, resources e prompts —, a arquitetura cliente-servidor que as organiza, a conta do sucesso e os limites que ninguém deveria ignorar: o MCP é *stateless*, não traz memória duradoura nem observabilidade por padrão, e a governança — inicialmente concentrada na Anthropic — tornou-se o ponto de tensão que a própria história do protocolo teve de resolver.

O problema das $N \times M$ integrações

Antes do MCP, cada aplicação conectava-se a cada ferramenta de forma artesanal. Um cliente conversacional que quisesse consultar um banco, ler o Slack, buscar no GitHub e olhar um calendário escrevia quatro integrações dedicadas — e o código de cada uma morria junto com o fornecedor. O capítulo 11 terminou com uma observação que agora se cobra: a camada de ferramentas tornou-se *commodity em evolução*, e a interface padronizada passou a ser pré-requisito, não diferencial.

O MCP formaliza essa interface. No lugar de integrações ponto a ponto, há uma camada única com dois papéis: o **cliente** (o host — a aplicação que hospeda o modelo) e o **servidor** (o adaptador que expõe uma ferramenta, um banco de dados ou um serviço). A comunicação usa **JSON-RPC** — o mesmo formato que já organiza os protocolos da web —, e o transporte funciona tanto para processos locais (stdio, entre o host e o servidor rodando na mesma máquina) quanto para servidores remotos (HTTP, a partir das revisões de 2025).



A conta é a tese econômica do protocolo. N aplicações $\times$ M fontes vira N adaptadores de cliente + M servidores de ferramenta — e, decisivamente, qualquer cliente passa a consumir qualquer servidor sem integração nova. O padrão ganha quando reduz manutenção real; e é por isso que a adoção do MCP foi a mais rápida da história recente de protocolos de desenvolvedor.

As primitivas: tools, resources e prompts

O MCP organiza o mundo em três primitivas — e quase toda a discussão sobre “o que dá para fazer com MCP” é uma discussão sobre qual primitiva usar:

Primitiva	O que é	Papel no sistema
Tools	ações executáveis, com schema de entrada	o modelo pode invocá-las para modificar o mundo
Resources	dados legíveis, identificados por URI	o modelo (ou o cliente) consulta informação
Prompts	templates de instrução reutilizáveis	o modelo recebe receitas prontas de uso

A divisão é a do capítulo 11 levada ao protocolo: **tools** são as funções do function calling (chamar uma API, executar uma ação); **resources** são as fontes de contexto que o RAG do capítulo 12 buscaria (um arquivo, uma tabela, um documento); **prompts** são os exemplos e instruções que moldam a chamada. O **cliente** orquestra — o host decide quais servidores carregar, quais tools expor ao modelo e como enquadrar o contexto —; o **servidor** expõe. É o mesmo desenho de periférico e porta que a analogia do USB-C sugere: o host não conhece os detalhes de cada dispositivo, apenas o protocolo.

Os primeiros servidores de referência da Anthropic — Slack, GitHub, Postgres, Google Drive — mostraram o escopo: ferramentas de colaboração, repositórios de código, bancos de dados e armazenamento de arquivos, todos atrás da mesma interface. Em seguida, a adoção explodiu: o MCP passou a ser suportado nativamente por ChatGPT, Claude, Cursor, Gemini, Microsoft Copilot e VS Code, entre outros, e por infraestruturas enterprise da AWS, Google Cloud e Azure.

Adoção relâmpago e governança

O ritmo de adoção do MCP não tem precedentes comparável na história de protocolos de infraestrutura. Em um ano:

```

nov/2024  lançamento e open-source pela Anthropic
mar/2025  OpenAI adota (ChatGPT + API)
2025      Google (Gemini), Microsoft (Copilot) e AWS anunciam suporte nativo
2025      revisões da spec em 2025-03-26, 2025-06-18 e 2025-11-25
dez/2025  ~97M downloads de SDK/mês; 10.000+ servidores públicos ativos
dez/2025  Anthropic doa o MCP à Linux Foundation (Agentic AI Foundation)

```

As revisões da especificação de 2025 (que a cronologia deste livro registra em 2025-03-26, 2025-06-18 e 2025-11-25) refinaram o núcleo: operações assíncronas, statelessness explícito, identidade de servidor e extensões oficiais — as peças que faltavam para uso enterprise. O marco de dezembro de 2025 resolveu a tensão que o ecossistema vinha apontando: a **governança** do protocolo, inicialmente concentrada na Anthropic, passou para a **Linux Foundation**, sob a recém-criada **Agentic AI Foundation**, com a Anthropic, a Block e a OpenAI como co-fundadoras e AWS, Google, Microsoft, Bloomberg e Cloudflare entre os membros fundadores. O mesmo movimento que o capítulo 17 descreve para o A2A — doação a governança neutra — repetiu-se para o MCP, com a promessa de que o padrão não pertencerá a nenhum fornecedor.

MCP e agentes: a camada vertical do mundo

Uma aplicação que a adoção revelou é que o MCP não conecta apenas *modelos* a ferramentas — conecta **agentes** a ferramentas e, indiretamente, agentes a agentes. Um agente (capítulo 13) precisa de tools para agir; o MCP entrega a interface padronizada para essas tools, independentemente de qual modelo ou framework as orquestra. A AWS demonstrou, já em maio de 2025, comunicação entre agentes construída sobre MCP — dois agentes trocando informações através da mesma camada cliente-servidor.

É útil fixar o mapa mental que o capítulo 17 vai completar: **o MCP é a camada vertical** — conecta o modelo ao mundo (ferramentas, dados, serviços). O que ele não faz é conectar dois agentes entre si com descoberta de capacidades, autenticação e troca de tarefas — esse é o espaço horizontal que o A2A ocupa. Um agente pode, perfeitamente, usar MCP para buscar dados e A2A para conversar com outro agente. São camadas complementares, não concorrentes.

Os limites que ninguém deveria ignorar

A mesma análise que explica o sucesso explica as fronteiras. O protocolo é **stateless**: cada conexão entre cliente e servidor não carrega memória entre sessões — não há, por padrão, histórico, estado de conversa nem memória duradoura. Quem precisa de persistência — e todo sistema em produção precisa — tem de construí-la fora do protocolo: banco, cache, camada de memória, pipeline de eventos. O MCP entrega o transporte; o estado é seu.

O segundo limite é a **observabilidade**. O protocolo não define, por padrão, linhagem completa das ações nem rastreamento uniforme das chamadas entre cliente e servidor. Em produção, isso significa camadas externas de monitoramento e auditoria — especialmente quando as chamadas de ferramentas têm efeitos no mundo (escrever, enviar, pagar). O terceiro é a **governança**: concentrada na Anthropic até dezembro de 2025, e só então aberta à Linux Foundation — um período em que a dependência de um único fornecedor foi, com razão, apontada como ponto de tensão.

E há o custo silencioso: protocolos “gastam tokens para falar” — cada primitiva trocada entre cliente e servidor consome contexto e latência, e a soma disso em sistemas com muitos servidores é real.

A lição estrutural

A lição é a do capítulo 11, agora elevada à infraestrutura: **a interface padronizada é pré-requisito, e o valor está no que você constrói sobre ela** — o estado, a memória, a observabilidade e a verificação continuam sendo responsabilidade do sistema. O MCP transformou a integração ponto a ponto em protocolo, mas o protocolo entrega o transporte, não o sistema: stateless e sem observabilidade por padrão, ele devolve ao engenheiro exatamente as camadas que um sistema de produção não pode dispensar. É a aritmética do capítulo em outra moeda — o padrão ganha quando reduz manutenção real, e por isso a adoção do MCP foi a mais rápida da história recente de protocolos de desenvolvedor. E a mesma forma se repetirá no capítulo 17: quando o mundo que o agente acessa é outro agente, a padronização continua sendo o pré-requisito — mas o valor continua estando no que se constrói sobre ela.

Para o engenheiro

MCP elimina o inferno das integrações $N \times M$ — use-o. Exponha suas APIs internas uma vez como servidor MCP e qualquer cliente compatível as consome; no sentido contrário, consuma as ferramentas do ecossistema sem integração dedicada. A conta é a do capítulo: $N + M$ em vez de $N \times M$. Se a sua integração se repete para múltiplos clientes, o servidor MCP já se pagou.

Modele o domínio nas três primitivas: tools para ações, resources para dados, prompts para receitas prontas. Se algo não cabe em nenhuma das três, provavelmente não precisa de MCP — o protocolo simplifica a integração, não a substitui. E comece pequeno: um servidor para um par de ferramentas internas vale mais que adotar o ecossistema inteiro de uma vez.

Não espere que o transporte resolva estado. O protocolo é stateless: persistência, memória, cache e observabilidade são suas — banco, pipeline de eventos, monitoramento. Trate cada chamada de tool como um efeito observável que precisa de log e, quando o efeito for no mundo, de validação antes e auditoria depois.

E acompanhe a governança de perto. A doação à Linux Foundation reduziu o risco de dependência de fornecedor, mas padrões vivos mudam — especificações, revisões e extensões. Construa sobre o protocolo, não sobre SDKs proprietários: é a diferença entre portabilidade e lock-in.

Fontes: [Anthropic, 2024] — Introducing the Model Context Protocol; [MCP Specification] — revisões 2024-2025; [AWS Open Source Blog, 2025] — interoperabilidade entre agentes sobre MCP; [InfoWorld, 2025] — MCP/A2A em produção, stateless e memória; [MIT Tech Review, 2025] — panorama de protocolos; [Taskade, 2026] — métricas de adoção; [Konishi, 2026] — timeline de MCP e tool use.

Capítulo 17 — A2A e a interoperabilidade entre agentes

O capítulo 16 apresentou o MCP como a camada que conecta o modelo ao mundo — ferramentas, dados, serviços. Este capítulo pergunta o passo seguinte: e quando o “mundo” que um agente precisa acessar é *outro agente*? Em 2025, a resposta estava clara para quem construía agentes em escala: agentes feitos por times e vendedores diferentes simplesmente não conversavam. Cada par exigia integração dedicada — o problema $N \times M$ do capítulo 16, agora multiplicado por agentes que se desdobram, se autenticam e coordenam tarefas.

Em 9 de abril de 2025, a **Google** lançou o **Agent2Agent (A2A)** para resolver exatamente isso: um protocolo aberto para agentes descobrirem capacidades uns dos outros, trocarem informação e coordenarem ações — com descoberta via *Agent Card*, autenticação e troca de tarefas em JSON-RPC. Diferente do MCP, que padroniza a conexão *modelo→ferramenta*, o A2A padroniza a conexão *agente→agente*: um agente pode ser construído por qualquer framework, em qualquer linguagem, e ainda assim conversar com qualquer outro agente que fale o mesmo protocolo.

Em junho de 2025, o A2A foi doado à **Linux Foundation**, com AWS, Cisco e Microsoft como participantes fundadores do projeto — o mesmo gesto de governança neutra que o MCP repetiria em dezembro. E a versão 0.3, em agosto de 2025, trouxe o que faltava para o enterprise: transporte gRPC e *security cards* assinados. Este capítulo monta a visão complementar das duas camadas — vertical e horizontal — e termina com a tese da Parte VII: quando os próprios agentes viram sistema, **protocolos são a camada de ferramentas** — o padrão REST/JSON que organizou as APIs da última década se repete, agora entre agentes.

Quando o mundo é outro agente

O MCP resolveu a conexão *modelo↔mundo*, mas o mundo não é feito só de ferramentas — é feito de outros agentes. Um agente de vendas precisa consultar o agente de estoque; um agente de suporte precisa delegar a um agente de faturamento. Na arquitetura ponto a ponto, cada par exigia uma integração escrita à mão: conhecer a API do outro, o formato de autenticação dele, o ciclo de vida das tarefas dele. Com poucos agentes, funciona; com dezenas — cada um em seu sistema, seu vendedor, sua linguagem —, o custo de integração explode exatamente como explodiu o $N \times M$ das ferramentas antes do MCP.

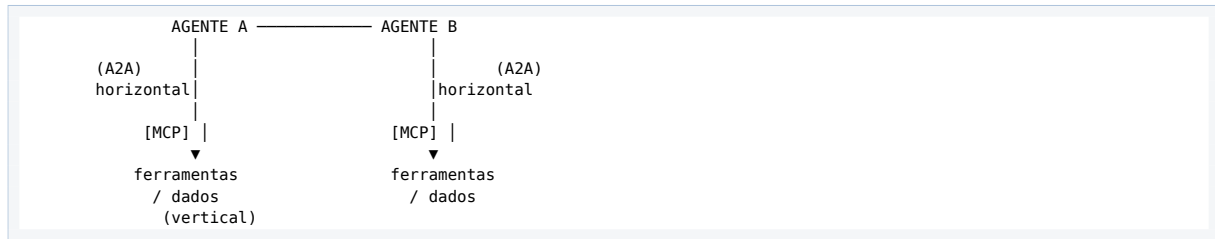
O A2A trata esse problema como o MCP tratou o dele: definindo um protocolo comum, não integrações dedicadas. Três conceitos formam o núcleo. O **Agent Card** é um documento JSON que todo agente publica (em um endpoint padrão, como `/well-known/agent-card.json`) e que descreve sua identidade, suas capacidades, suas *skills*, seu endpoint e os esquemas de autenticação que exige. A **tarefa (task)** é a unidade de trabalho: um agente envia uma tarefa ao outro via JSON-RPC, acompanha o estado (em execução, concluída, cancelada, com falha) e recebe o resultado. O **artefato** é o conteúdo produzido — texto, arquivo, dados estruturados —, o que permite transferir não só mensagens, mas entregas verificáveis.

cliente (agente orquestrador)	servidor (agente de domínio)
1. GET <code>/well-known/agent-card.json</code>	
↳ Agent Card: skills, endpoint, auth	
2. message/send → task (tarefa)	
↳ task em execução	
3. subscribe/stream → progresso	
↳ task concluída + artefato	

A descoberta resolve o problema que nenhum framework resolveu: um agente não precisa saber *de antemão* o que o outro faz — ele lê o Agent Card, encontra a skill que precisa e autentica. O *acoplamento* deixa de ser por implementação e passa a ser por contrato declarado. É o mesmo gesto do capítulo 13 (“framework não é mágica”): o que torna a cooperação possível não é a biblioteca, é o contrato.

Horizontal e vertical: A2A versus MCP

A pergunta inevitável de 2025 foi: *MCP ou A2A*? A resposta madura é que são camadas diferentes, e a confusão entre elas é o erro mais comum da era. A imagem que os dois protocolos juntos formam é de duas direções ortogonais:



O **MCP é a camada vertical**: conecta um modelo (ou agente) para baixo, ao mundo de ferramentas e dados — a “porta USB” de cada agente. O **A2A é a camada horizontal**: conecta agentes entre si, para os lados. Eles não competem; compõem-se. Um agente usa MCP para buscar dados e A2A para falar com outro agente — a AWS e a própria especificação do A2A descreveram essa composição. O mapa mental para o engenheiro é simples: **um agente fala com o mundo pelo MCP; fala com outros agentes pelo A2A**.

Adoção e governança: de 50 parceiros à Linux Foundation

O A2A nasceu com um ecossistema que o MCP não tinha no dia do lançamento. No anúncio de 9 de abril de 2025, a Google listou o apoio de **mais de 50 parceiros de tecnologia** — Atlassian, Box, Cohere, Intuit, LangChain, MongoDB, PayPal, Salesforce, SAP, ServiceNow, UKG e Workday, entre outros — além de provedores de serviço como Accenture, BCG, Capgemini, Deloitte, Infosys e PwC. A adoção enterprise chegou rápido: o **Microsoft** passou a oferecer suporte a A2A no Azure AI Foundry, o **SAP** integrou ao Joule, o **ServiceNow** ao Agent Fabric, e plataformas como Zoom e Vertex seguiram o mesmo caminho.

O movimento de governança veio em junho de 2025: a Google doou a especificação, os SDKs e as ferramentas de desenvolvimento do A2A à **Linux Foundation**, que anunciou o projeto *Agent2Agent* com AWS, Cisco, Google, Microsoft, Salesforce, SAP e ServiceNow como fundadores. Mais de 100 empresas apoiavam o protocolo naquele momento — número que passou de 150 até agosto. A doação respondeu à preocupação central que os observadores levantaram desde o lançamento: um protocolo de interoperabilidade controlado por um único fornecedor é uma contradição. Ao entregar o padrão a governança neutra, o A2A seguiu o caminho que consagrou os grandes padrões de infraestrutura — e que o MCP repetiria em dezembro de 2025.

A evolução 0.3: gRPC e segurança enterprise

A versão **0.3**, anunciada em julho/agosto de 2025, converteu o protocolo em candidato a infraestrutura enterprise. Duas novidades foram as principais. A primeira é o **gRPC** como transporte de alto desempenho, ao lado do JSON-RPC sobre HTTP — para cenários com grande número de agentes e requisitos de baixa latência. A segunda é a capacidade de **assinar security cards**: o documento que descreve as capacidades e políticas de segurança de um agente passa a ser assinado, criando a base para autenticação e autorização verificáveis entre agentes de domínios diferentes — um passo rumo à *trust* entre organizações, não apenas entre serviços.

A mesma rodada trouxe o suporte nativo no **Agent Development Kit (ADK)** e a abertura de um **marketplace** de agentes A2A no ecossistema Google Cloud, além de avaliação de sistemas A2A no Vertex GenAI Evaluation Service. A direção é a de qualquer protocolo maduro: menos demos, mais operação — descoberta, autenticação, monitoramento e mercado.

O cenário 2025-2026: dois padrões, uma mesma forma

No horizonte em que este livro fecha, o desenho ficou claro: **MCP lidera em adoção de desenvolvedores** — é a porta de entrada para tools e dados, com dezenas de milhões de downloads —, enquanto **A2A é forte em ecossistemas enterprise** — é o idioma da cooperação entre agentes de vendedores diferentes, com parceiros como Salesforce, SAP e ServiceNow já usando-o em plataformas próprias. Os especialistas divergem sobre qual prevalecerá ou se os dois convergirão — e há um debate real sobre a dupla segurança/abertura: o A2A nasceu com primitivas de segurança mais estruturadas (security cards

assinados, zero-trust), enquanto o MCP é mais flexível e depende de implementação cuidadosa por cada integrador.

A lição que interessa ao engenheiro não é a aposta em um dos lados — é a *forma* que os dois compartilham. O padrão REST/JSON que organizou a integração de APIs na última década está se repetindo, agora no nível de agentes: descoberta declarada (Agent Card), tarefas com ciclo de vida, autenticação padronizada, artefatos como unidade de entrega. Não é coincidência; é o mesmo problema de sempre — como fazer sistemas independentes cooperarem sem acoplar — reaparecendo uma camada acima.

A lição estrutural

O fio que liga o MCP ao A2A é o mesmo que liga o function calling ao MCP: **a interface padronizada transforma integração em commodity**. Quando o MCP padronizou a camada modelo→mundo, ferramentas deixaram de ser integração dedicada e viraram serviço. Quando o A2A padronizou a camada agente→agente, a cooperação entre agentes de vendedores diferentes deixou de ser projeto de engenharia por par e virou protocolo.

Para a tese deste livro, a consequência é a última peça do quebra-cabeça das Partes VI e VII. O capítulo 13 mostrou o agente como sistema com estado, ações, observações e critério de término; o capítulo 14 mostrou que múltiplos agentes só valem com divisão de trabalho, perspectivas ou heterogeneidade; os capítulos 15 e 16 mostraram como medir e como conectar ao mundo. O A2A mostra o agente *como serviço*: um sistema que declara suas capacidades, autentica-se, aceita tarefas e entrega artefatos — consumível por qualquer outro sistema que fale o mesmo contrato. É o mesmo gesto que transformou aplicações monolíticas em APIs; agora, aplicações viram agentes, e APIs viram protocolos de agente.

Para o engenheiro

A2A resolve o problema agente→agente: descoberta de capacidades via Agent Card, autenticação e troca de tarefas em JSON-RPC, sem integração ponto a ponto entre pares. Se você orquestra agentes de vendedores diferentes — ou vai orquestrar —, planeje em torno do protocolo, não de um SDK proprietário: é a diferença entre portabilidade e lock-in.

Tenha o mapa mental fixo: **MCP é vertical** (modelo→mundo) e **A2A é horizontal** (agente→agente). Não use um no lugar do outro, e não trate os dois como concorrentes a escolher — um agente usa MCP para ferramentas e A2A para falar com outros agentes.

Ao publicar um agente, pense em contrato antes de implementação: qual Agent Card ele vai expor, quais skills declara, como autentica e qual o ciclo de vida das tarefas que aceita. Um agente sem contrato declarado é um serviço que ninguém consegue descobrir — o equivalente exato a uma API sem documentação.

E adote o protocolo cedo, mesmo com o ecossistema ainda amadurecendo. Os padrões de interoperabilidade sobrevivem aos modelos que os usam: designs neutros de vendedor duram mais que qualquer framework da temporada. O valor está na forma, não na hype.

Fontes: [Google, 2025] — Announcing the Agent2Agent Protocol (A2A); [InfoQ, 2025] — A2A na Linux Foundation; [InfoWorld, 2025] — A2A 0.3, gRPC e segurança; [VentureBeat, 2025] — A2A e MCP em interoperabilidade; [MIT Tech Review, 2025] — panorama de protocolos; [Agentic AI Frameworks, 2025] — protocolos de comunicação entre agentes.

Parte VIII — Síntese

Capítulo 18 — O loop cognitivo: RAG + agentes + ferramentas como um só sistema

Este é o capítulo em que o livro fecha o círculo — e pede que você desconfie das divisões que ele próprio usou. RAG, ferramentas, agentes e protocolos parecem quatro tecnologias, contadas em quatro partes separadas. A tese deste capítulo é que são **uma única arquitetura vista de ângulos diferentes**. O RAG é memória externa; as ferramentas são ação; os agentes são o loop que liga memória a ação; os protocolos são a camada que permite a tudo isso cooperar entre sistemas. Nenhuma das quatro existe isolada em um sistema real — e o livro que as separou em capítulos agora tem a obrigação de juntá-las.

O problema que este capítulo resolve é conceitual: como encaixar tudo o que veio antes em um só padrão? A resposta é o **loop** — objetivo, observar, raciocinar, agir, observar, verificar, atualizar —, a mesma estrutura que o capítulo 10 (ReAct) introduziu, que o capítulo 13 elevou a sistema e que os capítulos 14 a 17 expandiram para muitos agentes e protocolos. A tabela de equivalências em que cada ferramenta é uma extensão cognitiva organiza a síntese: search é aquisição, RAG é recuperação, shell é experimentação, testes são verificação, arquivos são memória.

E a ambição didática é a maior do livro: um **projeto completo** — pergunta → recuperar → planejar → chamar ferramenta → executar → verificar → corrigir → responder — que mostra cada peça dos capítulos anteriores em ação. Não como código de framework, mas como a forma mínima de um sistema que percebe, busca, raciocina, age, verifica e corrige. Quando você reconhecer esse loop dentro de todo sistema que construir, os capítulos deste livro deixam de ser uma história e viram uma caixa de ferramentas.

Um só padrão, quatro ângulos

Os capítulos 5-17 contaram a história de quatro peças. A Parte II e a Parte V contaram o **RAG**: conhecimento externo recuperado e condicionando a geração — memória não-paramétrica. A Parte IV contou as **ferramentas**: o modelo que chama funções, executa código, navega em tela — ação. A Parte VI contou os **agentes**: loops com estado, objetivo e critério de término — o sistema que orquestra memória e ação. A Parte VII contou os **protocolos**: camadas padronizadas entre modelos, ferramentas e agentes — a interoperabilidade. Quatro histórias, um só ciclo:

```
objetivo
↓
observar (contexto, estado, memória)
↓
raciocinar (modelo)
↓
agir      (search | RAG | shell | edit | API | ...)
↓
observar  (resultado da ação)
↓
verificar (testes, execução, comparação)
↓
atualizar → loop ou finalizar
```

RAG e search alimentam o “observar”; o modelo faz o “raciocinar”; ferramentas e shell fazem o “agir”; testes e execução fazem o “verificar”; arquivos e banco são o “estado” que persiste entre iterações. Cada parte do livro construiu um pedaço do loop, e nenhum sistema real sobrevive com um pedaço só. A pergunta de desenho deixa de ser “qual tecnologia usar?” e vira “qual peça do loop o meu problema exige?”.

A tabela de equivalências: ferramentas como extensões cognitivas

O passo seguinte é mapear cada ferramenta para a capacidade cognitiva que ela estende. A metáfora não é decorativa: diz por que cada ferramenta existe e o que ela acrescenta que o modelo sozinho não tem:

Ferramenta	Extensão cognitiva	O que o modelo ganha
Search	aquisição de informação	acesso a fontes externas ao conhecimento paramétrico
RAG	recuperação contextual	evidência específica condicionando a geração
Shell	experimentação	executar hipóteses e observar os resultados
Calculador / código	computação exata	precisão onde a inferência probabilística falha
Arquivos / banco	memória externa	estado que persiste entre sessões e iterações
Testes	verificação	evidência objetiva de que a mudança funciona
Compilador	verificação formal parcial	erros de tipo e sintaxe detectados antes de executar

A consequência é dupla. Primeiro, nenhuma dessas capacidades é do *modelo*: é do sistema composto. O capítulo 0 prometeu isso e os capítulos 1-17 provaram por partes. Segundo, a escolha de ferramenta vira uma questão de *custo da capacidade*: search quando falta informação, shell quando uma hipótese pode ser testada, calculadora quando o número precisa ser exato, testes quando a propriedade pode ser verificada. O modelo decide; a ferramenta confere. Essa é a separação generator/verifier que o capítulo 10 introduziu e que aqui se generaliza: **o modelo propõe, o mundo confirma**.

RAG não é só retrieval: é construção de evidência

O capítulo 12 terminou no agentic RAG: recuperação como *decisão* dentro do loop. Este capítulo completa a frase: o RAG não é um mecanismo de busca — é um processo de **construção de evidência**. A diferença é de propósito:

```

query
↓
evidência: aquisição (qual fonte, qual consulta, quantas buscas)
↓
evidência: organização (rerank, filtrar, fundir, priorizar)
↓
evidência: raciocínio (a resposta é consequência das evidências?)
↓
answer (+ proveniência)

```

Cada estágio responde a uma pergunta de qualidade. A aquisição responde “a evidência certa chegou?”; a organização, “a evidência está na ordem e no peso certos?”; o raciocínio, “a conclusão decorre da evidência?” — e a pergunta final, que o capítulo 15 transformou em método de avaliação, é “a resposta é consequência de evidências suficientes, sem extrapolar além delas?”. Quando o RAG vira peça do loop, essas perguntas deixam de ser de um pipeline e passam a ser do sistema inteiro: o agente decide quando já tem evidência suficiente (retrieval como política), quando a evidência contradiz a hipótese (comparação) e quando precisa buscar de novo (iteração). Retrieval não é o mecanismo; a construção de evidência é o objetivo.

Geração não é validação

O loop tem uma porta de saída que os sistemas reais frequentemente esquecem: o **verificar**. E este capítulo repete, agora com o peso de todas as partes do livro, a distinção que o AGENTS.md (§24-25) deste projeto tornou princípio: **geração não é validação**. Reflexão do próprio modelo — Reflexion, Self-Refine, LATS, LLM-as-judge — melhora candidatos, mas não é evidência independente: o mesmo sistema que gera também avalia, e vieses se repetem. O capítulo 15 mostrou o custo disso em avaliação; aqui a regra é para operação.

O verificador deve ser tão independente quanto possível da geração:

teste unitário	→ propriedade verificável do resultado
execução	→ o código roda? a resposta do shell confirma?
comparação	→ a afirmação bate com a fonte recuperada?
cálculo independente	→ a aritmética confere?
validação estrutural	→ o schema foi respeitado? o contrato foi cumprido?

A hierarquia é a do capítulo 15, agora aplicada dentro do loop: quando uma propriedade pode ser testada por execução ou comparação, prefira o teste à autoavaliação. O modelo explora; o verificador decide; o loop corrige. Um sistema que ignora o verificar não é um agente — é um gerador otimista.

O projeto didático: pergunta → evidência → ação → verificação

Para fechar, o projeto prometido: um sistema completo, mínimo, em que cada peça dos capítulos anteriores aparece com nome. A pergunta do exemplo é prosaica — “quanto custou a última campanha de marketing, e o gasto passou do orçamento?” — porque o valor didático está no fluxo, não no domínio.

1. pergunta: "Quanto custou a campanha de março e passou do orçamento?"
2. observar: contexto = usuário + histórico (arquivos/memória)
3. raciocinar: que informação falta? valores e regra de orçamento
4. agir 1: RAG → buscar("campanha março custo") → top-k de chunks
5. observar: chunks com valores parciais; falta o valor do orçamento
6. agir 2: RAG → buscar("orçamento anual marketing 2026")
7. agir 3: calculadora → $48250.00 + 12350.00 + 9150.00 = 69750.00$
8. verificar: $69750.00 > 60000.00?$ → sim, estourou em 9750.00
9. corrigir: confrontar o total com a planilha recuperada (fonte)
10. responder: resposta + proveniência (chunks e cálculo)

Cada passo usa uma peça do livro. O passo 2 é o estado e a memória do capítulo 13; os passos 4–7 são do RAG (capítulo 12) e a ferramenta de cálculo (capítulos 9–11); o passo 8 é a verificação do capítulo 15; o passo 9 é a iteração corrigindo a resposta; o passo 10 é a proveniência que o RAG sempre prometeu. Se a verificação falhasse — o total não batesse com a planilha —, o loop voltaria ao passo 4 com uma consulta nova: é a iteração dos capítulos 10 e 12 em ação.

A versão sua desse projeto é um template, não um produto: escolha **uma** ferramenta real — search, calculadora ou shell —, um objetivo e um critério de parada, e rode o loop explicitamente. É o passo a passo mínimo de um agente, e é também o menor sistema que honra a tese deste livro: modelo + contexto + recuperação + ferramentas + ambiente + memória + iteração + verificação.

A lição estrutural

RAG, ferramentas, agentes e protocolos são uma arquitetura só: **objetivo → observar → raciocinar → agir → observar → validar → atualizar**. As “eras” do livro foram uma organização didática de uma estrutura que sempre esteve inteira. O RAG sem agente é um pipeline sem decisão; o agente sem RAG é um raciocinador sem evidência; o agente sem verificação é um gerador otimista; e o agente sem protocolo é um sistema que não conversa.

A pergunta que o capítulo 0 fez — que combinação de modelo, contexto, ferramentas, ambiente, memória, feedback e verificação torna o sistema confiável? — encontrou aqui a sua resposta estrutural: **o loop**. É o padrão que o capítulo 10 descobriu, que o capítulo 13 sistematizou, que o capítulo 14 multiplicou, que os capítulos 16–17 conectaram ao mundo — e que o próximo e último capítulo vai avaliar, limitar e projetar para o horizonte.

Para o engenheiro

Quando for desenhar um produto, comece pelo loop, não pela stack. Mapeie cada peça do ciclo para um componente concreto do código: retriever, tools, scheduler, logger, validador, armazenamento de estado. Se alguma peça não tem dono no seu projeto, ela está invisível — e uma peça invisível é uma falha latente. A maioria dos problemas de sistemas agentic está na orquestração (evidência, estado, validação), não no modelo.

Use a tabela de equivalências como checklist de capacidades: o seu sistema tem aquisição de informação? recuperação? experimentação? computação exata? memória persistente? verificação? Cada “não” é um risco — e cada resposta adiada é um custo futuro. Antes de trocar o modelo por um maior, feche as lacunas do loop: quase sempre rende mais.

Teste o loop antes de otimizar o modelo. Construa a versão mínima do projeto didático com uma ferramenta real e um critério de parada; meça onde ele quebra; só então escale. E nunca deixe o “verificar” ser feito só pelo próprio modelo: quando a propriedade puder ser testada por execução ou comparação, teste. O loop explora; o verificador decide; o seu sistema, se estiver bem desenhado, corrige e responde.

Fontes: Síntese conceitual baseada nos capítulos anteriores e na orientação do AGENTS.md; [Survey de tool use, 2026] — single-tool vs. multi-tool orchestration; [Gao et al., 2023] e [Li et al., 2024] — RAG como paradigma.

Capítulo 19 — Avaliação, limites e o horizonte

Todo bom livro de engenharia termina com a parte desconfortável: o que sabemos e o que não sabemos. Este capítulo de fechamento recusa o final feliz. A narrativa chegou a 2026 com agentes, RAG, ferramentas e protocolos — mas o elo mais fraco de toda a corrente é o mesmo que o capítulo 15 anunciou: **medir sistemas agentic de forma confiável continua sendo um problema em aberto**. A consistência entre tentativas é o gargalo, o LLM-as-judge não é validação independente, e avaliar processo custa muito mais do que avaliar resposta.

Os capítulos 0-18 montaram uma tese completa: um LLM é o núcleo de um sistema maior composto de contexto, recuperação, ferramentas, ambiente, memória, iteração e verificação. Este capítulo cobra a conta da tese. Para cada componente, quais são os limites conhecidos — segurança e reversibilidade das ações, custo dos loops, critérios de parada? O que o campo sabe medir, e o que ainda não sabe? E, no horizonte de 2026+, quais estruturas prometem fechar essas lacunas — orquestração multi-ferramenta, modelos de raciocínio, agentes verificáveis, a coexistência do RAG com o contexto longo?

A pergunta final é a mesma do capítulo 0, agora com as mãos sujas: *que combinação de modelo, contexto, ferramentas, ambiente, memória, feedback e verificação torna isso confiável — e como sabemos que é confiável?* A diferença é que, depois de dezoito capítulos, a pergunta deixou de ser retórica e virou especificação de engenharia.

O elo fraco: a consistência entre tentativas

O capítulo 15 apresentou o dado que organiza todo este capítulo: o tau-bench mostrou que modelos fortes falham em mais da metade das tarefas de agente e, decisivamente, que a **consistência entre execuções é pior do que a precisão**. A consequência é implícita e brutal: um sistema que funciona em uma demo pode falhar na mesma tarefa minutos depois, com o mesmo prompt e o mesmo modelo.

Por que a consistência é o elo fraco? Porque uma tarefa de agente é uma *cadeia* de decisões — escolher a ferramenta, formular a consulta, interpretar a observação, decidir se já chegou —, e a probabilidade de a cadeia inteira dar certo é o produto das probabilidades de cada elo. Se cada passo acerta com 90% de probabilidade, dez passos sucessivos dão ~35% de sucesso — e o modelo “forte” deixa de ser forte no sistema. A robustez não se herda do modelo: constrói-se no processo — mais verificação, mais estados de recuperação, mais testes. É por isso que o capítulo 18 insistiu que o verificador é parte do loop: em sistemas agentic, a verificação é o que transforma 35% em 90%.

Para o engenheiro, o diagnóstico muda a política de produto. Uma resposta certa por acaso não é um sistema robusto — e isso vale do protótipo à produção. A primeira pergunta de qualquer sistema de agente não deve ser “qual modelo?”, mas “qual a consistência deste processo — e como a medimos?”.

Segurança e reversibilidade: a escala das ações

O segundo limite é a classificação das ações que um agente pode tomar. Um agente que apenas lê arquivos tem um risco; um agente que escreve, envia, paga ou apaga tem outro. A classificação que o AGENTS.md (§35) deste projeto estabeleceu é o mapa de risco:

ação informativa	→ ler, buscar, calcular	→ risco baixo
ação reversível	→ escrever rascunho, criar	→ risco médio
ação potencialmente destrutiva	→ enviar, modificar, excluir	→ risco alto
ação irreversível	→ pagar, enviar email, apagar	→ risco máximo

A regra de ouro é proporcionalidade: **quanto maior o impacto, maior a verificação** — e isso se materializa em guardrails (o termo da indústria para as camadas de controle). Frameworks da era, como os que o survey *Agentic AI Frameworks* (2025) documentou, passaram a embutir políticas: confirmação humana para ações destrutivas (*human-in-the-loop*), permissões por escopo, logs obrigatórios, orçamentos de tokens e tempo. O capítulo 13 viu esse movimento nascendo com o LangGraph; aqui ele é princípio. Ferramentas poderosas exigem mais disciplina, não menos — e a disciplina é o que separa uma capacidade de um acidente.

A reversibilidade também define a estratégia de teste. Ações informativas podem ser executadas livremente em produção; ações destrutivas devem ser testadas em ambientes de replicação antes de tocar o mundo real. Um agente que envia e-mails deveria ter, por padrão, um modo de ensaio — a mesma lógica de *staging* que o shell deste projeto usa para experimentação controlada. O custo de errar cresce com a irreversibilidade; a inspeção deve crescer na mesma proporção.

Custo e o critério de parada

O terceiro limite é o mais prosaico e o mais negligenciado: **custo**. Cada iteração do loop consome tokens, e o custo de um loop longo é a diferença entre uma demo e um sistema de produção. O critério de parada do capítulo 13 — *quando parar?* — tem aqui a sua formulação econômica:

continue enquanto:	ganho de informação esperado	>	custo da ação
pare quando:	ganho de informação esperado	≤	custo da ação

É a mesma lógica do capítulo 18 aplicada ao orçamento: iterar sem informação nova apenas repete o erro pagando por ele. Os protocolos da Parte VII acrescentaram uma dimensão nova a essa conta — o *custo da comunicação*. MCP e A2A “gastam tokens para falar”: cada primitiva trocada entre cliente, servidor e agentes consome contexto e latência, e em sistemas com muitos servidores ou muitos agentes a soma é real. O capítulo 14 já vira esse custo no multi-agente; aqui ele aparece como propriedade dos protocolos — e exige a mesma disciplina de orçamento: medir tokens por tarefa, limitar iterações, monitorar deriva.

O horizonte 2026+

Com os limites mapeados, o horizonte tem quatro movimentos que prometem — com a honestidade de quem viu a hype de 2023 de perto — *avancar*, não *resolver*.

Orquestração multi-ferramenta. O survey *From Single-Tool Call to Multi-Tool Orchestration* (2026) registra a transição das chamadas únicas para trajetórias longas com estado, em que o agente alterna ferramentas, reutiliza observações e gerencia contexto entre chamadas. É o loop do capítulo 18 em escala industrial — e o problema de desenho passa a ser segurança, custo e verificabilidade de trajetórias, não a chamada individual.

Modelos de raciocínio. O raciocínio explícito como etapa de inferência — o *test-time compute*, o “pensar antes de responder” — mudou a economia do loop: mais tokens gastos *dentro* da inferência podem reduzir o número de iterações *externas* de busca e correção. A pergunta de pesquisa é quanta deliberação compensa em cada tarefa — o mesmo trade-off do critério de parada, agora dentro do modelo.

Agentes verificáveis. A fronteira estrutural que os protocolos stateless não entregam por padrão — proveniência, linhagem e memória duradoura — virou objeto de design. Agentes verificáveis são aqueles em que cada ação tem rastro, cada resposta tem fonte e o estado persiste entre sessões; o movimento completa o que o capítulo 15 começou (medir processo) e o capítulo 18 tornou princípio (verificação como peça do loop).

RAG e contexto longo. O capítulo 12 terminou com o roteamento dinâmico — contexto longo quando a resposta exige o documento inteiro, RAG quando exige trechos. O horizonte aponta a *coexistência*: janelas de 1M+ de tokens tornaram o contexto longo uma opção econômica em casos específicos, e o desenho maduro roteia por consulta, em vez de apostar em uma arquitetura única. A conclusão de Li et al. (2024) — na maioria das consultas RAG e contexto longo concordam — permanece a bússola da decisão.

As perguntas que o campo ainda não respondeu

Por fim, o inventário honesto do que permanece aberto — porque um livro de engenharia termina melhor apontando o desconhecido do que fingindo que acabou:

- **Consistência:** por que a variabilidade entre execuções é tão alta, e como projetar processos que a reduzam? O tau-bench mediu o sintoma; a cura é pesquisa em aberto.
- **Avaliação objetiva de processo:** como medir planejamento, seleção de ferramenta e recuperação de falha sem depender de LLM-as-judge — o verificador que o capítulo 18 exige para operação ainda é frágil para avaliação?
- **Governança de protocolos:** o MCP e o A2A migraram para a Linux Foundation, mas o poder de

definir o padrão — e de estendê-lo — é um equilíbrio em movimento; quem decide, com que critério e em que velocidade?

- **Segurança de recuperação:** o RAG herda os riscos das suas fontes — envenenamento de documentos, distratores injetados, jailbreaks via contexto; como o loop verifica a *fonte*, não apenas a resposta?
- **O custo de tudo:** quando o loop, os protocolos e os multi-agentes somam tokens, latência e complexidade, qual a lei de custo dos sistemas agentic — e quem a está medindo com rigor?

Nenhuma dessas perguntas tem resposta fechada em 2026. A disciplina que o livro ofereceu — avaliar por estágio, verificar com independentes, medir consistência, classificar risco, definir critério de parada — é exatamente a ferramenta para trabalhar dentro dessa incerteza: não para eliminá-la, mas para torná-la observável.

A lição estrutural

A herança deste capítulo não é um veredito, é um método — e o método é a tese do livro testada no limite: **a confiabilidade de um sistema agentic é propriedade do processo, não do modelo**. A consistência é o elo fraco porque uma tarefa de agente é uma cadeia de decisões, e a cadeia inteira só dá certo quando cada elo acerta — mas é também a prova de que a robustez se constrói: mais verificação, mais estados de recuperação e mais testes são o que transforma um sistema que acerta 35% das vezes em um que acerta 90%. As quatro disciplinas do capítulo — medir por estágio, verificar com mecanismos independentes do gerador, classificar ações por risco e definir critério de parada — não fecham as perguntas que a seção anterior deixou em aberto; elas fazem o que a engenharia sempre faz com o desconhecido: torná-lo observável. E é esse o fecho do arco: cada era do livro adicionou uma peça ao sistema — contexto, recuperação, ferramentas, agentes, protocolos —, e a última peça é a que mede se as outras funcionam.

A pergunta final

O capítulo 0 abriu o livro com uma pergunta que parecia de filosofia e era de engenharia: *que combinação de modelo, contexto, ferramentas, ambiente, memória, feedback e verificação torna o sistema confiável?* Dezoito capítulos depois, a pergunta se desdobrou em especificações: a recuperação precisa construir evidência (capítulos 5-12); as ferramentas precisam ser extensões com contrato (capítulos 9-11); os agentes precisam de estado, objetivo e critério de término (capítulos 13-14); a avaliação precisa medir processo e consistência (capítulo 15); os protocolos precisam padronizar o mundo (capítulos 16-17); e o loop precisa verificar, corrigir e parar (capítulos 10 e 18).

A metade que falta é a que este capítulo deixou em aberto: *como sabemos que é confiável?* A resposta madura não é um benchmark novo, nem um modelo maior — é a combinação das duas disciplinas que este livro repetiu até a exaustão: **medir o processo, não apenas a resposta, e verificar com mecanismos independentes do gerador**. Quem construir sistemas assim não terá eliminado a incerteza — ninguém tem —, mas terá feito o que a engenharia sempre faz com o desconhecido: transformado-o em observável, em medível e, portanto, em gerenciável.

Para o engenheiro

O elo fraco atual é a consistência: um sistema que falha em mais da metade das tarefas não é de produção, por mais forte que seja o modelo por baixo. Meça pass@k e a variabilidade entre execuções antes de confiar em qualquer demo — e não aceite “funcionou na minha máquina” como evidência de robustez.

Avaliar sistema custa mais que avaliar resposta. Reserve orçamento e construa uma suíte própria de testes de agente — tarefas reais, critérios explícitos, execução repetida — exatamente como você já faz com testes unitários. Sem essa suíte, qualquer melhoria é opinião, não medida.

Classifique as ações do seu agente por risco: informativa → reversível → destrutiva → irreversível. Quanto mais irreversível, mais confirmação explícita, mais logs e mais ambiente de ensaio. Guardrails não são burocracia: são o que permite dar autonomia sem acidente.

E quanto ao horizonte 2026+ — orquestração multi-ferramenta, modelos de raciocínio, agentes verificáveis, RAG e contexto longo — acompanhe, mas meça no seu caso antes de confiar. Capacidade demonstrada em demo não é robustez, e a regra que fechou este livro vale para as tecnologias que ainda não existem: o modelo propõe, o processo decide, e o sistema verifica — é nessa combinação, e não em nenhuma peça isolada, que a confiabilidade se constrói.

Fontes: [From Language to Action, 2025] — revisão de benchmarks e avaliação; [Agentic AI Frameworks, 2025] — guardrails; [Survey de tool use, 2026] — multi-tool orchestration; [Yao et al., 2024] — tau-bench, consistência; [Li et al., 2024] — RAG vs. long-context; [InfoWorld, 2025] — limites de estado/observabilidade dos protocolos; [MIT Tech Review, 2025] — custo de tokens em comunicação entre agentes; [Revisão sistemática, 2025] — lacunas de avaliação de RAG.

Anexo A — Cronologia

Cronologia

Linha do tempo dos marcos que estruturam o livro. Cada entrada liga a uma fonte em fontes.md. Datas de artigos seguem a primeira publicação (preprint/arXiv); datas de produtos seguem o anúncio oficial.

Sementes (1950-2016)

Data	Marco	Por que importa
1950	Turing, "Computing Machinery and Intelligence"	Define o objetivo de conversar como critério de inteligência.
1954	Experimento de tradução de Georgetown	Primeira demonstração pública de tradução por máquina.
1986	Backpropagation (Rumelhart et al.)	Torna viável treinar redes profundas.
1997	LSTM (Hochreiter & Schmidhuber)	Mitiga o problema do gradiente que some; base das RNNs modernas.
2003	Modelo neural de linguagem (Bengio et al.)	Linguagem aprendida por rede neural + embeddings aprendidos.
2013	word2vec (Mikolov et al.)	Embeddings distribucionais: "rei - homem + mulher $\approx$ rainha".
2014	Seq2seq (Sutskever et al.; Cho et al.)	Encode-decode de sequências; base das arquiteturas modernas.
2015	Atenção (Bahdanau et al.)	Alinhamento dinâmico entre entrada e saída.
2016	Google Neural Machine Translation	Seq2seq + atenção em produção; derrota a tradução estatística.

O Transformer e o pré-treinamento (2017-2019)

Data	Marco	Fonte
2017-06-12	Attention Is All You Need — o Transformer	Vaswani et al. 2017
2018-02	ELMo — representações contextuais (bi-LSTM)	Peters et al. 2018
2018-06-11	GPT-1 — pré-treinamento generativo	Radford et al. 2018
2018-10-11	BERT — encoder bidirecional pré-treinado	Devlin et al. 2018
2019-02-14	GPT-2 — modelos são aprendizes multitarefa não supervisionados	Radford et al. 2019
2019-10-23	T5 — tudo é texto-para-texto	Raffel et al. 2019

Escala, emergência e as leis de escala (2020-2022)

Data	Marco	Fonte
2020-05-28	GPT-3 (175B) — in-context learning	Brown et al. 2020
2020	Leis de escala de modelos de linguagem	Kaplan et al. 2020
2022-03-29	Chinchilla — computação ótima	Hoffmann et al. 2022

Data	Marco	Fonte
2022-04-04	PaLM 540B	Chowdhery et al. 2022
2022-06-16	Emergent abilities of LLMs	Wei et al. 2022

Nascimento do RAG (2020-2022)

Data	Marco	Fonte
2020-05-22	RAG — Retrieval-Augmented Generation	Lewis et al. 2020
2020	REALM	Guu et al. 2020
2020	kNN-LM	Khandelwal et al. 2020
2020	DPR — dense passage retrieval	Karpukhin et al. 2020
2021	FiD — fusion-in-decoder	Izacard & Grave 2021
2021-12-17	WebGPT — modelo que navega em um browser	Nakano et al. 2021
2022	RETRO — retrieval no pré-treinamento	Borgeaud et al. 2022

Alinhamento e o momento ChatGPT (2021-2022)

Data	Marco	Fonte
2017	Origem do RLHF — reward model + RL com preferências humanas (Atari/controle)	Christiano et al. 2017
2021	LaMDA — diálogo com grounding	Thoppilan et al. 2022
2022-01-28	Chain-of-Thought prompting	Wei et al. 2022
2022	InstructGPT / RLHF	Ouyang et al. 2022
2022-11-30	ChatGPT (produto)	OpenAI 2022

Tool use: a pesquisa (2021-2023)

Data	Marco	Fonte
2022-05-01	MRKL — sistemas neuro-simbólicos	Karpas et al. 2022
2022-10-06	ReAct — raciocínio + ação	Yao et al. 2022
2022-11	PAL — programa-aided language models	Gao et al. 2022
2023-02-09	Toolformer — o modelo que se ensina a usar APIs	Schick et al. 2023
2023-03-30	HuggingGPT — LLM como controlador de modelos	Shen et al. 2023
2023-05-24	Gorilla — fine-tuning consciente de APIs	Patil et al. 2023

Tool use: a indústria (2023-2024)

Data	Marco	Fonte
2023-03-23	ChatGPT plugins (browser, code interpreter, retrieval)	OpenAI 2023
2023-06-13	Function calling na API (GPT-3.5/4)	OpenAI 2023
2023-11-06	DevDay: parallel function calling, JSON mode, Assistants	OpenAI 2023
2023-11-21	Claude 2.1: tool use beta + 200K de contexto	Anthropic 2023
2023-12-13	Gemini Pro API: function declarations	Google 2023
2024-02	Berkeley Function-Calling Leaderboard	Gorilla Group 2024
2024-05-30	Tool use GA na Anthropic; Bedrock Converse API	Anthropic/AWS 2024
2024-08-06	Structured Outputs (strict: true)	OpenAI 2024

Data	Marco	Fonte
2024-10-22	Computer use (Claude 3.5)	Anthropic 2024

RAG avançado (2023-2025)

Data	Marco	Fonte
2023	Self-RAG (reflexão)	Asai et al. 2023
2023	FLARE (recuperação ativa)	Jiang et al. 2023
2023-12	Survey RAG: Naive/Advanced/Modular	Gao et al. 2023
2024	RAFT (treinado para usar/ignorar contexto)	Zhang et al. 2024
2024	RAPTOR (retrieval hierárquico)	Sarathi et al. 2024
2024	RAG vs. contexto longo (Self-Route)	Li et al. 2024
2024-10	Survey abrangente de RAG	Gupta et al. 2024
2025	LightRAG, PGraphRAG, agentic RAG	diversos

Agentes (2023-2025)

Data	Marco	Fonte
2023-03/04	AutoGPT, BabyAGI, AgentGPT, CAMEL	Richards 2023; Nakajima 2023; Li 2023
2023-04	Generative Agents (simulação social)	Park et al. 2023
2023-07	ChatDev; ToolBench	Qian 2023; Qin 2023
2023-08	AgentBench ; MetaGPT; AutoGen; AgentVerse	Liu 2023; Hong 2023; Wu 2023; Chen 2023
2023-08	Survey de agentes autônomos (perfil/memória/planejamento/ação)	Wang et al. 2023/2024
2023-09	"The Rise and Potential of LLM Agents"	Xi et al. 2023
2024-06-17	tau-bench — consistência de agentes	Yao et al. 2024
2025	WebArena/OSWorld; surveys de multi-tool orchestration	diversos

Protocolos (2024-2026)

Data	Marco	Fonte
2024-11-05	Especificação inicial do MCP	MCP spec 2024-11-05
2024-11-25	MCP anunciado e open-source (Anthropic)	Anthropic 2024
2025-03-26	Revisão da spec MCP	MCP spec 2025-03-26
2025-04-09	A2A anunciado (Google)	Google 2025
2025-06-23	A2A doado à Linux Foundation (com AWS, Cisco, Microsoft)	Google Cloud 2025; InfoQ 2025
2025-06-18	Revisão da spec MCP	MCP spec 2025-06-18
2025-07-31	A2A 0.3: gRPC e security cards assinados	Google Cloud Blog 2025; InfoWorld 2025
2025-08	A2A: 150+ organizações apoiando	Google Cloud Blog 2025
2025-11-25	Revisão da spec MCP (async, statelessness, identidade de servidor)	MCP spec 2025-11-25
2025-12-09	MCP doado à Linux Foundation (Agentic AI Foundation)	Anthropic 2025
2025-12	MCP: ~97M downloads de SDK/mês, 10.000+ servidores	Taskade 2026; Anthropic 2025
2026-03	Survey "Evolution of Tool Use in LLM Agents"	arXiv 2603.22862

Observação de método

Datas de *preprints* podem divergir entre repositórios (arXiv vs. conferência). Sempre que houver conflito, o livro segue a primeira data de publicação pública e registra a divergência. As datas de produtos seguem anúncios oficiais das empresas, citados em `fontes.md`.

Anexo B — Glossário

Glossário

Termos definidos no livro, com uma nota prática para quem implementa. Mantido em crescimento; cada capítulo adiciona entradas.

Fundamentos de modelos de linguagem

Termo	Definição	Por que importa
Modelo de linguagem (LM)	Sistema que atribui probabilidade a sequências de palavras; na forma autoregressiva, prevê a próxima palavra dado o contexto.	É a unidade básica: quase todo sistema moderno começa "prevê o próximo token" e adiciona contexto e ferramentas por fora.
Modelo de linguagem de grande escala (LLM)	LM baseado em Transformer, treinado em corpus massivo, com bilhões de parâmetros ou mais.	É o que você chama via API hoje; a escala é o que faz capacidades emergirem.
Pré-treinamento	Treinamento genérico autossupervisionado em texto; objetivo de prever tokens mascarados (BERT) ou a próxima palavra (GPT).	Rótulo grátis (o texto é o rótulo): é por isso que dá para treinar com a internet inteira.
Fine-tuning	Continuação do treinamento em dados específicos de uma tarefa.	É a alavanca quando zero-shot não basta; com LoRA custa uma fração do treino original.
LoRA	Adapta o modelo treinando só pequenas matrizes de low-rank (~1-2% dos parâmetros), deixando o resto congelado.	O padrão de ouro para customizar: um adaptador por cliente/tarefa, trocável sem re-treinar o base.
Zero-shot / few-shot	Executar tarefa sem exemplos / com poucos exemplos dados no prompt, sem atualizar pesos.	Teste isso antes de qualquer fine-tuning: é barato, reversível e cobre cada vez mais casos.
In-context learning	Capacidade de executar uma tarefa a partir apenas do conteúdo do prompt.	O prompt é código de produção: versionar, testar e monitorar como qualquer código.
Embedding	Vetor numérico que representa palavra/texto; captura similaridade semântica.	Base de todo RAG/busca semântica; a qualidade do embedding decide o teto do retrieval.
Atenção	Mecanismo que pondera a relevância entre posições; base do Transformer.	A ideia de "onde olhar" reusada em cross-attention, retrieval e agentes.
Transformer	Arquitetura baseada apenas em atenção, sem recorrência; proposta em 2017.	Todo modelo moderno que você consome é um Transformer.
Encoder-only / Decoder-only / Encoder-decoder	Famílias derivadas do Transformer: entendimento (BERT), geração causal (GPT), transformação (T5).	Escolha pela tarefa: classificar → encoder; chat/geração → decoder; tradução → encoder-decoder.
Autoregressivo / causal	Geração token a token, cada previsão olha só para trás.	Geradores não podem "ver o futuro": respeite isso ao montar prompts e fluxos.
Embeddings posicionais	Vetores que codificam a ordem das posições no input.	Reordenar input muda o significado; preserve ordem ao serializar documentos.

Termo	Definição	Por que importa
Alucinação	Geração de conteúdo factualmente incorreto com confiança.	O motivo central de RAG e ferramentas: ancorar a resposta em evidência externa.
Leis de escala (Kaplan / Chinchilla)	Relações que predizem perda/qualidade em função de parâmetros, tokens e FLOPs.	Use para dimensionar o modelo pelo orçamento, não pelo hype.
Emergência	Capacidade que "aparece" acima de certo tamanho de modelo.	Desconfie: muitas vezes é a métrica saturando, não uma habilidade nova real.

RAG e recuperação

Termo	Definição	Por que importa
RAG (Retrieval-Augmented Generation)	Paradigma que recupera evidência de uma fonte externa e a injeta no contexto antes de gerar.	O jeito padrão de dar conhecimento atualizado e auditável ao modelo.
Retriever	Componente que seleciona passagens/documentos de uma fonte externa.	Primeira linha do pipeline; é medido separadamente (recall@k, NDCG).
Generator	LLM que produz a resposta a partir de pergunta + evidências.	É medido por fidelidade: a resposta segue as evidências recuperadas?
Memória paramétrica	Conhecimento armazenado nos pesos do modelo (fixo após o treino).	Barata de usar, cara de atualizar; fonte das alucinações sobre dados recentes.
Memória não-paramétrica (externa)	Conhecimento armazenado fora do modelo, recuperado sob demanda (atualizável).	É o seu banco/índice: atualizar o conhecimento não exige re-treinar.
Grounding	Ancorar a resposta em evidência externa para reduzir alucinação.	Critério de qualidade de qualquer RAG em produção.
Proveniência	Capacidade de rastrear a fonte de uma afirmação.	Diferencia RAG de "adivinhação": sempre devolva a fonte junto da resposta.
Top-k	Número de passagens recuperadas/consideradas pelo retriever.	Hiperparâmetro clássico: k pequeno corta recall, k grande polui o contexto.
BM25	Ranking lexical por correspondência de termos (TF-IDF probabilístico).	Rápido e preciso para termos exatos; falha com paráfrases.
Retrieval denso (ex.: DPR)	Ranking por similaridade de embeddings de query e documento.	Captura paráfrase; precisa que encoders de query e documento sejam consistentes.
Busca híbrida	Combina scores lexicais e densos.	É o padrão de produção: você quer os dois, com peso combinado.
Reranker (cross-encoder)	Modelo que reordena o top-k olhando query+documento juntos.	Correção barata de ruído; quase sempre compensa a latência extra.
Query rewriting / expansão	Reescrever ou expandir a pergunta para melhorar a recuperação.	RAG avançado: o mesmo usuário mal-formula, o sistema reformula antes de buscar.
HyDE	Gera uma resposta hipotética e usa o embedding dela para buscar.	Truque de recall quando a pergunta e o documento não compartilham vocabulário.
Chunking	Divisão dos documentos em trechos indexáveis.	Decisão de qualidade barata: tamanho, sobreposição e quebra por seção importam mais que o índice.
kNN-LM	Aplica memória externa no nível do token, buscando vizinhos do contexto atual.	Mostrou que dá para recuperar no <i>nível do token</i> , não só no nível de documento.

Termo	Definição	Por que importa
FiD	Funde evidências no decoder, codificando cada passagem separadamente.	A saída quando o prompt não comporta concatenar muitas passagens.
RETRO / REALM	Recuperação aprendida <i>durante</i> o pré-treinamento.	Evidência de que retrieval pode ser parte do treino, não só da inferência.
Self-RAG	O modelo decide se recupera e critica suas próprias respostas via tokens de reflexão.	Torna a recuperação adaptativa: só busca quando necessário e auto-avalia a resposta.
FLARE	Recuperação ativa <i>durante</i> a geração quando a confiança do modelo cai.	Antecipa a busca no momento exato em que o modelo "não sabe", sem prompt manual.
RAFT	Modelo treinado a ignorar distratores e citar apenas fontes relevantes.	Ensina o modelo a filtrar ruído do retrieval — crucial para RAG robusto.
Prompt caching	Reutiliza a computação de atenção do prefixo do prompt em chamadas repetidas.	Corta latência e custo em chat multi-turno e agentes com prefixos fixos (system prompt).

Alinhamento e ferramentas

Termo	Definição	Por que importa
Instruction tuning	Fine-tuning em pares instrução→resposta para o modelo obedecer comandos.	Foi o que transformou "completar texto" em "assistente".
RLHF	Aprendizado por reforço a partir de feedback humano: SFT → reward model → RL.	O custo está nos dados de preferência, não na matemática.
Reward model	Modelo que pontua respostas; usado como sinal no RLHF.	Se for viesado, o modelo final herda o viés — avalie-o à parte.
Alinhamento	Ajuste para que o modelo siga intenções e valores humanos.	Defina o comportamento-alvo, não só a tarefa.
Chain-of-Thought (CoT)	Padrão de gerar passos intermediários de raciocínio antes da resposta.	Ferramenta de prompt barata para tarefas que exigem passos; custa tokens.
Tool use	Capacidade de um LLM invocar ferramentas externas (APIs, calculadoras, busca, código).	Quando um humano resolveria com ferramenta, o sistema deve ter a ferramenta.
Function calling	Primitiva de API em que o modelo retorna nome + argumentos de uma função.	Use o formato nativo do provedor em vez de "faça o modelo emitir JSON".
Parallel function calling	Múltiplas chamadas independentes numa só resposta do modelo.	Corta round-trips e latência do loop.
JSON mode / Structured Outputs	Garantia de que a saída obedece a um JSON Schema (ex.: strict: true).	Fim da era "o modelo devolveu JSON inválido": parser nunca mais quebra.
Computer use	Uso do computador inteiro (tela, teclado) como ferramenta.	Reserve para ambientes sem API; para produção, prefira integrações estruturadas.
MRKL	Roteia partes da pergunta para ferramentas específicas (calculadora, banco...).	Molde mental de <i>roteamento</i> : nem toda pergunta vai para o modelo.
PAL	O modelo escreve o código e a máquina executa a resposta.	Delegue matemática e lógica exata a código em vez de pedir ao modelo que calcule.
Toolformer	Aprende sozinho, por auto-supervisão, quando chamar cada API.	Em 2026, o function calling nativo cobre a maioria dos casos que ele demonstrou.

Agentes e avaliação

Termo	Definição	Por que importa
ReAct	Padrão que intercala raciocínio (Thought) e ação (Action) com observações, em loop.	A <i>forma</i> de todo agente: desenhe agentes nesse molde.
Agente (LLM-based)	Sistema em que o modelo participa de um ciclo observar → raciocinar → agir → observar → validar.	O poder vem do loop, não do modelo isolado.
Loop de agente	Ciclo de decisão-ação com observação e condição de parada.	Sem critério de parada, o custo explode — define <code>max_iterations</code> antes de rodar.
Observabilidade	Capacidade de ver o que o agente pensou, chamou e observou.	É o que permite debugar e reexecutar; logue sempre.
Multi-agente	Sistema com múltiplos agentes com papéis, comunicação e coordenação.	Vale quando há divisão real de trabalho; senão, um loop bem feito ganha.
Chatter	Agentes conversando sem produzir nada.	Evite com artefatos estruturados e papéis claros, em vez de mensagens soltas.
AgentBench / WebArena / OSWorld	Benchmarks que colocam agentes em ambientes realistas (web, terminal, desktop).	Pontos de partida para avaliação; crie a sua suite de tarefas reais.
tau-bench	Bench de tarefas realistas (viagens, atendimento) com avaliação multi-turno.	Mostrou o dado incômodo: modelos fortes falham na maioria das tarefas.
Consistência	Estabilidade do agente entre tentativas da mesma tarefa.	Acertar 1 em 3 não é produção: meça <code>pass@k</code> e variabilidade.
LLM-as-judge	Uso de um modelo para avaliar respostas.	Barato e viesado: rubrica explícita + amostra humana de calibração.
Guardrails	Mecanismos que validam saídas, impõem segurança e mantêm integridade de fluxos.	Camada de produção: valide schema, alcance e políticas antes de liberar a saída.

Protocolos e sistema

Termo	Definição	Por que importa
MCP (Model Context Protocol)	Protocolo aberto para conectar modelos a ferramentas/dados (modelo ↔ mundo).	Elimina integrações N×M: exponha uma vez, consuma de qualquer cliente.
tools / resources / prompts (MCP)	As três primitivas do MCP: ações, dados e receitas prontas.	Se o seu domínio não cabe nessas três, talvez não precise de MCP.
A2A (Agent2Agent)	Protocolo aberto para interoperabilidade entre agentes (agente ↔ agente).	Agentes de vendedores diferentes conversam sem integração ponto a ponto.
Estateless protocol	Protocolo que não mantém estado entre chamadas.	Persistência é responsabilidade sua (banco, memória, cache).
Loop cognitivo	Síntese do livro: objetivo → observar → raciocinar → agir → observar → validar → atualizar.	Use como checklist de arquitetura: cada peça precisa de um dono no seu código.

Anexo C — Fontes

Fontes

Bibliografia consolidada do livro. Agrupada por tema. Toda afirmação factual nos capítulos deve remeter a uma fonte aqui. Links apontam para publicações originais (arXiv, proceedings, anúncios oficiais).

Modelagem de linguagem e redes neurais (fundamentos)

- **Mikolov et al. (2013)** — *Efficient Estimation of Word Representations in Vector Space*. <https://arxiv.org/abs/1301.3781>
- **Hochreiter & Schmidhuber (1997)** — *Long Short-Term Memory*. Neural Computation.
- **Bengio et al. (2003)** — *A Neural Probabilistic Language Model*. JMLR.
- **Sutskever, Vinyals & Le (2014)** — *Sequence to Sequence Learning with Neural Networks*. <https://arxiv.org/abs/1409.3215>
- **Bahdanau, Cho & Bengio (2015)** — *Neural Machine Translation by Jointly Learning to Align and Translate*. <https://arxiv.org/abs/1409.0473>
- **Turing (1950)** — *Computing Machinery and Intelligence*. Mind.
- **Wu et al. (2016)** — *Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation*. <https://arxiv.org/abs/1609.08144>

O Transformer e o pré-treinamento

- **Vaswani et al. (2017)** — *Attention Is All You Need*. <https://arxiv.org/abs/1706.03762> (NeurIPS 2017)
- **Peters et al. (2018)** — *Deep Contextualized Word Representations (ELMo)*. NAACL 2018.
- **Radford et al. (2018)** — *Improving Language Understanding by Generative Pre-Training (GPT-1)*.
- **Devlin et al. (2018)** — *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. <https://arxiv.org/abs/1810.04805>
- **Radford et al. (2019)** — *Language Models are Unsupervised Multitask Learners (GPT-2)*. OpenAI Blog 1(8):9.
- **Raffel et al. (2019)** — *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5)*. <https://arxiv.org/abs/1910.10683>
- **Wikipedia** — *Transformer (deep learning architecture)* (contexto histórico; verificação cruzada). [https://en.wikipedia.org/wiki/Transformer_\(deep_learning_architecture\)](https://en.wikipedia.org/wiki/Transformer_(deep_learning_architecture))

Escala e emergência

- **Kaplan et al. (2020)** — *Scaling Laws for Neural Language Models*. <https://arxiv.org/abs/2001.08361>
- **Brown et al. (2020)** — *Language Models are Few-Shot Learners (GPT-3)*. <https://arxiv.org/abs/2005.14165>
- **Hoffmann et al. (2022)** — *Training Compute-Optimal Large Language Models (Chinchilla)*. <https://arxiv.org/abs/2203.15556>
- **Wei et al. (2022)** — *Emergent Abilities of Large Language Models*. <https://arxiv.org/abs/2206.07682>
- **Chowdhery et al. (2022)** — *PaLM: Scaling Language Modeling with Pathways*. <https://arxiv.org/abs/2204.02311>

RAG — papers fundadores

- **Lewis et al. (2020)** — *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. <https://arxiv.org/abs/2005.11401>
- **Guu et al. (2020)** — *REALM: Retrieval-Augmented Language Model Pre-Training*. <https://arxiv.org/abs/2002.08909>
- **Khandelwal et al. (2020)** — *Generalization through Memorization: Nearest Neighbor Language Models (kNN-LM)*. <https://arxiv.org/abs/1911.00172>
- **Karpukhin et al. (2020)** — *Dense Passage Retrieval for Open-Domain Question Answering (DPR)*. <https://arxiv.org/abs/2004.04906>
- **Izacard & Grave (2021)** — *Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering (FiD)*. <https://arxiv.org/abs/2007.01282>
- **Borgeaud et al. (2022)** — *Improving Language Models by Retrieving from Trillions of Tokens (RETRO)*. <https://arxiv.org/abs/2112.04426>
- **Nakano et al. (2021)** — *WebGPT: Browser-assisted question-answering with human feedback*. <https://arxiv.org/abs/2112.09332>

RAG — surveys e avanços (2023-2025)

- **Gao et al. (2023)** — *Retrieval-Augmented Generation for Large Language Models: A Survey (Naive/Advanced/Modular)*. <https://arxiv.org/abs/2312.10997>
- **Gupta et al. (2024)** — *A Comprehensive Survey of Retrieval-Augmented Generation (RAG)*. <https://doi.org/10.48550/arxiv.2410.12837>
- **Fan et al. (2024)** — *A Survey on RAG Meeting LLMs*. <https://arxiv.org/abs/2405.06211>
- **Zhao et al. (2024)** — *A Survey on Retrieval-Augmented Text Generation for Large Language Models*. <https://arxiv.org/abs/2404.10981>
- **Gao et al. (2022)** — *Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE)*. <https://arxiv.org/abs/2212.10496>
- **Edge et al. (2024)** — *From Local to Global: A Graph RAG Approach to Query-Focused Summarization (GraphRAG)*. <https://arxiv.org/abs/2404.16130>
- **Asai et al. (2023)** — *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*. <https://arxiv.org/abs/2310.11511>
- **Jiang et al. (2023)** — *Active Retrieval Augmented Generation (FLARE)*. <https://arxiv.org/abs/2305.06983>
- **Trivedi et al. (2023)** — *Interleaving Retrieval with Chain-of-Thought Reasoning (IRCoT)*. <https://arxiv.org/abs/2212.10509>
- **Zhang et al. (2024)** — *RAFT: Adapting Language Model to Domain Specific RAG*. <https://arxiv.org/abs/2403.10131>
- **Sarathi et al. (2024)** — *RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval*. <https://arxiv.org/abs/2401.18059>
- **Li et al. (2024)** — *Long Context vs. RAG / Self-Route (RAG-LC comparação e roteamento)*. <https://arxiv.org/abs/2406.18621>
- **Revisão sistemática (2025)** — *A Systematic Literature Review of Retrieval-Augmented Generation*. <https://www.mdpi.com/2504-2289/9/12/320>
- **Mombaerts et al. (2024)** — *Data-centric RAG workflow (referido no survey 2410.12837)*.

Alinhamento e produtos

- **Christiano et al. (2017)** — *Deep Reinforcement Learning from Human Preferences (origem do RLHF)*. <https://arxiv.org/abs/1706.03741>
- **Ouyang et al. (2022)** — *Training language models to follow instructions with human feedback (InstructGPT/RLHF)*. <https://arxiv.org/abs/2203.02155>
- **Wei et al. (2022)** — *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. <https://arxiv.org/abs/2201.11903>

- **Thoppilan et al. (2022)** — *LaMDA: Language Models for Dialog Applications*. <https://arxiv.org/abs/2201.08239>
- **OpenAI (2022-11-30)** — *Introducing ChatGPT*. <https://openai.com/blog/chatgpt>
- **Touvron et al. (2023)** — *LLaMA: Open and Efficient Foundation Language Models*. <https://arxiv.org/abs/2302.13971>
- **OpenAI (2023)** — *GPT-4 Technical Report*. <https://arxiv.org/abs/2303.08774>
- **Hu et al. (2021)** — *LoRA: Low-Rank Adaptation of Large Language Models*. <https://arxiv.org/abs/2106.09685>

Tool use — pesquisa

- **Karpas et al. (2022)** — *MRKL Systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discretized reasoning*. <https://arxiv.org/abs/2205.00445>
- **Yao et al. (2022)** — *ReAct: Synergizing Reasoning and Acting in Language Models*. <https://arxiv.org/abs/2210.03629>
- **Gao et al. (2022)** — *PAL: Program-Aided Language Models*. <https://arxiv.org/abs/2211.10435>
- **Schick et al. (2023)** — *Toolformer: Language Models Can Teach Themselves to Use Tools*. <https://arxiv.org/abs/2302.04761> (NeurIPS 2023)
- **Shen et al. (2023)** — *HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face*. <https://arxiv.org/abs/2303.17580>
- **Patil et al. (2023)** — *Gorilla: Large Language Model Connected with Massive APIs*. <https://arxiv.org/abs/2305.15334>
- **Shinn et al. (2023)** — *Reflexion: Language Agents with Verbal Reinforcement Learning*. <https://arxiv.org/abs/2303.11366>
- **Madaan et al. (2023)** — *Self-Refine: Iterative Refinement with Self-Feedback*. <https://arxiv.org/abs/2303.17651>
- **Schick, Dwivedi-Yu et al. (2023)** — detalhes do Toolformer (calculadora, search, QA, tradutor, calendário). https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf
- **The Evolution of Tool Use in LLM Agents (2026)** — *From Single-Tool Call to Multi-Tool Orchestration* (survey). <https://arxiv.org/abs/2603.22862>

Tool use — indústria e APIs

- **OpenAI (2023-03-23)** — *ChatGPT plugins*. <https://openai.com/blog/chatgpt-plugins>
- **OpenAI (2023-06-13)** — *Function calling and other API updates*. <https://openai.com/blog/function-calling-and-other-api-updates>
- **OpenAI (2023-11-06)** — *DevDay: New models, parallel function calling, JSON mode, Assistants API*. <https://openai.com/blog/new-models-and-developer-products-announced-at-devday-2023>
- **Anthropic (2023-11-21)** — *Claude 2.1: 200K context, tool use beta*. <https://www.anthropic.com/news/claude-2-1>
- **Google (2023-12-13)** — *Gemini API function calling*.
- **OpenAI (2024-08-06)** — *Introducing Structured Outputs in the API*. <https://openai.com/index/introducing-structured-outputs-in-the-api/>
- **Anthropic (2024-10-22)** — *Introducing computer use* (Claude 3.5 Sonnet/Haiku). <https://www.anthropic.com/news/3-5-models-and-computer-use>
- **Berkeley Function-Calling Leaderboard (2024)** — Gorilla group. <https://gorilla.cs.berkeley.edu/leaderboard.html>
- **Yao et al. (2024)** — *tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. <https://arxiv.org/abs/2406.12045>

Agentes

- **Richards (2023)** — *AutoGPT*. <https://github.com/Significant-Gravitas/AutoGPT>
- **Nakajima (2023)** — *BabyAGI*. <https://github.com/yoheinakajima/babyagi>
- **Li et al. (2023)** — *CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society*. <https://arxiv.org/abs/2303.17760>
- **Park et al. (2023)** — *Generative Agents: Interactive Simulacra of Human Behavior*. <https://arxiv.org/abs/2304.03442>
- **Qian et al. (2023)** — *ChatDev: Communicative Agents for Software Development*. <https://arxiv.org/abs/2307.07924>
- **Hong et al. (2023)** — *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework*. <https://arxiv.org/abs/2308.00352>
- **Wu et al. (2023)** — *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. <https://arxiv.org/abs/2308.08155>
- **Chen et al. (2023)** — *AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors*. <https://arxiv.org/abs/2308.10848>
- **Liu et al. (2023)** — *AgentBench: Evaluating LLMs as Agents*. <https://arxiv.org/abs/2308.03688>
- **Wang et al. (2023/2024)** — *A Survey on Large Language Model Based Autonomous Agents*. <https://arxiv.org/abs/2308.11432> (Frontiers of Computer Science, 2024)
- **Xi et al. (2023)** — *The Rise and Potential of Large Language Model Based Agents: A Survey*. <https://arxiv.org/abs/2309.07864>
- **Yao et al. (2023)** — *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. <https://arxiv.org/abs/2305.10601>
- **Xu et al. (2023)** — *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*. <https://arxiv.org/abs/2305.18323>
- **Zhou et al. (2023/2024)** — *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models (LATS; ICML 2024)*. <https://arxiv.org/abs/2310.04406>
- **Masterman et al. (2024)** — *The Landscape of Emerging AI Agent Architectures*. <https://arxiv.org/abs/2404.11584>
- **Guo et al. (2024)** — *Large Language Model Based Multi-agents: A Survey of Progress and Challenges (IJCAI 2024)*. <https://www.ijcai.org/proceedings/2024/890>
- **Li et al. (2024)** — *A Review of Prominent Paradigms for LLM-Based Agents: Tool Use, Planning, and Feedback Learning*. <https://arxiv.org/abs/2406.05804>
- **Agentic AI Frameworks (2025)** — *Architectures, Protocols, and Design Challenges (CrewAI, LangGraph, AutoGen, Semantic Kernel, Agno, ADK, MetaGPT)*. <https://arxiv.org/abs/2508.10146>
- **From Language to Action (2025)** — *A Review of Large Language Models as Autonomous Agents and Tool Users*. <https://arxiv.org/abs/2508.17281>

Protocolos e interoperabilidade

- **Anthropic (2024-11-25)** — *Introducing the Model Context Protocol*. <https://www.anthropic.com/news/model-context-protocol>
- **Anthropic (2025-12-09)** — *Donating the Model Context Protocol and establishing the Agentic AI Foundation* (MCP na Linux Foundation; 97M+ downloads/mês, 10.000+ servidores). <https://www.anthropic.com/news/donating-the-model-context-protocol-and-establishing-of-the-agentic-ai-foundation>
- **MCP Specification** — revisões 2024-11-05, 2025-03-26, 2025-06-18, 2025-11-25. <https://modelcontextprotocol.io/>
- **Google (2025-04-09)** — *Announcing the Agent2Agent Protocol (A2A)*. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>
- **Google Cloud (2025-06-23)** — *Google Cloud donates A2A to the Linux Foundation* (projeto A2A com AWS, Cisco, Microsoft, Salesforce, SAP, ServiceNow; 100+ empresas). <https://developers.googleblog.com/google-cloud-donates-a2a-to-linux-foundation/>
- **Google Cloud Blog (2025-07-31)** — *Agent2Agent protocol (A2A) is getting an upgrade (v0.3: gRPC, security cards assinados, 150+ organizações)*. <https://cloud.google.com/blog/products/ai-machine-learning/agent2agent-protocol-is-getting-an-upgrade>

- **MIT Technology Review (2025-08-04)** — *These protocols will help AI agents navigate our messy lives.* <https://www.technologyreview.com/2025/08/04/1120996/protocols-help-agents-navigate-lives-mcp-a2a/>
- **InfoQ (2025-07-01)** — *Google's Agent2Agent Protocol Enters the Linux Foundation.* <https://www.infoq.com/news/2025/07/agent2agent-linux-foundation/>
- **InfoWorld (2025-08-01)** — *Google upgrades A2A with gRPC and enterprise-grade security.* <https://www.infoworld.com/article/4032776/google-upgrades-agent2agent-protocol-with-grpc-and-enterprise-grade-security.html>
- **VentureBeat (2025-06-03)** — *AI's big interoperability moment: Why A2A and MCP are key.* <https://venturebeat.com/technology/ais-big-interoperability-moment-why-a2a-and-mcp-are-key-for-agent-collaboration>
- **AWS Open Source Blog (2025-05-20)** — *Open Protocols for Agent Interoperability Part 1.* <https://aws.amazon.com/blogs/opensource/open-protocols-for-agent-interoperability-part-1-inter-agent-communication-on-mcp/>
- **InfoWorld (2025-08-27)** — *Beyond AI protocols: Preparing for MCP and A2A in production.* <https://www.infoworld.com/article/4046484/beyond-ai-protocols-preparing-for-mcp-and-a2a-in-production.html>
- **Konishi (2026-07-05)** — *Tool Use and Agent Protocol History and Timeline — Function Calling, MCP, and A2A.* https://hidekazu-konishi.com/entry/tool_use_and_agent_protocol_history_and_timeline.html
- **Taskade (2026-08-02)** — *How LLMs Got Hands: A History of Tool Use.* <https://www.taskade.com/blog/tool-use-history>

Visões de conjunto (história e survey de LLMs)

- **Zhao et al. (2023)** — *A Survey of Large Language Models.* <https://arxiv.org/abs/2303.18223>
- **Springer (2025)** — *History of Large Language Models* (capítulo). https://link.springer.com/chapter/10.1007/978-3-031-92285-5_2
- **arXiv (2024)** — *History, Development, and Principles of Large Language Models* (revisão acessível). <https://arxiv.org/abs/2402.06853>
- **MDPI (2023)** — *From Turing to Transformers: A Comprehensive Review and Tutorial.* <https://www.mdpi.com/2413-4155/5/4/46>
- **ACM (2024)** — *A Comprehensive Overview of Large Language Models.* <https://dl.acm.org/doi/10.1145/3744746>
- **Ding (2022)** — *From Transformers to ChatGPT* (nota de progresso 2017-2022). <https://www.dingran.me/from-transformer-to-llm/>

Critério de inclusão e verificação

1. **Preferência por fontes primárias:** papers originais (arXiv/proceedings) e anúncios oficiais.
2. **Verificação cruzada:** datas de produtos conferidas contra ao menos duas fontes quando possível (ex.: function calling — OpenAI + Taskade + Konishi).
3. **Datas:** artigos usam a primeira publicação pública; divergências são registradas na cronologia.md.
4. **Atualização:** o campo evolui rápido; fontes com data de 2025-2026 devem ser revistas a cada nova sessão.

Anexo D — NOTAS: manutenção do livro

NOTAS — Manutenção do livro

Instruções operacionais para futuras sessões que trabalhem neste livro. Leia antes de editar.

Como este livro é gerido

- O livro vive em `livro/`, em Markdown, organizado por eras cronológicas.
- `README.md` é a porta de entrada e contém o **estado do livro** (checkbox). Atualize-o ao concluir capítulos.
- `SUMARIO.md` é a fonte da verdade da estrutura. Capítulos novos começam aqui.
- `cronologia.md` é a linha do tempo dos marcos; toda data relevante deve estar registrada lá.
- `fontes.md` é a bibliografia; todo capítulo cita códigos aqui (ex.: [Vaswani et al. 2017]).
- `NOTAS.md` (este arquivo) descreve o workflow.

Formato dos capítulos

Cada capítulo em `capitulos/` segue a mesma estrutura:

```
# N – Título

Abertura em prosa: situa o capítulo na história – o problema que a era enfrentava, o porquê e o
que se ambicionava. (Não há subseções "Objetivo/Contexto/Ideia"; a abertura faz esse papel.)

## <seções temáticas>
O corpo. Cada seção desenvolve uma estrutura ou ideia; jargão técnico é definido na primeira
ocorrência; diagramas ASCII quando ajudam.

## A lição estrutural
Seção final do corpo (imediatamente antes do engbox) que sintetiza a herança conceitual da era; obrigatória em todos os capítulos, sempre com este
título.

## Para o engenheiro
Quadro final (engbox) com takeaways práticos: decisões, armadilhas, o que implementar.

---

**Fontes:** [Autor, ano] – descrição curta; [Autor, ano] – ...
```

Regras de escrita

1. **Didático:** explique do zero; não assuma conhecimento prévio de redes neurais além de conceitos básicos.
2. **Verificável:** afirmações factuais citam [Fonte, ano]. Não invente datas ou números.
3. **Estrutural:** ensine a estrutura que se repete, não apenas o caso concreto.
4. **Em português:** o livro é em português; termos técnicos podem manter o original (ex.: *few-shot*, *grounding*).
5. **Sem jargão não definido:** a primeira ocorrência de um termo técnico recebe definição.
6. **Diagramas:** prefira diagramas ASCII simples (o estilo usado em `AGENTS.md`).

Workflow de escrita

1. Ao iniciar uma sessão, leia `README.md` (estado) e `SUMARIO.md` (estrutura).
2. Escolha um capítulo pendente (mais antigo primeiro).
3. Antes de escrever, consulte `cronologia.md` e `fontes.md` para o período.

4. Escreva o capítulo respeitando o formato acima.
5. Atualize README.md (estado) e marque o capítulo como feito.
6. Se descobrir fontes novas, registre-as em fontes.md e a data em cronologia.md.

Verificação de fatos

- Datas de papers: usar primeira publicação (arXiv). Conferir se a URL está acessível.
- Datas de produtos: anúncio oficial; cruzar com ao menos uma segunda fonte.
- Não afirmar números de parâmetros/tokens sem fonte.
- Se uma data divergir entre fontes, registrar a divergência e escolher a mais confiável.

Contexto e ferramentas

- Use web search para atualizar o período recente (2025-2026) — o campo muda rápido.
- Sempre que um número ou data for crítico para a narrativa, verifique-o antes de publicar no capítulo.
- Capítulos ainda não escritos podem conter apenas o cabeçalho e a seção "Fontes" como esqueleto.

Próximos passos sugeridos

1. Revisão cruzada de datas e fontes: conferir cronologia.md contra fontes.md e as citações dos capítulos.
2. Consolidação e formatação final do livro (a revisão estilística geral já foi feita).